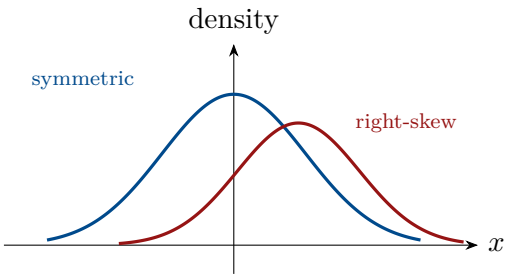
Exploratory Data Analysis

A Book for Absolute Beginners

One idea, one picture, one hand calculation — then the library call

No prior stats assumed. The class board is the spine. The studios are the muscle.



Prepared by

Krushna Parmar

Roll No. **202611032** • Course IT462

Grounded in the IT462 lecture boards (23 Jul – 20 Aug 2026). Pedagogy borrowed (rewritten, never pasted) from Berkeley Data 8 / Data 100, Think Stats 3, OpenIntro, and Stanford STATS 60: look, then compress, then code from scratch, then check the library. Polars, DuckDB, and LLM-EDA live in the appendix, not in the course.

Contents

How to Read This Book **vii**

I Data **1**

1 The Two Tables We Will Not Leave **3**
1.1 Why a running table beats a new mystery every page . . . 3
1.2 File 1 — the dirty five . . . 4
1.3 File 2 — thirty students . . . 4
1.4 Toy lists from the board, kept forever . . . 5

2 What Did Averaging Hide? **7**
2.1 Why the tools are a posture, not a shopping list . . . 7
2.2 Four numbers on the board are not yet a claim . . . 8
2.3 The same list, compressed three ways . . . 8
2.4 A loop first, then the library, then an assert . . . 9

3 Who Handed You This Rectangle? **13**
3.1 When is a folder not yet a table? . . . 13
3.2 One story, three distances from the room . . . 14
3.3 What a comma is doing in our file . . . 14

4 Can You Use These Five Rows? **17**
4.1 Usable is four words, not a feeling . . . 17
4.2 Five rows, walked in order . . . 18
4.3 Five questions that are not a cleanup . . . 19

5 Why Did Age Zero Send Us Back? **21**
5.1 Three boxes, and the arrow people skip . . . 21
5.2 What a column is, and what a row is . . . 22
5.3 The first fork: label versus quantity . . . 23

II Measurement scales **25**

6 Why Can't You Average a Colour? **27**
6.1 A potato, used once . . . 27
6.2 Twelve columns, one decision each . . . 27
6.3 The fourth label: binomial (binary) features . . . 29
6.4 Why a Likert bar is not a histogram . . . 29
6.5 Mean 3.4 or median 3, on the same five ticks . . . 30

7 When Is Twice Allowed? 33
7.1 Four ticks, accumulated left to right . . . 33
7.2 Age is ratio, even if no newborn sits in class . . . 33
7.3 Twenty degrees is not twice ten — same pair, two rulers . . . 34
7.4 Which verbs are legal? . . . 35
7.5 Descriptive, inferential, and five families with no formulas yet . . . 36

III Univariate summaries 39

8 Why One Pin Is Never Enough 41
8.1 What if every row pulls equally? . . . 41
8.2 What if the story is a product, not a sum? . . . 43
8.3 What if we averaged rates? . . . 43
8.4 Do the three pins even agree? . . . 44
8.5 What if we refuse the extremes? . . . 45
8.6 What if some rows should count more? . . . 45

9 The Middle After a Sort, and the Name That Won 47
9.1 Where is the middle once the line is sorted? . . . 47
9.2 Which value showed up most? . . . 49
9.3 Which pin is legal for this column? . . . 50

10 How Wide Is the Pile? 53
10.1 What do the two ends already tell you? . . . 53
10.2 Why square the red sticks? . . . 54
10.3 Why $n - 1$? One degree of freedom already spent . . . 55
10.4 What width survives a wild row? . . . 56

11 Where Do You Sit in the Sorted Line? 59
11.1 How does the board number a seat? . . . 59
11.2 What does numpy do to the same ten numbers? . . . 61

12 When the Pile Leans, Peaks, or Goes Flat 63
12.1 Which way does the pile lean? . . . 63
12.2 Is every symmetric pile a bell? . . . 65
12.3 Are the tails heavier than a bell's? . . . 66

IV Probability distributions 69

13 Probability Distributions, and How to Compute Them in Python 71
13.1 The four functions every distribution has . . . 71
13.2 Discrete distributions (counts) . . . 72
13.2.1 Bernoulli — one yes/no . . . 72
13.2.2 Binomial — successes in n trials . . . 72
13.2.3 Poisson — counts of rare events . . . 72
13.3 Continuous distributions (measurements) . . . 73
13.3.1 Uniform — every value equally likely . . . 73
13.3.2 Normal (Gaussian) — the bell, and the star of EDA . . . 73
13.3.3 Exponential — waiting times . . . 74
13.4 Fitting a named shape to real data . . . 74
13.5 One reference table . . . 75

V Bivariate association **77**

14 When Two Columns Move Together **79**
14.1 Four tiny tables, four different stories . . . 79
14.2 How much straight line is in the cloud? . . . 80
14.3 What if we only believe the order? . . . 82
14.4 What if one column is a yes/no? . . . 82
14.5 What if both columns are names? . . . 83

15 Covariance and Correlation, in Depth **87**
15.1 Covariance from scratch . . . 87
15.2 From covariance to correlation . . . 88
15.3 Many columns at once: the covariance and correlation matrices . . . 88
15.4 Kendall's τ — correlation by counting pairs . . . 89
15.5 The traps: correlation is not causation (and other lies of r) . . . 90

VI From looking to designing **93**

16 Who Did We Observe? Sampling **95**
16.1 N is a parameter. n is a statistic. . . . 95
16.2 Simple random sampling: list every pair . . . 96
16.3 A 12-row toy: cluster is not stratified . . . 97

17 Experiments, Bias, and the Two Questions **101**
17.1 A vaccination story with four labels . . . 101
17.2 The three Rs . . . 102
17.3 Sampling bias is a design failure, not a rude mean . . . 103
17.4 IDA asks "reliable?" EDA asks "patterns?" . . . 103

18 The Import Lab: A One-Page EDA Report **107**
18.1 Where this sits in the course, and how data gets in . . . 107
18.2 Open the file. Do not model. . . . 108
18.3 Half 1 — columns . . . 109
18.4 Half 2 — cells . . . 110
18.5 The one-page report (filled in for the thirty) . . . 111

VII Cleaning and transforming (Insem 2) **113**

19 Missing Data: What It Is and Why It Went Missing **115**
19.1 Definition and causes . . . 115
19.2 The notation on the board . . . 116
19.3 The three mechanisms . . . 117
19.3.1 MCAR — Missing Completely At Random . . . 118
19.3.2 MAR — Missing At Random . . . 119
19.3.3 MNAR — Missing Not At Random . . . 120
19.4 Seeing the three mechanisms on one dataset . . . 120
19.5 Ignorable vs non-ignorable . . . 122

20 Detecting Missingness: Counts, Pictures and MCAR Tests **125**
20.1 Numerical detection . . . 125
20.2 Visual detection: compare observed vs missing groups . . . 126
20.3 The `missingno` package . . . 127
20.4 A two-minute primer on hypothesis tests . . . 131
20.5 Little's MCAR test (overall test) . . . 134
20.6 Dixon's test (univariate) . . . 135
20.7 Hawkins' test (multivariate) . . . 136
20.8 The detection workflow . . . 137

21 Treating Missing Values: Delete, Impute, or Model **139**
21.1 Deletion . . . 139
21.1.1 Complete case analysis (listwise deletion) . . . 140
21.1.2 Available case analysis (pairwise deletion) . . . 140
21.2 Simple imputation . . . 141
21.3 Imputation by a function: regression . . . 142
21.4 Model-based: maximum likelihood (EM) . . . 143
21.5 Matching the fix to the mechanism . . . 143
21.6 The full workflow, from the students' notes . . . 144

22 Outliers: Detection and Treatment **147**
22.1 Definition . . . 147
22.2 Univariate detection: numerical rules . . . 148
22.2.1 The z-score rule . . . 148
22.2.2 The IQR rule (Tukey's fences) . . . 148
22.2.3 A robust z-score (modified z) . . . 149
22.3 Univariate detection: graphical . . . 150
22.4 Multivariate outliers . . . 150
22.5 Treatment . . . 152

23 Data Transformation I: Scaling, Function, Power and Quantile Transforms **155**
23.1 The three stages . . . 155
23.2 Scaling . . . 157
23.2.1 Standardization (z-score) . . . 157
23.2.2 Normalization (min–max) . . . 157
23.2.3 Robust scaling . . . 158
23.3 Function transformation . . . 159
23.4 Power transformation . . . 160
23.4.1 Box–Cox (positive data only) . . . 160
23.4.2 Yeo–Johnson (any real data) . . . 161
23.5 Quantile transformation . . . 161
23.6 Choosing a transform . . . 163

24 Data Transformation II: Binning and Encoding **165**
24.1 Stage II: numeric → category (discretization / binning) . . . 165
24.1.1 Equal-width binning . . . 166
24.1.2 Equal-frequency (quantile) binning . . . 166
24.1.3 Domain-specific binning . . . 166
24.1.4 Cluster-based binning . . . 167
24.2 Stage III: category → numeric (encoding) . . . 167
24.2.1 One-hot encoding . . . 168
24.2.2 Label encoding . . . 169

24.2.3 Ordinal encoding . . . 169
24.2.4 Binary encoding . . . 169
24.2.5 Target (mean) encoding . . . 169
24.2.6 Frequency (count) encoding . . . 170
24.2.7 Hash encoding (feature hashing) . . . 170
24.2.8 Entity embedding . . . 170
24.3 Choosing an encoder . . . 170

25 Insem 2 Revision Kit 173
25.1 What Insem 2 covers . . . 173
25.2 Category 3: the Python functions, in one place . . . 174
25.2.1 The nine distributions on the list . . . 174
25.2.2 "Generate 100 random numbers from an exponential distribution between a and b" . . . 175
25.2.3 "Convert int64 to string" and other type changes . . . 175
25.2.4 Features and values: the checklist functions . . . 175
25.2.5 Sampling, univariate, bivariate in a few lines . . . 176
25.3 Category 1: MCQ bank . . . 176
25.4 Category 2: short-answer bank . . . 178
25.5 Last-night summary . . . 178

A Optional 2026 Toolkit (Not the Course) 181
A.1 Polars and DuckDB — faster rectangles, same NOIR . . . 181
A.2 ydata-profiling — a portrait, not a substitute . . . 182
A.3 LLM-EDA, with an audit trail or not at all . . . 182

B Where Every Page of the Notes Lives in This Book 183
B.1 *sir*'s board notes (Insem 2, 33 pages) . . . 183
B.2 Students' handwritten notes (16 pages) . . . 184
B.3 Corrections made while transcribing . . . 185

C Answers to studios 187
C.1 Chapter 0 — the two tables . . . 187
C.2 Chapter 1 — data vs information . . . 188
C.3 Chapter 2 — sources and files . . . 188
C.4 Chapter 3 — IDA on the dirty five . . . 189
C.5 Chapter 4 — the pipeline . . . 189
C.6 Chapter 5 — nominal and ordinal . . . 190
C.7 Chapter 6 — interval, ratio, operations . . . 190
C.8 Chapter 7 — means . . . 191
C.9 Chapter 8 — median and mode . . . 192
C.10 Chapter 9 — dispersion . . . 192
C.11 Chapter 10 — percentiles . . . 193
C.12 Chapter 11 — shape . . . 193
C.13 Chapter 12 — association . . . 194
C.14 Chapter 13 — sampling . . . 195
C.15 Chapter 14 — experiments . . . 195
C.16 Chapter 15 — the import lab . . . 196
C.17 Missing data: mechanisms (Chapter 19) . . . 197
C.18 Missing data: detection (Chapter 20) . . . 197
C.19 Missing data: treatment (Chapter 21) . . . 198
C.20 Outliers (Chapter 22) . . . 198

C.21 Transformation I (Chapter 23) 198
C.22 Transformation II (Chapter 24) 199

Closing **201**

Bibliography **203**

How to Read This Book

This book assumes **you have never taken a statistics course**. It is the sister of the FOML theory book: same density, a different subject. EDA is not "plot stuff until it looks pretty". It is the habit of *looking* before you *model*.

Every concept is taught with the same eight beats, so you always know what to do next:

1. **From the class board** (orange) — the exact formula or table *sir* wrote.
2. **Worked by hand** (teal) — six to ten numbers, every square shown.
3. **Picture** — a histogram, stick plot, or flowchart of *those* numbers.
4. **Derivation** (blue) — why the formula is that shape, not a name-drop.
5. **Another way** (navy) — a second school, a second `ddof`, a second percentile recipe, on the *same* numbers.
6. **Python** (grey) — a loop first, then `numpy`/`pandas`, then `assert` they match.
7. **Watch Out** (red) — the failure case (zip codes as ratio, Pearson on a boomerang).
8. **Studio** (gold) — three problems. Full solutions live in the answers chapter.

A grey header **Right now we are:** tells you which of five jobs you are doing (collect, manage, explore, communicate — we skip *predict*; that is FOML / CDA).

Two tables, used everywhere. There are no mystery CSVs. Chapter 0 introduces `it462_dirty.csv` (five dirty rows from the board) and `it462_marks.csv` (thirty invented students: marks, age, gender, Likert, city, hours). Every later chapter recomputes something on one of those two files.

What this book is not. It is not a survey of 2026 tooling, not a probability course, and not a pirate copy of Tukey, Wickham, Data 8, or OpenIntro. The lecture boards are the spine. The surrounding worked examples are the muscle. After the last studio you can classify a column on sight, pick a legal summary, refuse a fake correlation, enumerate a tiny sample space, and write the code.

Do not skip Chapter 0 and Chapter 1. The words *feature*, *observation*, *nominal*, *usable* are defined there and then reused without apology.

Part I

Data

Chapter 1

The Two Tables We Will Not Leave

Right now we are: collect — name the data this course will actually use

This chapter has no new board formula. It exists so that later chapters never say "consider some dataset" and wave. Two files live in `theory-book/data/` and again in `notebooks/labs/data/`. Every mean, every bar chart, every association coefficient in this book is computed on one of them, or on a toy list of four or ten numbers copied from the lecture board.

1.1 Why a running table beats a new mystery every page

Berkeley Data 8 keeps *Little Women* and a babies table around for weeks. Think Stats 3 keeps the NSFG. The point is not the subject matter. The point is that your brain stops spending energy on "what is this column?" and starts spending it on "what did the mean just hide?"

IT462 is not a novel. We invent a class list of thirty students and we keep it.

In Plain Words

A summary number is a compression. You cannot judge a compression unless you have already seen the uncompressed thing. That is why the histogram of `it462_marks.csv` appears before the formula for $\bar{x}$, not after.

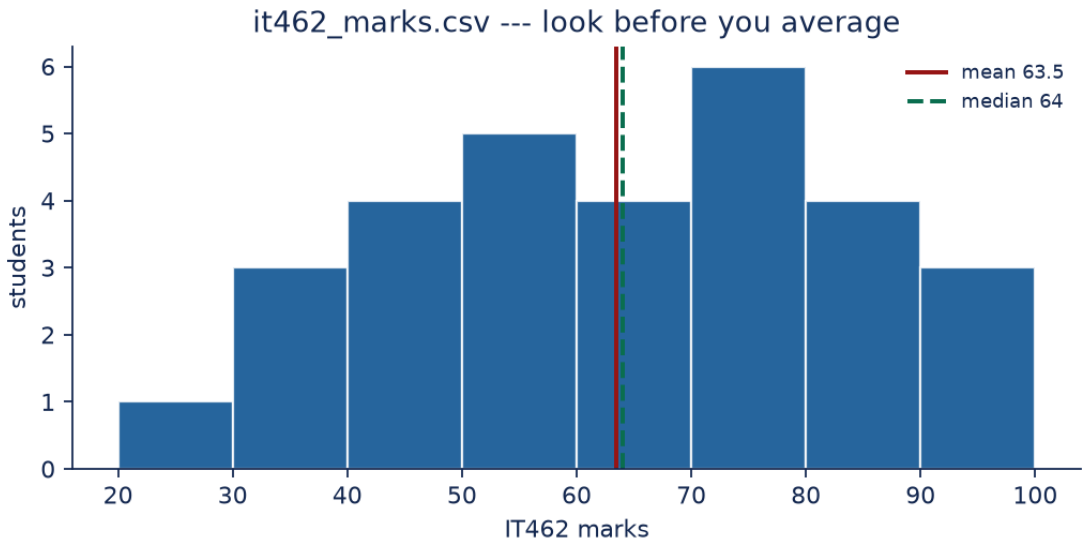


Figure — Thirty IT462 marks. Mean 63.5, median 64. Almost symmetric — which is why this file is a fair playground, not a gotcha.

1.2 File 1 — the dirty five

From the class board

A table with columns ID, IT462, Age, Gender. Five rows. One ID is `2026.` with a trailing period. Two IDs are the integer `2026`. One ID is blank. One score is −20. Ages include 0 and 100. Gender and age go missing on the last row.

That table is `it462_dirty.csv`. It is deliberately short so you can IDA it with a pencil.

ID	IT462	Age	Gender
`2026.`	10	0	M
2026	50	10	F
2026	90	20	M
(blank)	−20	100	F
2023	17	(blank)	(blank)

You will walk every cell in Chapter 3. Do not clean it yet. Look first.

1.3 File 2 — thirty students

`it462_marks.csv` is invented. The columns are the ones the later lectures need: a ratio mark, a ratio age, a nominal gender, an ordinal Likert, a nominal city, a ratio `hours_studied`, and a binary `passed` (yes if marks ≥ 40).

id	marks	age	gender	Likert	city	hours
S01	74	19	F	4	Ahmedabad	6
S02	45	20	M	3	Surat	3
S03	88	18	F	5	Ahmedabad	9
S04	61	21	M	3	Vadodara	5
S05	39	19	F	2	Rajkot	2
S06	94	22	M	5	Ahmedabad	10
S07	55	20	F	3	Surat	4
S08	70	18	M	4	Vadodara	6
S09	28	23	M	1	Rajkot	1
S10	81	19	F	5	Ahmedabad	8

First ten of thirty. Full file: `theory-book/data/it462_marks.csv`.

Sanity numbers you should memorize

$n = 30$. Marks sum to 1905, so

$$\bar{x}_{\text{marks}} = \frac{1905}{30} = 63.5.$$

Sorted marks (you will need this for the median and for percentiles):

$$28, 31, 35, 39, 41, 42, 45, 47, 50, 53,$$
$$55, 58, 59, 61, 62, 66, 68, 70, 73, 74,$$
$$74, 76, 77, 81, 84, 85, 88, 90, 94, 99.$$

Positions 15 and 16 are 62 and 66, so the median is 64. Gender is balanced (15 F, 15 M). Four students fail (`passed = no`): S05 (39), S09 (28), S18 (35), S29 (31). Mean age is exactly 20.

The same thirty rows also exist as `it462_marks.json`, so Chapter 2 can load the identical table two ways.

How to write it in Python

```python
import pandas as pd
df = pd.read_csv("it462_marks.csv")
assert df.shape == (30, 8)
assert df["marks"].sum() == 1905
assert df["marks"].mean() == 63.5
```

If those asserts fire, you opened the wrong file. There is no third dataset in this book.

1.4 Toy lists from the board, kept forever

Three short lists are copied from the lecture boards and reused whenever a hand calculation should fit in a margin:

- L01 scores: $\{17, 41, 30, 99\}$. Exact mean 46.75; the board said "about 50".
- L07 toy: $\{1, 2, 4, 6\}$. AM / GM / HM live here.
- L07 percentiles: $2, 3, 5, 6, 8, 10, 12, 15, 18, 20$ with $n = 10$.

Watch Out

Do not mix the lists. The mean of the thirty marks is 63.5. The mean of $\{17, 41, 30, 99\}$ is 46.75. If a viva answer jumps lists, it is wrong even if the algebra is pretty.

Studio — now you try

0.1 Without a calculator, confirm that the thirty marks sum to 1905. (Hint: add the ten numbers in each block of the sorted list, then add the three block-sums.)
0.2 A classmate "helps" by sending you a file named `it462_marks.csv` whose `marks` mean is 71. What is the first check you run, and why is "the mean looks nicer" not a reason to keep their file?
0.3 Which column of `it462_marks.csv` is a *target* if the question is "did the student pass?", and which columns are then *features*? (You will need this split in the L10 lab.)

Chapter 2

What Did Averaging Hide?

Right now we are: explore — compress four board scores three ways and name what each way throws away

This chapter has one job. You will take the four numbers *sir* wrote on the board, compress them three times, and refuse to pretend the three compressions are the same sentence. Later chapters will give you better compressions. None of them is allowed to start until you can see what a mean deletes.

2.1 Why the tools are a posture, not a shopping list

From the class board

IT462 — EDA. Tools: an **open mind**, a **closed heart**. Python on Google Colab. `numpy`. `pandas`.

An **open mind** lets the table surprise you. If the four scores contain a 99 sitting next to a 17, you do not decide in advance that "the class was average". A **closed heart** refuses the story you brought from home. You do not fall in love with "this must have been a hard paper" and then hunt for a number that agrees.

Colab is a browser tab that already has Python. You are not installing a career. You are borrowing a calculator that can loop. `numpy` is the fast array. `pandas` is the labelled rectangle. Every notebook this semester starts with those two imports. The rest of the 2026 catalogue — faster tables, query engines, LLM-EDA — waits in the appendix until the board's habits are in your hands.

In Plain Words

Data is the pile you recorded. Information is a sentence you can act on after an honest compression. A mean is one such sentence. It is not the pile.

How to write it in Python

```
import numpy as np
import pandas as pd
print(np.__version__, pd.__version__)
```

If this cell fails, you are not ready to compute a mean. Fix the kernel first. The algebra below does not care which laptop you used.

2.2 Four numbers on the board are not yet a claim

From the class board

Scores: 17, 41, 30, 99. Circled: "scores with AVG" → information ≈ 50.

Those four marks are **data**. They do not yet say how the quiz went. They are a list. The board's move — replace the list by one number — is the first compression of the course. The circled 50 is **information**: short, useful, and already slightly dishonest, because the exact mean is not 50.

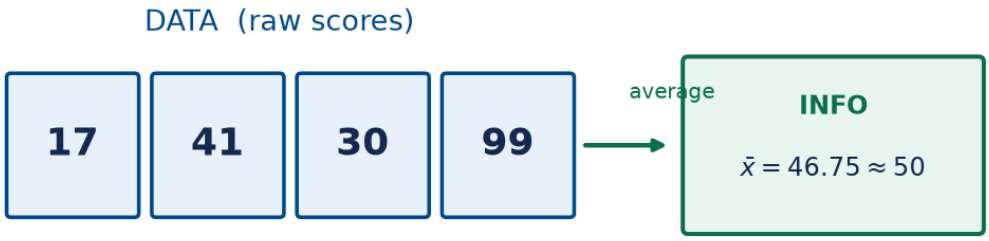


Figure — Left: the four scores as recorded. Right: one number the board was willing to say out loud. The arrow is a compression, not a discovery.

Derivation — step by step

The arithmetic mean of n numbers $x_1, \ldots, x_n$ is

$$\bar{x} \;=\; \frac{1}{n}\sum_{i=1}^{n} x_i.$$

You add. You divide by how many you added. You do not sort. Plug in the board:

$$17 + 41 + 30 + 99 \;=\; 187, \qquad \bar{x} \;=\; \frac{187}{4} \;=\; 46.75.$$

The board wrote 50. That is rounding to a number a human will remember, not a second formula. In a viva, say both: exact mean 46.75; board sketch 50.

Why divide by n? The sum 187 grows if you duplicate the class. Dividing by 4 makes "average score" comparable to a class of forty. The mean is a per-person sentence. The sum is a pile sentence. They answer different questions.

2.3 The same list, compressed three ways

A mean is not the only compression. On these four numbers we will always keep three.

x_1	x_2	x_3	x_4
17	41	30	99

Three compressions of $\{17, 41, 30, 99\}$

Mean. No sort.

$$\bar{x} = \frac{187}{4} = 46.75 \quad \text{(board said} \approx 50\text{)}.$$

Median. Sort first: 17, 30, 41, 99. Even $n = 4$, so average the two centre positions (2 and 3):

$$m = \frac{30 + 41}{2} = \frac{71}{2} = 35.5.$$

The picture with nothing hidden. Four bars, one per score, heights $17, 41, 30, 99$. No number has been dissolved. That picture is also a compression — it threw away names and times — but it kept the 99 visible as a bar, not as a tug on a decimal.

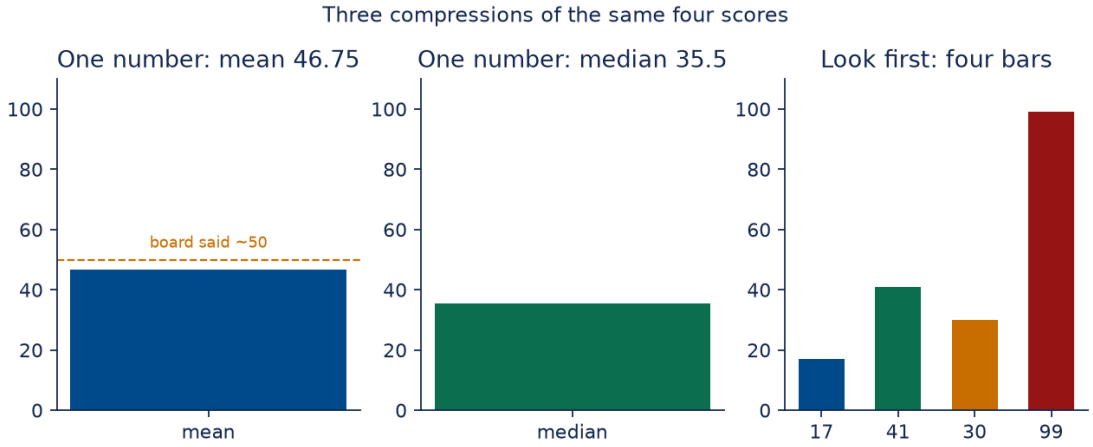


Figure — The same four scores. Left: mean pin at 46.75 *(board* 50*). Middle: median pin at* 35.5*. Right: four bars. Only the bars still show that one student was a* 99*.*

Drop the 99, same recipe

The other three scores sum to $17 + 41 + 30 = 88$, so

$$\bar{x}_{\text{without 99}} = \frac{88}{3} = 29.\overline{3}.$$

The four-number mean was 46.75. The 99 pulled the pin by

$$46.75 - \frac{88}{3} = \frac{187}{4} - \frac{88}{3} = \frac{561 - 352}{12} = \frac{209}{12} \approx 17.42$$

points. Three of the four original scores (17, 30, 41) sit *below* 46.75. The mean is not "the typical student". It is the centre of mass, and the 99 has mass. The median did not move as violently: the sorted middle pair without 99 is not even defined the same way, but on the four-number list the median 35.5 still lives next to 30 and 41, where most of the list actually is.

That is what averaging hid. Not a conspiracy. A property. Every later mean in this book is the same kind of pin. You are required to ask what sat on the far side of it.

2.4 A loop first, then the library, then an assert

The formula is a loop. If you cannot write the loop, you do not yet own the formula.

How to write it in Python

```
xs = [17, 41, 30, 99]
total = 0
n = 0
for x in xs:
    total = total + x
    n = n + 1
mean_loop = total / n

import numpy as np
assert mean_loop == 46.75
assert mean_loop == np.mean(xs)
assert mean_loop == np.mean(np.array(xs))
```

The asserts are the course's honesty check. A pretty `np.mean` you cannot rebuild with a running total is a button, not a method. When a later chapter changes n, you will change the loop first.

pandas will do the same job on a labelled column. That is not a different mean. It is the same sum, the same n, a different container.

How to write it in Python

```
s = pd.Series([17, 41, 30, 99], name="board")
assert s.mean() == 46.75
assert s.median() == 35.5
```

Watch Out

Do not average a column just because it is written with digits. The IDs in `it462_dirty.csv` look like 2026. Their mean is not a year of birth and not a class statistic. It is a noise number that pandas will happily print. The failure case of this chapter is not "the mean formula was wrong". It is "you compressed something that was not a quantity". Chapter 5 will give that mistake a name (nominal). For today: if you cannot say out loud what $\bar{x}$ is a typical *amount of*, do not compute it.

The thirty-row file is waiting, but it is a different list. Its marks mean is 63.5. If a viva answer swaps 46.75 and 63.5, the algebra can be perfect and the answer is still wrong. Four scores from the board. Thirty scores from `it462_marks.csv`. Do not mix them.

Questions & Variations

Q. Why did the board write 50 if the mean is 46.75?
A. Because 50 is a sketch, not a second definition. State the exact value, then the sketch.

Q. Three students scored below the mean. Is the mean wrong?
A. No. The mean is doing what means do: the 99 pulls it. That is a reason to also report the median and the bars, not a reason to "correct" 46.75.

Q. Is the four-bar picture information or data?
A. It is still a compression — you chose an axis and a height — but it is a compression that keeps one bar per recorded fact. The mean keeps none.

Studio — now you try

1.1 Without a calculator, drop the 99 and compute the mean of $\{17, 41, 30\}$. Then compute how many points the 99 pulled the four-number mean (exact fraction, then a decimal). Do not use `it462_marks.csv`.

1.2 You must send one compression of $\{17, 41, 30, 99\}$ to three different people. (a) A parent who asked "how did a typical student do?" (b) A dean who asked "was anyone far from the pack?" (c) A ranking formula that is only allowed one number per quiz. Which compression do you send each person, and what does that choice hide?

1.3 Load `it462_marks.csv`. Write asserts that $n = 30$ and that the `marks` mean is 63.5. In the same notebook, compute the mean of $\{17, 41, 30, 99\}$ from a Python list. Write one sentence explaining why those two means must never be reported as if they came from the same class.

Chapter 3

Who Handed You This Rectangle?

Right now we are: collect — name the pile, name the source, then open the file as a grid

Chapter 1 compressed four numbers that were already sitting on a board. Real work starts one step earlier: stuff in the world is not a grid. This chapter is how a mess becomes a rectangle you are allowed to average, and how you record *who* built that rectangle so you do not later pretend you did.

3.1 When is a folder not yet a table?

From the class board

Data: raw, unorganized / unstructured. Examples pointing at information: text, audio, visuals, records. Structured = a grid (rows × columns).

Unstructured data has no pre-agreed slots. A lecture recording, a zip of ID-card photos, a WhatsApp export, a PDF of handwritten marks — you cannot type `df["marks"].mean()` because there is no `marks` column. **Structured** data has been poured into a rectangle: every row is one entity, every column is one property of that entity.

The four examples on the board are four *kinds of pile*, not four file extensions. Text is words in order. Audio is a pressure wave over time. Visuals are pixels (or vectors) in a plane. Records are already half-way to a grid: a register, a logbook, a roll sheet. EDA in IT462 lives on the rectangle. Getting there — parsing, labelling, typing — is already analysis. It is not a prelude you skip.

In Plain Words

A grid is a decision. Someone chose what a row means and what a column means. You inherit those decisions when you inherit the file.

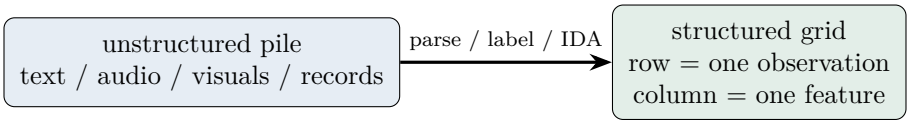


Figure — EDA starts to look like a class once the pile has been forced into a grid. The forcing is already a claim about the world.

3.2 One story, three distances from the room

From the class board

Source → Primary ("I am involved") / Secondary ("you are involved") / Tertiary.

Keep a single story in your head for the whole semester, not three unrelated slogans.

You want the IT462 marks for thirty students.

If *you* sat in class, called the roll, and typed the thirty rows, the file is **primary**. "I am involved." You know that S09 looked ill and that S22 retake was on a different day. The bruises are yours.

If you walk to the DA-IICT academic office and they export the same thirty names from their system, the file is **secondary**. "You are involved" — *they* collected it for *their* question (transcripts, eligibility, a statutory report). You inherit their bruises without their afternoon. Secondary is not a moral insult. Almost every class dataset is secondary. The sin is pretending you collected it.

If you later read a Wikipedia table titled "average engineering marks in Gujarat" and treat that one cell as if it were S01–S30, the number is **tertiary**. Someone already compressed a pile you cannot see. You are two rooms away from the students.

Watch Out

Tertiary is not forbidden. It is a different object. You may quote a published mean. You may not then compute a median, a histogram, or a correlation from that mean as if the rows were still there. Compression does not reverse.

Write the source next to the file, not in your memory. Chapter 3 will need it when an age of 0 appears and you have to decide whether you are allowed to ask the collector.

3.3 What a comma is doing in our file

A **CSV** is a text file that pretends commas are columns and newlines are rows. That is the whole format. Open `it462_marks.csv` as text — not yet as pandas — and look.

The first line is not a student. It is a header:

```
student_id,marks,age,gender,likert,city,hours_studied,passed
```

Count the commas: seven. Count the fields: eight. A comma *separates*. A newline *ends a record*. The next line is the first observation:

```
S01,74,19,F,4,Ahmedabad,6,yes
```

Eight fields again. `Ahmedabad` stays one field because it contains no comma. If a city were written `Gandhinagar, GEC` without quotes, the row would suddenly have nine fields and every column to the right would shift. That is not a statistics bug. That is the format keeping a promise you broke.

The first record, two spellings

CSV (one line, commas): `S01,74,19,F,4,Ahmedabad,6,yes`
JSON stores the same eight facts as keys, not as positions. Different punctuation. The file `theory-book/data/it462_marks.json` is the same thirty rows, not a second class.

How to write it in Python

```
{"student_id": "S01", "marks": 74, "age": 19, "gender": "F",
 "likert": 4, "city": "Ahmedabad", "hours_studied": 6, "passed": "
    yes"}
```

That object is row 0 of both loaders below.

From the class board

Data → file formats: `.csv`, `.xlsx`. Branch: `.json`, `.html`, webscraping, Python. Separate oval: text.

In Plain Words

`.csv` is the default rectangle-as-text. `.xlsx` is Excel's zipped XML workbook — useful when a human already lives in sheets, annoying when you wanted a diff. `.json` is nested key-value boxes; the internet speaks it. `.html` is a page; scraping is reading its tables with code. **Python** is on the board because the skill is the parser, not the file suffix. **Text** is the oval off to the side: a novel, a statute, a chat log — unstructured on purpose.

How to write it in Python

```
import pandas as pd

csv_df = pd.read_csv("it462_marks.csv")
json_df = pd.read_json("it462_marks.json")

assert csv_df.shape == (30, 8)
assert json_df.shape == (30, 8)
assert list(csv_df.columns) == list(json_df.columns)
assert csv_df.iloc[0].equals(json_df.iloc[0])
assert csv_df["student_id"].iloc[0] == "S01"
assert int(csv_df["marks"].iloc[0]) == 74
```

If the first-row assert fires, you did not load the same thirty students. Typical causes: a different sort order, a column renamed, JSON nested one level deeper than `read_json` expected, or — the classic — you pointed at `it462_dirty.csv` by accident.

The same first row, without pandas

You can prove the match with two loops. Split the CSV header on commas. Split the first data line on commas. Walk the JSON object's keys in header order. If every value agrees (after turning `"74"` into 74), the files agree on S01. pandas is a short cut for that walk, not a different notion of equality.

How to write it in Python

```
# the other suffixes you will actually meet
# xlsx = pd.read_excel("it462_marks.xlsx")    # needs openpyxl
# tables = pd.read_html("file.html")          # a *list* of
    DataFrames
```

`read_html` is not a licence to vacuum the web. Many pages forbid it, wrap the table in

JavaScript, or serve you a skeleton. In this course you parse pages you are allowed to parse. The board circled Python because you should own the conversion, not because scraping is the syllabus.

Watch Out

`pd.read_csv` guesses types. `student_id` looks like text and stays text. A column of zip codes that happen to be digits will arrive as integers, and then a later you will try to average them. The file format did not decide the measurement scale. You will, in Chapter 5. For today: opening the file is not finishing IDA.

The dirty five-row file is also a CSV. Its blank ID is two commas in a row. Its trailing period on `2026.` is a character, not a footnote. You will walk those cells in Chapter 3. Do not "fix" them in the loader and forget what you fixed.

Questions & Variations

Q. Is a JSON file structured?
A. It is structured as nested boxes. It becomes the rectangular kind of structured when you choose a row grain — here, one object per student — and flatten it. Until then you cannot take a column mean.

Q. Who is the primary source of `it462_marks.csv`?
A. Nobody in the room: the file is invented for this book. Treat it as a teaching primary source with a known codebook, not as DA-IICT's archive. If a question asks you to role-play office versus Wikipedia, keep the three-distance story and do not mix it with the toy.

Studio — now you try

2.1 Open `it462_marks.csv` as raw text. How many commas are in the header line? How many fields? Why does the first data line still have eight fields even though the word Ahmedabad is long? What would go wrong if a city were stored as `Old, Rajkot` with no quotes?
2.2 For each situation, say primary, secondary, or tertiary, and name the person who still has the bruises: (a) you record IT462 quiz marks on paper and type them; (b) the DA-IICT office emails you an export; (c) you copy a Wikipedia cell "mean marks 63.5" into a slide and discuss the class.
2.3 Load `it462_marks.csv` and `it462_marks.json`. Assert that the first rows are equal, as in the listing above. Then name two edits that would make the assert fail *without* changing the numerical marks of S01.

Chapter 4

Can You Use These Five Rows?

Right now we are: manage — hunt bruises in the dirty five before anyone averages them

Statistics, on the board, is not a religion. It is a tools package aimed at data. The package is useless if the rectangle is not **usable**. This chapter is the hunt that decides usability. The name of the hunt is **IDA** — Initial Data Analysis. You will not finish the hunt by producing a morally clean table. You will finish it by writing down decisions.

4.1 Usable is four words, not a feeling

From the class board

Statistics = a tools package aimed at Data. Goal: **usable** — validity, reliability, authenticity, error-free. The name of this step: **IDA**.

Concept

Four words from the board, in working English.
Validity. Are you measuring the thing you named? A column called `Age` that contains 0 might be a live birth, a missing-code, or a typing slip onto the wrong key. Validity is not "the number is a number". It is "this number is an age".
Reliability. If you measured again, would you get close to the same value? A mark re-entered from a smudged sheet that flips between 17 and 71 is valid-ish (still "marks") and unreliable.
Authenticity. Is this the real table from the real process — not a copy of a copy with a formula paste, not a screenshot of a screenshot? Chapter 2's primary / secondary / tertiary story is authenticity in source-language.
Error-free. It will not be. The phrase means *errors have been hunted*, not *errors do not exist*. IDA is the hunt.

In Plain Words

IDA asks "may I use this rectangle at all?" EDA asks "what shape is the rectangle I was allowed to use?" If you skip IDA, every pretty histogram is a rumour.

4.2 Five rows, walked in order

From the class board

A table with columns ID, IT462, Age, Gender. Five rows. Deliberately broken.

That table is `it462_dirty.csv`. It is short so you can IDA it with a pencil. Copy it into the margin. Then walk *cells*, not vibes.

ID	IT462	Age	Gender
`2026.`	10	0	M
2026	50	10	F
2026	90	20	M
(blank)	−20	100	F
2023	17	(blank)	(blank)

Row by row, not "the data is messy"

Row 1. ID is `2026.` with a trailing period. That is a string, not an integer that forgot a digit. IT462 = 10 is a possible mark. Age = 0 is either a newborn (ratio-legal, Chapter 6) or a missing-code that stole the name `Age`. Gender M is a complete cell.
Row 2. ID 2026 without a period. Compare with row 1: same digits, different characters. Age 10 is a possible child, an implausible undergraduate, or a column shift. Score 50, gender F: locally fine.
Row 3. ID 2026 again. Duplicate-ish: two people sharing a label, one person typed twice, or three rows (1–3) that are really one ID with a dirty suffix. Score 90, age 20, gender M: the most "ordinary" row in the file, which is why you must not let it launder the others.
Row 4. ID blank. Score −20: not a humble student. Below-zero marks are a sign error, a "not attempted" sentinel, or a column that is not marks. Age 100 is a possible centenarian, a sentinel, or a 10 with a trapped zero. Gender F is present while the ID is not — the row is not empty; it is *partial*.
Row 5. ID 2023, a different year-token from the 2026 block. Score 17, legal as a number. Age blank, gender blank. Missingness is a fact. It is not the number 0, and it is not the string `"blank"`.

How to write it in Python

```python
import pandas as pd
import numpy as np

df = pd.read_csv("it462_dirty.csv", dtype={"ID": "string"})
print(df.to_string())
print(df.isna().sum())
print(df["ID"].tolist())
print(df["IT462"].tolist())
print(df["Age"].tolist())
```

Read `ID` as string or the trailing period will be "helpfully" dropped and you will invent a cleaner table than the disk owns. Print the lists. IDA starts with what is there, not with what a loader coerced.

4.3 Five questions that are not a cleanup

A standard data-science checklist names five worries: structure, granularity, scope, temporality, faithfulness. We will ask them in IT462 words, on *this* table, not on a mystery CSV from another course.



Figure — The five questions, aimed at `it462_dirty.csv`*. Each arrow should land on a cell you can point to, not on a slogan.*

Concept

Structure. Is this a rectangle with a stable number of fields per row? The header has four names. Row 4's blank ID is an empty field, which is still a field. The trailing period on `2026.` means the ID column is not a pure integer, even if three other IDs look like years. Structure also means: did the commas stay in their lanes (Chapter 2)?
Granularity. What is one row supposed to be? One student? One attempt? One year-of-sitting? Rows 1–3 all wearing 2026 make this question sharp. If one row is one student, you have a duplicate-key problem. If one row is one attempt, three rows with the same ID might be legal — and then Age 0, 10, 20 on the same ID is a new bruise, not a duplicate.
Scope. Who is missing from the table? Five rows are not a class. The blank ID is a present row with an absent key. The students who never sat, or never got typed, are absent rows. Scope is the people *not* here. You cannot see them in `df`. You can only remember that they exist.
Temporality. When is this about? The ID values look like years: 2026, 2026, 2023. Is the year a cohort, a calendar stamp, a roll-number prefix, or an accident? Row 5's 2023 sitting next to three 2026s is either a mix of batches or a typo of one digit. IDA does not vote. It writes the two readings down.
Faithfulness. Does each cell mean what the column name claims? Age 0: newborn, or a typo? Age 100: centenarian, or a sentinel? IT462 = −20: a score, or a code? Faithfulness is validity at cell scale. It is the question that stops you from averaging first and asking later.

Two legal decisions on the same Age 0

Codebook A: "Age is years since live birth; 0 means a newborn; keep the row; do not treat 0 as missing." Codebook B: "Age 0 is the missing-code used by the old form; recode to NaN; flag the row." Both can be professional. Neither is licensed by the five numbers alone. What is *not* professional is silently turning 0 into 20 because undergraduates "must be" twenty, or dropping the row and then claiming $n = 4$ as if the file had always had four students.

Watch Out

Do not finish this chapter with a cleaned table and a halo. If you strip the period, impute the blanks, clip −20 to 0, and recode age 100 to 20, you have created a new dataset. That can be the right engineering move *after* the codebook exists. It is the wrong moral of IDA. The board's five rows exist to slow you down.

A loop that hunts, rather than a loop that "fixes":

How to write it in Python

```
id_raw = df["ID"].astype("string")
print("trailing period:", id_raw.str.endswith(".", na=False).tolist())
print("duplicate ID digits:", id_raw.str.replace(r"\.$", "", regex=True).tolist())
print("negative marks:", (df["IT462"] < 0).tolist())
print("age 0 or 100:", df["Age"].isin([0, 100]).tolist())
print("missing age or gender:", df[["Age", "Gender"]].isna().any(axis=1).tolist())
```

Each printed list is a boolean mask aligned with five rows. IDA is mostly masks plus a codebook. A mask is not a deletion.

Questions & Variations

Q. Is a negative mark always an error?
A. It is always a faithfulness alarm. It becomes an error only after the codebook says marks cannot be negative. Some processes use −1 as "absent".

Q. Are rows 1–3 the same student?
A. You do not know. Granularity is the question. Identical ID digits are a clue, not a proof, and row 1 is not even identical as a string.

Q. Why not interpolate the missing age on row 5?
A. Because you would invent a person-year. Missing is information about the process. A filled number pretends the process succeeded.

Studio — now you try

3.1 For each of the five rows, name *one* bruise (cell or clash) and choose drop, recode, or keep-and-flag. You must use all three actions at least once. Do not write a finished clean table.
3.2 Point at one cell (or missing cell) in `it462_dirty.csv` for each of: structure, granularity, scope, temporality, faithfulness. The same cell may not answer two questions.
3.3 Write two four-line codebooks for Age 0 on row 1 that make opposite keep/recode decisions both look legal. Then explain in two sentences why "always drop age 0" is not a rule of IDA.

Chapter 5

Why Did Age Zero Send Us Back?

Right now we are: manage then explore — name the three boxes, then walk the feedback arrow

IDA asked whether the rectangle is usable. This chapter names what happens next, and why "next" is a lie if you treat the arrows as a one-way street. You will also name the parts of the rectangle — columns, rows, categorical, numerical — so later chapters can stop saying "the vertical bits".

5.1 Three boxes, and the arrow people skip

From the class board

Vertical flow: IDA → EDA → CDA. IDA ↔ problem, collection, (preprocessing) cleaning. EDA ↔ statistical aspects, features of data, variables (feedback arrow). CDA ↔ modelling, evaluation.



Figure — Three boxes for the semester. The arrow from EDA back into IDA is the job, not a confession of failure.

Concept

IDA (Initial Data Analysis) is "can I trust this rectangle?" Problem definition, collection, the hunt from Chapter 3. **EDA** (Exploratory Data Analysis) is "what shape is in here?" — distributions, holes, relationships, the features and variables the board circled. **CDA** (Confirmatory Data Analysis) is "does a claim survive a test I did not tune while staring?" — modelling and evaluation.

Mixing EDA and CDA on the same split is how you overfit a story. Look in a sandbox. Confirm on a piece you did not use to choose the story. This book is almost entirely IDA and EDA. CDA is named so you do not call a histogram a proof.

The board's feedback arrow is the sentence this chapter exists to make uncomfortable.

Age 0 is not a footnote, it is a loop

You finish a first IDA on `it462_dirty.csv` and decide, provisionally, that Age is a number of years and that 0 might be rare but legal. You move to EDA: a histogram, or even a sorted list $0, 10, 20, 100, \text{NaN}$. The 0 sits on the axis like a live birth. The 100 sits like a centenarian. Neither looks like the rest of an IT462 class.
That surprise is EDA doing its job. It does *not* license you to clip Age to $[18, 22]$ inside the plotting cell and carry on. It sends you *back* to IDA: open the codebook, ask the collector, decide whether 0 is a newborn, a missing-code, or a typo for 20. Only then do you return to EDA with a rectangle whose Age column means one thing.
The loop on this file:

IDA (read the five rows) → EDA (notice Age 0) → IDA (codebook for 0) → EDA (describe the ages you kept).

If you skip the return, your histogram is a picture of a decision you refuse to admit.

In Plain Words

EDA is allowed to discover that IDA was unfinished. That discovery is success. Hiding it inside `df = df[df.Age > 0]` without a note is how a pipeline starts lying.

Watch Out

CDA is not "I liked the scatterplot so I ran a regression on the same thirty rows and starred $p < 0.05$." Confirmation needs a claim written before the last look, or a split you did not explore. We will not pretend IT462's thirty invented students are a population. We will still refuse to treat an EDA plot as a verdict.

5.2 What a column is, and what a row is

From the class board

Structured data: columns = variables or features. Rows = observations or values. Then a fork: categorical (qualitative) vs numerical (quantitative).

One **row** is one thing in the world: one student, one attempt, one day. One **column** is one question you asked of every thing: age, city, marks. Machine learning people say **feature** for column and **example** for row. Statisticians say **variable** and **observation**. Same rectangle, two dialects. This book uses both and will say so when a distinction matters.

On `it462_marks.csv`, $n = 30$ observations. Eight columns. If the question is "did the student pass?", then `passed` is a **target** (the thing you wanted to know) and the other columns are **features** (the things you might use to talk about it). That split is a question-split, not a file-split. Tomorrow a different question can promote `marks` to the target.

columns = features / variables

rows = observations

id	marks	age	gender
S02	45	20	M

Figure — S02 from `it462_marks.csv`. Every later formula is a computation down a column, or across two columns.

S02, read as observation then as features

Row S02: marks 45, age 20, gender M, Likert 3, city Surat, hours 3, passed yes. As an observation, this is one student-attempt. As a list of features, it is seven answers plus an id. The number 45 is not "the dataset". It is one cell. Averages move down `marks`, not sideways along S02.

5.3 The first fork: label versus quantity

Before NOIR (next chapter) there is a coarser fork the board drew in the same breath.

Categorical / qualitative columns are labels. Gender, city, passed, Likert-as- words. You may count how many sit in each bucket. You may not, without a further argument, add the buckets.

Numerical / quantitative columns are amounts. Marks, age, hours studied. You may add them, once you know the scale (Chapter 6). Digits on the page do not decide the fork: `student_id` is written with letters and digits and is still a label. A future zip code column would be digits and still a label.

column in `it462_marks.csv`	first fork	why, in one line
`student_id`	categorical	a name for a row, not an amount
`marks`	numerical	marks out of 100, 0 means none
`age`	numerical	years lived; Chapter 6 will say ratio
`gender`	categorical	M / F are names of groups
`likert`	categorical	ordered names stored as 1..5
`city`	categorical	Ahmedabad is not less than Surat
`hours_studied`	numerical	hours are an amount
`passed`	categorical	yes / no are names (with a tempting order)

Likert is the troublemaker: stored as integers, born as ordered labels. The next chapter will not let you shrug. For the pipeline, put it on the categorical side and refuse a Likert mean until you can defend the extra assumption.

How to write it in Python

```python
import pandas as pd
df = pd.read_csv("it462_marks.csv")
assert df.shape == (30, 8)
print(df.columns.tolist())
print(df.dtypes)
# int64 is the computer's storage guess, not NOIR
print(df["gender"].value_counts())      # categorical: LEGAL
print(df["marks"].mean())               # numerical: LEGAL (63.5)
```

```
# print(df["student_id"].mean())          # categorical-with-digits: a
    type error, thankfully
```

`dtypes` saying `int64` does not mean "numerical in the board's sense". Zip codes would also arrive as `int64`. The fork is a human decision. Lecture 10 will put `df.shape` first and measurement scale immediately after.

The same eight columns, after the Age 0 loop

Suppose we had histogrammed `it462_dirty.csv` instead of `it462_marks.csv`. `Age` would still be declared numerical — that is the column's job. The feedback loop does not change the fork. It changes whether row 1's 0 is allowed to stay in the numerical column. A missing-code recoded to NaN is still a numerical column with a hole. A recode of Age into `"unknown"` would quietly move the column across the fork. Write that move down if you make it.

Questions & Variations

Q. Is `passed` a feature or a target?
A. It depends on the question. "Did they pass?" makes it a target. "How many hours did passers study?" makes it a grouping feature. The file does not know your question.

Q. Where does scraping HTML sit in IDA / EDA / CDA?
A. The page is a pile. Extraction is IDA (structure). Looking at the extracted table is EDA. A test of a claim about the website's population would be CDA, and this course will not pretend you have that population.

Studio — now you try

4.1 Draw the IDA → EDA → IDA → EDA loop for this event: you plot a histogram of Age on `it462_dirty.csv`, see a bar at 0, and only then read the codebook. Mark which box each of these four actions sits in: (i) opening the CSV, (ii) plotting the histogram, (iii) deciding 0 is a missing-code, (iv) replotting Age without that row.
4.2 The question is "did the student pass?" On `it462_marks.csv`, which column is the target? Which columns are then features? Which one column would you *not* feed a model that is supposed to predict pass/fail, and why?
4.3 Classify every column of `it462_marks.csv` as categorical or numerical. Then name one column whose storage type in pandas will disagree with your classification, and what a careless `.mean()` would invent.

Part II

Measurement scales

Chapter 6

Why Can't You Average a Colour?

Right now we are: explore — split labels into names versus ranks, then refuse a fake mean

Chapter 4's fork was coarse: label versus quantity. This chapter splits the label side. The acronym you will reuse for the rest of the book is **NOIR** — Nominal, Ordinal, Interval, Ratio — but we will earn only the first two letters today. Interval and ratio arrive with a zero in Chapter 6.

6.1 A potato, used once

From the class board

Measurement scales. Categorical → Nominal (names, indexes, tags) / Ordinal. Potato: colour = nominal, size = ordinal. Numerical → Interval / Ratio. Mnemonic: N–O–I–R.

Analogy

A potato is a teaching device, not a grocery list. **Colour** (red / yellow / white) is a name: none of the colours is "more potato". That is **nominal**. **Size** (S / M / L / XL) is a rank: L is above M. That is **ordinal**. You still may not claim that the jump S→M equals the jump L→XL. Weight in grams would be a quantity; that is next chapter. We will not mention the potato again.

In Plain Words

Nominal partitions the world into buckets that refuse to line up. Ordinal lines the buckets up but does not tell you the gap. Coding the buckets as $1, 2, 3$ is a storage trick. It is not permission.

6.2 Twelve columns, one decision each

Walk a column with two questions. Can I only *name* the group? Stop at nominal. Can I *rank* every pair but not measure a gap? Stop at ordinal. If you can measure a gap, you have left this chapter.

The table mixes columns from `it462_marks.csv` with columns you will meet in forms, sports lists, and weather screens. Classify the column, not the integer it happens to be stored as.

column	example	scale	trap
`student_id`	S01	nominal	looks like a code; still a name
`gender`	F	nominal	two buckets, no honest order
`city`	Ahmedabad	nominal	do not sort as if north meant better
zip code	382007	nominal	digits ≠ quantity
jersey number	7	nominal	14 is not twice 7 as a player
blood type	B+	nominal	tags, not ranks
phone area code	079	nominal	a region name written in digits
`likert`	4	ordinal	board's five ticks; gaps unproven
movie star rating	∗∗∗	ordinal	more stars is more; gaps unproven
rank in class	3rd	ordinal	1st beats 2nd; the gap is not marks
`passed`	yes	nominal	two names; order exists only if you add it
`hours_studied`	6	ratio*	amount; true zero; tease Chapter 6

*Twelve decisions. *hours leave the categorical side; they sit here so you feel the fork close. Celsius, Kelvin, and year CE are interval/ratio — next chapter, same habit.*

`marks` and `age` from `it462_marks.csv` are quantities too. Phone area codes and zip codes are the failure case of this chapter: pandas will store them as integers and compute a mean that names a town nobody lives in.

Watch Out

`df["zip"].mean()` is not a syntax error. It is a scale error. The computer does not know NOIR. You do. Jersey number 18 averaged with 22 is not a player.

How to write it in Python

```python
import pandas as pd
df = pd.read_csv("it462_marks.csv")
g = df["gender"]
print(g.value_counts(dropna=False))      # LEGAL on nominal
print(g.mode())                          # the only "centre" you may quote
# g.mean()                               # TypeError if stored as text --- good

city = df["city"].astype("category")     # still nominal: no order
print(city.cat.ordered)                  # False

likert = pd.Categorical(
    df["likert"], categories=[1, 2, 3, 4, 5], ordered=True
)
print(likert.min(), likert.max())        # LEGAL: order exists
# likert.mean()                          # illegal while it is categorical
```

`ordered=True` is the entire ordinal idea in software. Without it, pandas thinks "5" and "1" are two city names that happen to be digits.

6.3 The fourth label: binomial (binary) features

From the class board

Imported data (CSV) → features / columns / variables → **nominal**, **ordinal**, **binomial**, **numeric** (interval, ratio). Features → values / observations / records → ranges, feasibility, duplication, missing values and outliers.

Binomial (binary, dichotomous) feature

A nominal feature with exactly **two** categories: passed yes/no, smoker yes/no, spam/not spam, gender recorded as two values. It is a special case of nominal, but it earns its own name in the course because of three privileges:

1. Coded as 0/1, its **mean is the proportion** of 1s: $\overline{\text{passed}} = 26/30 = 0.867$ means 86.7% passed. (For a nominal column with three or more labels, no mean exists.)
2. Each value is one Bernoulli trial, and the count of 1s in n rows is **Binomial**(n, p) — hence the course's word "binomial".
3. It unlocks its own association measures: **point-biserial** against a numeric column and **Cramér's** ϕ against another binary column (association chapter).

How to write it in Python

```
df["passed"].value_counts()                          # yes 26, no 4
df["passed"].nunique() == 2                          # quick test: is
    this column binary?
passed01 = df["passed"].map({"yes": 1, "no": 0})   # recode text to
    0/1 first
passed01.mean()                                      # 0.8667 =
    proportion who passed
```

Feature type	Example	Legal centre
Nominal (≥ 3 labels)	city, blood group	mode
Binomial (2 labels)	passed, smoker	mode; mean = proportion if coded 0/1
Ordinal	Likert, size, grade	mode, median
Numeric: interval	°C, calendar year	mode, median, mean
Numeric: ratio	marks, age, income	all of these (+ geometric/harmonic mean)

6.4 Why a Likert bar is not a histogram

From the class board

Likert scale. Five ticks: strongly disagree, disagree, neutral, agree, strongly agree. Side note: 3-point, 5-point, 7-point variants.

A **Likert item** is a sentence plus an ordered row of agreement. 3-point is coarse (disagree / neutral / agree). 5-point is the board's drawing. 7-point inserts "slightly" on each side. More ticks are more resolution, not more proof that the gaps are equal.

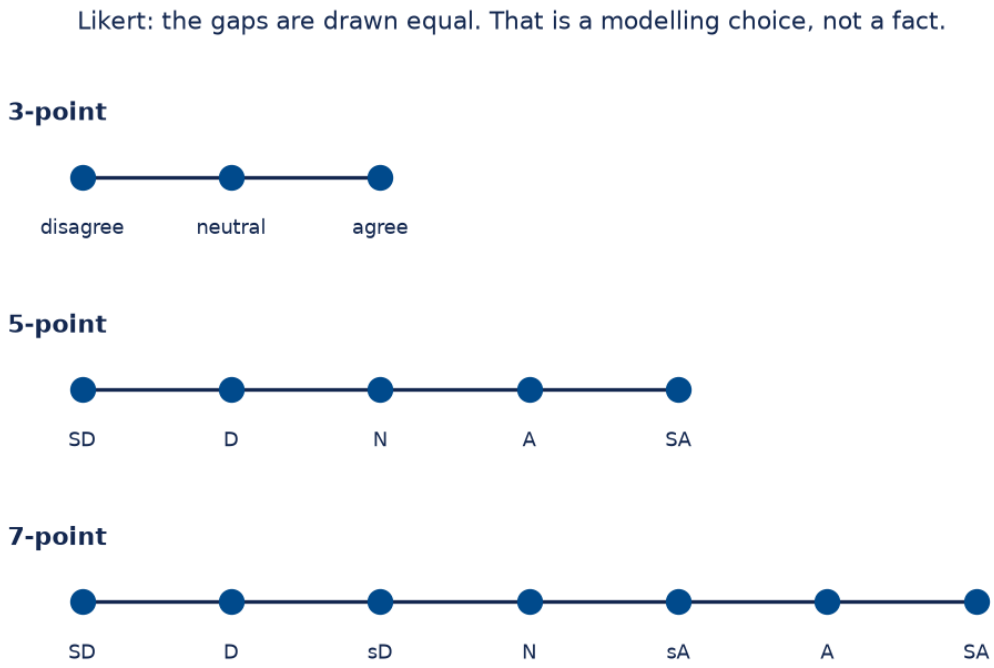


Figure — 3-point sits inside 5-point; 7-point adds two more ticks. The axis is a list of labels, not a ruler until you argue otherwise.

Plot Likert with a **bar chart of counts**, one bar per label. A histogram on the integers 1..5 pretends there is a continuum between 3 and 4 that a student could have occupied. There is not. The student ticked a box.

On `it462_marks.csv` the first five Likert values are already a tiny table you can finish by hand: S01 = 4, S02 = 3, S03 = 5, S04 = 3, S05 = 2.

id	S01	S02	S03	S04	S05
Likert	4	3	5	3	2

Counts: 2 once, 3 twice, 4 once, 5 once. Mode = 3. That sentence used only names-with-order. It did not need equal gaps.

6.5 Mean 3.4 or median 3, on the same five ticks

From the class board

Arrow: Ordinal $\xrightarrow{\text{transform, Likert scale}}$ Interval. Goals: (1) make quantifiable equivalence with numbers; (2) make the difference equal in length.

The board's transform prints $1, 2, 3, 4, 5$ under the five labels and then treats those integers as if "disagree → neutral" equalled "neutral → agree". Once you accept equal length, + and − become legal and a mean of agreement-points is a defined arithmetic. Stevens, who named NOIR, would call this a *convention*, not a discovery. Many working scientists do it anyway, especially when they sum several items into a scale. We will not settle the philosophy. We will run both recipes on S01–S05.

The same five answers, two centres

Treat as **ordinal**: sort $2, 3, 3, 4, 5$. Odd $n = 5$, median = the middle tick = 3. Mode = 3. Legal without any gap story.
Treat as **interval** (board's transform):

$$\bar{x} = \frac{4+3+5+3+2}{5} = \frac{17}{5} = 3.4.$$

The extra 0.4 is the 5 pulling against the 2, after you granted every step a length of 1. The equation

$$(2-1) = (3-2) = (4-3) = (5-4) = 1$$

is true of the numbers you wrote. It is an assumption about the minds you surveyed. If people are shy of "strongly", the jump $4 \to 5$ is larger in the world than the jump $3 \to 4$, and 3.4 is a neat number about a ruler nobody used.

Derivation — step by step

You may quote 3.4 if you state the convention. You may not then take ratios. "Agree (4) is twice disagree (2)" is illegal even after the transform. The transform, at best, licenses $+$ and $-$. $\times$ and $\div$ still want a true zero, and "strongly disagree" is not the absence of opinion. That last sentence is Chapter 6 leaking in on purpose: ordinal-as-interval is not ratio.

How to write it in Python

```python
import numpy as np
import pandas as pd
codes = [4, 3, 5, 3, 2]
assert sum(codes) / len(codes) == 3.4
assert np.mean(codes) == 3.4
assert np.median(codes) == 3.0
# bar chart of counts, not a histogram of a continuum
print(pd.Series(codes).value_counts().sort_index())
```

Questions & Variations

Q. Is passed ordinal because fail < pass?
A. You may impose that order. The file's yes/no are still two names. If you rank them, say so; do not average yes= 1 with no= 0 and call it a mean pass.

Q. Why not a histogram of Likert?
A. A histogram claims a numeric axis with bins you chose. Likert has five labels. Bars keep the labels. Histograms invent the between.

Studio — now you try

5.1 Classify each as nominal or ordinal (or "left this chapter"): blood type, rank in class, city, movie star rating, jersey number, `hours_studied`. One sentence each.
5.2 Using only S01–S05 Likert values $4, 3, 5, 3, 2$, give the mode and the median. Sketch the five-label bar chart (counts, not a histogram). Why would a histogram of those

integers be the wrong picture even though the median is legal?
5.3 Zip code 382007 and hours studied 6 are both integers in a spreadsheet. Name the scale of each. What fake sentence would `.mean()` invent for the zip codes of five students living in 382007, 380001, 395003, 390001, 360001?

Chapter 7

When Is Twice Allowed?

Right now we are: explore — add equal gaps and a true zero, then name which verbs are legal

Nominal named. Ordinal ranked. This chapter buys two more powers: a gap you can measure, and a zero that means none. The powers decide the algebra. They also decide, later, which summary families you are even allowed to reach for.

7.1 Four ticks, accumulated left to right

From the class board

Four rows of properties, four columns N O I R, ticks accumulating as you walk right: labelled; inherent order; measurable difference; absolute (true) zero.

property	N	O	I	R
labelled / a category exists	✓	✓	✓	✓
inherent order	×	✓	✓	✓
measurable equal difference	×	×	✓	✓
absolute (true) zero	×	×	×	✓

Each level keeps everything to its left and gains one verb. Ratio is nominal + order + equal gaps + true zero. You do not skip a tick because the column is important.

In Plain Words

Interval: order plus equal gaps, but 0 is a convention. Calendar year, Celsius. $20 - 10$ equals $10 - 0$ as a gap; 20 is not twice 10. **Ratio:** interval plus a zero that means none. Age, marks if 0 is no marks, Kelvin, hours studied, counts. Twice is allowed.

7.2 Age is ratio, even if no newborn sits in class

From the class board

Age — Ratio (because of Absolute Zero). Parenthetical: "a person with age 0 is impossible". Variables / features hold values.

Absolute zero means the number 0 on this scale is the absence of the quantity. Age from birth: 0 years is "just born", not "we have no idea". You cannot be -3 years old. A 20-year-old has lived twice a 10-year-old's life. That is the ratio license.

The board's quip that a person of age 0 is impossible is a joke about *this room*: you meet undergraduates, not live births. It is not a claim about the scale. The scale has a true zero whether or not the sample contains it. `it462_dirty.csv` row 1's Age 0 is still a faithfulness problem (newborn, missing-code, or typo). Solving that problem does not demote Age from ratio to interval. A conceptual zero can exist on the ruler while being absent from the classroom.

On `it462_marks.csv`, mean age is exactly 20. That 20 is a legal mean because Age is a quantity with equal years and a true zero. The legality does not prove every recorded age was faithful. IDA and scale are two different questions.

7.3 Twenty degrees is not twice ten — same pair, two rulers

The board's numerical example is Celsius versus Kelvin, not a new formula for Age.

20°C and 40°C, both rulers

Difference on Celsius:

$$40°\text{C} - 20°\text{C} = 20°\text{C}.$$

The degree is an equal gap. Interval operations + and − are legal. The number 0 is the freezing point of water, a convention, not "no temperature".
Ratio on Celsius:

$$\frac{40°\text{C}}{20°\text{C}} = 2.$$

That 2 is arithmetic. It is not physics. 40°C is not "twice as hot" as 20°C.
Shift the same two rooms onto Kelvin with the board's 273 offset ($T_K = T_C + 273$), so $20°\text{C} \mapsto 293\,\text{K}$ and $40°\text{C} \mapsto 313\,\text{K}$. The difference is unchanged:

$$313 - 293 = 20,$$

as it must be — Kelvin and Celsius share a degree size. The ratio is now

$$\frac{313}{293} \approx 1.068,$$

about seven percent hotter on an absolute scale, not twice. Twice would have required a true zero under the 20. Celsius never had one. Kelvin does (absence of thermal energy, at 0 K).

The same two temperatures, the illegal sentence written out

"It is twice as hot" is a ratio sentence. On Celsius it points at $40/20 = 2$ and lies. On Kelvin it points at $313/293 \approx 1.07$ and is at least the right *kind* of sentence. EDA's job is not to ban arithmetic. It is to refuse the sentence the scale does not license.

How to write it in Python

```
c = [20, 40]
assert c[1] - c[0] == 20
assert c[1] / c[0] == 2          # arithmetic only --- not a heat
    claim

k = [c_i + 273 for c_i in c]     # 293, 313
assert k[1] - k[0] == 20
assert abs((k[1] / k[0]) - 313 / 293) < 1e-12
```

```
print(313 / 293)                    # 1.0682...  not 2
```

Year CE is the other interval the twelve-column table postponed: year 0 is a calendar convention (and a disputed one), not "no time". Age derived from two years *is* ratio, because a duration can be zero.

7.4 Which verbs are legal?

From the class board

Operations table: N none, O none, I + and −, R × and ÷.

The board drew a large X under N and O because you do not add genders or multiply spicy-levels. Under I it wrote + and −. Under R it wrote × and ÷ — not because you cannot add masses, but because the *new* power that appears at R is the ratio.

	Nominal	Ordinal	Interval	Ratio
$=\ \neq$ (same / not)	yes	yes	yes	yes
$<\ >$ (rank)	no	yes	yes	yes
$+\ -$ (gap)	no	no*	yes	yes
$\times\ \div$ (ratio)	no	no	no	yes

*Likert-as-interval is the Chapter 5 convention: people subtract the codes anyway. Even then, ratios stay illegal.

Walk a mystery column from the top. Stop at the first "no".

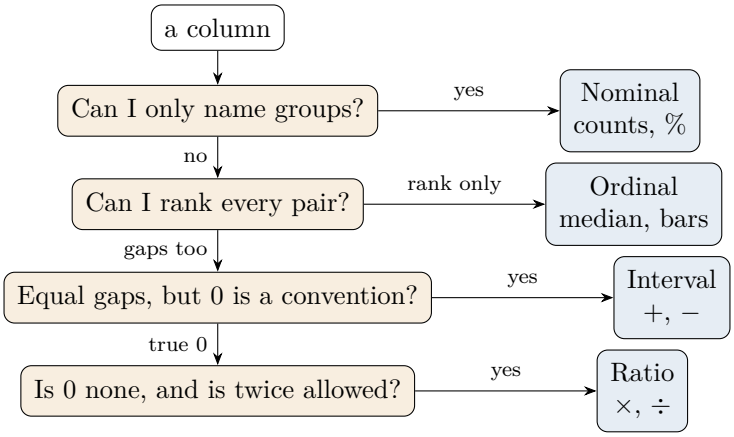


Figure — Walk top-down. Gender stops at the first yes. Likert can stop at ordinal. Year CE takes the interval leaf. Marks and Age should reach ratio.

Four walks on columns you already know

`student_id`: only a name → nominal. Do not average roll numbers. `likert`: names with order, gaps unproven → ordinal (or interval after the Chapter 5 convention, still not ratio). Year CE: equal years, year 0 a convention → interval. $2026 - 2023 = 3$ is legal; $2026/1013 = 2$ is not "twice as late". `marks`: 0 can mean no marks, 80 is twice 40 as a score → ratio.

Watch Out

The failure case is a ratio sentence on an interval ruler. 40°C is not twice 20°C. Year 2000 is not "twice" year 1000 in any historical sense the arithmetic suggests. If a viva asks "may I divide?", answer with the zero, not with the vibe of the column.

7.5 Descriptive, inferential, and five families with no formulas yet

From the class board

Statistical tools split into **Descriptive** and **Inferential**. Descriptive = summary of *this* data. "Five different measures".

Descriptive statistics compress the table you have: the thirty marks, the five dirty rows after a codebook. **Inferential** statistics use this table to say something about a larger world you did not fully see. EDA is mostly descriptive, with the humility to remember the world is bigger. Sampling and experiments (later) are how inference stops being a wish.

The board's five "measures" are families, not five formulas. This book will fill them in later chapters. Learn the names now so a mean does not have to pretend it is the whole toolkit.

family	question it answers	example you already met
central tendency	where is the pin?	mean 46.75, median 35.5
dispersion	how spread are the pins?	17 versus 99 on one quiz
shape	is the pile lopsided?	a 99 with three low scores
relative standing	where is *this* value?	S05's 39 among thirty marks
association	how do two piles move?	hours versus marks, later

No formula in that table is an accident. A family chosen on the wrong scale is the same crime as $40/20 = 2$ in Celsius: legal arithmetic, illegal sentence. Central tendency on nominal is a mode. On ordinal, a median. On interval and ratio, a mean is finally a candidate — and Chapter 1 already showed that a candidate can hide a 99.

Questions & Variations

Q. Why is a count of students ratio rather than interval?
A. Zero students is none. Forty is twice twenty. Counts are the poster child of ratio.

Q. Is GPS latitude ratio?
A. Interval-ish: 0° is a convention (the equator), not "no latitude". Differences can be kilometres north. Ratios of latitudes are not "twice as north".

Studio — now you try

6.1 Copy the N/O/I/R tick table. For `age` on `it462_marks.csv` and for room temperature in Celsius, tick or cross each of the four properties. Where do the two columns first disagree, and which operation does that disagreement forbid?
6.2 Take 20°C and 40°C. Compute the Celsius ratio and the Kelvin ratio using 273 as the offset (293 and 313). Which of the two numbers is allowed to support a "twice as hot" claim, if either? Show the difference in both units.

6.3 Walk the decision tree on `student_id`, `likert`, year CE, and `marks`. Stop at the first honest leaf. For each, name one verb from the operations table that is illegal at that leaf.

Part III

Univariate summaries

Chapter 8

Why One Pin Is Never Enough

Right now we are: explore — pin a centre, then refuse to trust a single pin

Lecture 7 puts three means on the board, then two tweaks (trimmed, weighted). Every formula below is run on the same four numbers $\{1, 2, 4, 6\}$. The thirty marks in `it462_marks.csv` wait at the end of each section so the toy never becomes the whole course: their arithmetic mean is 63.5, and that is the only mean the file has earned until we compute another.

In Plain Words

A mean is a compression: n numbers in, one number out. Different compressions keep different facts. Additive piles keep the arithmetic mean. Growth rates keep the geometric mean. Rates with a fixed numerator keep the harmonic mean. Outliers ask you to trim. Unequal importance asks you to weight. The crime is not choosing one. The crime is choosing one before you know which fact you wanted to keep.

8.1 What if every row pulls equally?

From the class board

Arithmetic mean, no sort required:

$$\bar{x} \;=\; \frac{1}{n}\sum_{i=1}^{n} x_i.$$

Tiny example from the board: $x = (1, 2, 4, 6)$, $n = 4$.

Four additions, one division

$$\bar{x} \;=\; \frac{1+2+4+6}{4} \;=\; \frac{13}{4} \;=\; 3.25.$$

On `it462_marks.csv` the same formula is not a vibe. Marks sum to 1905, $n = 30$:

$$\bar{x}_{\text{marks}} \;=\; \frac{1905}{30} \;=\; 63.5.$$

Memorise 63.5. Later chapters will accuse you if you drift.

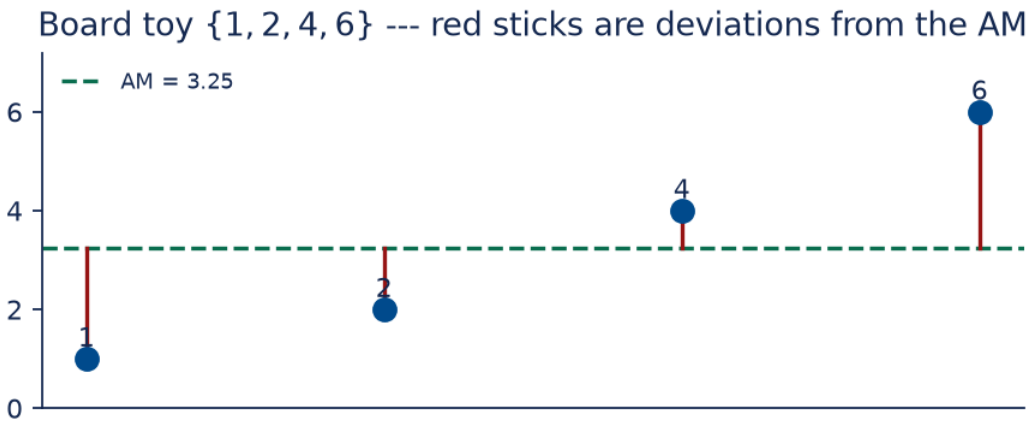


Figure — The board toy $\{1, 2, 4, 6\}$. The teal line is the AM 3.25. Each red stick is a signed deviation — they will sum to zero in a moment.

Derivation — step by step

Why divide by n? The sum 13 grows if we copy the class. Dividing by n makes "typical marks" comparable between four rows and four hundred.
The signed deviations from $\bar{x}$ are a torque balance:

$$\sum_{i=1}^{4}(x_i - \bar{x}) = (1 - 3.25) + (2 - 3.25) + (4 - 3.25) + (6 - 3.25)$$
$$= (-2.25) + (-1.25) + (0.75) + (2.75) \;=\; 0.$$

In general $\sum(x_i - \bar{x}) = \sum x_i - n\bar{x} = 0$ by substituting the definition. The AM is the unique fulcrum at which the table does not tip. That is why a single 99 in a later lecture can drag the pin: it is a long lever, and every row gets the same mass.

Another way to do it

A running mean is the same pin, one arrival at a time: $\bar{x}_{k+1} = \bar{x}_k + (x_{k+1} - \bar{x}_k)/(k+1)$. On $\{1, 2, 4, 6\}$: 1, then 1.5, then $2.\overline{3}$, then 3.25. Same $13/4$, no second pass.

How to write it in Python

```
xs = [1.0, 2.0, 4.0, 6.0]
s = 0.0
for x in xs:
    s += x
am = s / len(xs)

import numpy as np
assert abs(am - np.mean(xs)) < 1e-12
assert abs(am - 13 / 4) < 1e-12
```

If the assert fires, you averaged the wrong list. The thirty-row mean 63.5 is a different compression of a different table.

8.2 What if the story is a product, not a sum?

From the class board

For $x_i > 0$,

$$\text{GM} = \Big(\prod_{i=1}^{n} x_i\Big)^{1/n} = \exp\Big(\frac{1}{n}\sum_{i=1}^{n} \log x_i\Big).$$

Product, then fourth root

$$1 \cdot 2 \cdot 4 \cdot 6 = 48, \qquad \text{GM} = 48^{1/4}.$$

Logs, every term: $\log 1 = 0$, $\log 2 \approx 0.693147$, $\log 4 = 2\log 2 \approx 1.386294$, $\log 6 \approx 1.791759$. Sum ≈ 3.871200, mean ≈ 0.967800, $\exp(0.967800) \approx 2.632148$. Check: $2.632148^4 \approx 48$.

Derivation — step by step

The product form overflows on long growth-rate lists. Averaging logs and exponentiating is the same number. Use GM when the mechanism is multiplicative: "up 10% then down 10%." AM of 1.10 and 0.90 is 1.00 (broke even, says the liar). GM is $\sqrt{0.99} < 1$.

Another way to do it

Same four numbers: $\text{GM} = 2 \cdot 3^{1/4}$ because $48 = 16 \cdot 3$ and $16^{1/4} = 2$. A factorisation of $48^{1/4}$, not a new mean.

How to write it in Python

```python
import math
from scipy.stats import gmean
logsum = 0.0
for x in xs:
    logsum += math.log(x)
gm = math.exp(logsum / len(xs))
assert abs(gm - gmean(xs)) < 1e-12
assert abs(gm - 48 ** 0.25) < 1e-12
```

8.3 What if we averaged rates?

From the class board

Harmonic mean, $x_i > 0$:

$$\text{HM} = \frac{n}{\sum_{i=1}^{n} 1/x_i}.$$

Reciprocals first

$$\frac{1}{1} + \frac{1}{2} + \frac{1}{4} + \frac{1}{6} = \frac{12}{12} + \frac{6}{12} + \frac{3}{12} + \frac{2}{12} = \frac{23}{12}, \qquad \text{HM} = \frac{4}{23/12} = \frac{48}{23} \approx 2.086957.$$

Decimals $1 + 0.5 + 0.25 + 0.1\overline{6} = 1.91\overline{6}$ are the same $23/12$. Do not leave the sum as 1.9167 and hope.

Derivation — step by step

HM is the reciprocal of the AM of the reciprocals. If you travel the same *distance* at speeds x_i, time is proportional to $1/x_i$, so the fair average speed is harmonic. Equal *times* would have licensed the AM. The formula does not know about cars. You have to know which quantity was held fixed.

Another way to do it

Same four numbers: $\mathrm{AM}(1/x) = (23/12)/4 = 23/48$, so $\mathrm{HM} = 1/\mathrm{AM}(1/x) = 48/23$. The n lives inside the AM of the reciprocals; forgetting it produces the trap $12/23$.

How to write it in Python

```python
from scipy.stats import hmean
inv = 0.0
for x in xs:
    inv += 1.0 / x
hm = len(xs) / inv
assert abs(hm - hmean(xs)) < 1e-12
assert abs(hm - 48 / 23) < 1e-12
```

8.4 Do the three pins even agree?

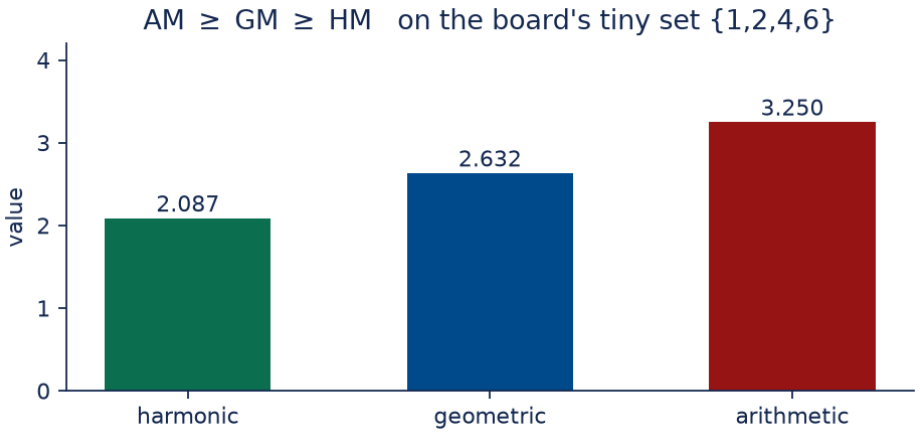


Figure — AM 3.250, GM 2.632, HM 2.087 on $\{1, 2, 4, 6\}$. The inequality is a theorem. These four numbers already witness it.

Derivation — step by step

For positive numbers, AM $\geq$ GM $\geq$ HM, with equality if and only if every x_i is equal. Witness, first, by arithmetic we already own:

$$3.25 \;\geq\; 2.632148 \;\geq\; 2.086957.$$

Optional pair argument: $(1+6)/2 = 3.5 \geq \sqrt{6} \approx 2.449$ and $(2+4)/2 = 3 \geq \sqrt{8} \approx 2.828$. The four-point AM is the AM of those pair-AMs ($(3.5+3)/2 = 3.25$); the four-point GM is the GM of the pair-GMs ($(\sqrt{6} \cdot \sqrt{8})^{1/2} = 48^{1/4}$). HM versus GM is AM–GM on the reciprocals $\{1, 1/2, 1/4, 1/6\}$, whose AM is $23/12$ and whose GM is $48^{-1/4}$.

8.5 What if we refuse the extremes?

From the class board

Trimmed mean: sort, drop the lowest and highest (the board: "mean after removing the extremes x_i"), then take the AM of what remains.

Drop 1 and 6

Sorted $\{1, 2, 4, 6\}$ is already sorted. Drop one value from each end (25% each side on $n = 4$):

$$\frac{2+4}{2} = 3.$$

The AM was 3.25. Trimming pulled the pin toward the middle two, which is the point. On `it462_marks.csv` a 10% trim would drop the three lowest and three highest marks and average the remaining twenty-four — a later studio, not a new formula.

How to write it in Python

```
from scipy.stats import trim_mean
assert abs(trim_mean(xs, 0.25) - 3.0) < 1e-12
```

`proportiontocut=0.25` means "drop a quarter from each side", which on four rows is exactly one row each side.

8.6 What if some rows should count more?

From the class board

Weighted mean, weights $w_i > 0$:

$$\bar{x}_w = \frac{\sum w_i x_i}{\sum w_i}.$$

Ordinary AM is the special case $w_i = 1$.

Toy weights, then three students

On $\{1, 2, 4, 6\}$ take $w = (1, 1, 1, 3)$ so the 6 counts three times:

$$\bar{x}_w = \frac{1 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 + 3 \cdot 6}{1 + 1 + 1 + 3} = \frac{25}{6} \approx 4.1667.$$

Now S09, S02, S08 of `it462_marks.csv`: marks $28, 45, 70$, hours $1, 3, 6$ (S01 also studied

6 hours; we are not using S01 here). Weight marks by hours:

$$\frac{1 \cdot 28 + 3 \cdot 45 + 6 \cdot 70}{1 + 3 + 6} = \frac{28 + 135 + 420}{10} = 58.3.$$

Unweighted AM of those three marks: $143/3 \approx 47.67$. Hours pull toward 70.

Another way to do it

`numpy.mean` cannot see weights. `numpy.average` can. `np.average(xs)` with no weights *is* the AM 3.25. Passing `weights=[1,1,1,3]` is the 25/6 we just wrote. Same function, a different contract. Assert both.

How to write it in Python

```
w = [1.0, 1.0, 1.0, 3.0]
num = den = 0.0
for wi, xi in zip(w, xs):
    num += wi * xi
    den += wi
wmean = num / den
assert abs(wmean - np.average(xs, weights=w)) < 1e-12
assert abs(np.average(xs) - np.mean(xs)) < 1e-12

marks = [28.0, 45.0, 70.0]
hours = [1.0, 3.0, 6.0]
assert abs(np.average(marks, weights=hours) - 58.3) < 1e-12
```

Watch Out

GM and HM are undefined, or they lie, the moment a value is ≤ 0. $\log 0$ is $-\infty$; $\log(-1)$ is not real; $1/0$ explodes. A single zero in a growth-rate column kills the GM. A single negative return does the same if you naively log it. The AM of $\{1, 2, 4, 0\}$ is still 1.75 and will not warn you that the other two means have left the building. On marks this is rare. On "percent change" it is the default disaster.

Studio — now you try

7.1 By hand, AM, GM, and HM of $\{2, 4, 8\}$. Confirm AM $\geq$ GM $\geq$ HM by writing the three numbers. Then replace 8 by 0 and say, in one sentence each, which of the three means still exist.

7.2 On `it462_marks.csv`, recompute the arithmetic mean of marks after dropping the four fails (S05, S09, S18, S29). How far did 63.5 move? What does the size of that move tell you about whether the class mean is being held hostage by the left tail?

7.3 Using hours as weights, the weighted mean of marks on {S09, S02, S08} is 58.3. Write the unweighted mean of those three marks next to it. Which sentence is legal — "typical marks", or "marks per hour of study, averaged" — and which sentence did the weights actually license?

Chapter 9

The Middle After a Sort, and the Name That Won

Right now we are: explore — pick a centre that the measurement scale actually licenses

Lecture 7's remaining two pins do not average. The median asks for a sort. The mode asks who showed up most often. Both are computed below on tiny lists and then on the thirty marks of `it462_marks.csv`, whose sorted column you were told to keep in Chapter 0.

In Plain Words

The mean is a fulcrum. The median is a seat in a queue: half the ordered table on one side, half on the other. The mode is a vote. Changing the largest mark to a billion moves the fulcrum and does not move the seat. That is not a personality quirk. It is the definition.

9.1 Where is the middle once the line is sorted?

From the class board

Sort into order statistics $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$. Odd n: median $x_{((n+1)/2)}$. Even n: average of the two central values, $\frac{1}{2}\big(x_{(n/2)} + x_{(n/2+1)}\big)$. Sort required: yes.

Odd four-plus-one, even four, then thirty

Odd toy, append an 8 to the board list: $(1, 2, 4, 6, 8)$, $n = 5$.

$$\frac{n+1}{2} = 3, \qquad \tilde{x} = x_{(3)} = 4.$$

Even toy, the board's four: $(1, 2, 4, 6)$, $n = 4$.

$$\tilde{x} = \frac{1}{2}\big(x_{(2)} + x_{(3)}\big) = \frac{1}{2}(2 + 4) = 3.$$

The AM of those four was 3.25. The median is 3. They disagree because 1 and 6 are not mirror images of 2 and 4.

Now `it462_marks.csv`, $n = 30$ even. Sorted marks, copied from Chapter 0:

$$28, 31, 35, 39, 41, 42, 45, 47, 50, 53,$$
$$55, 58, 59, 61, 62, 66, 68, 70, 73, 74,$$
$$74, 76, 77, 81, 84, 85, 88, 90, 94, 99.$$

Positions $n/2 = 15$ and $n/2 + 1 = 16$ are 62 and 66:

$$\tilde{x}_{\text{marks}} = \frac{62 + 66}{2} = 64.$$

Mean 63.5, median 64. Almost the same, which is why this file is a fair playground and not a gotcha.

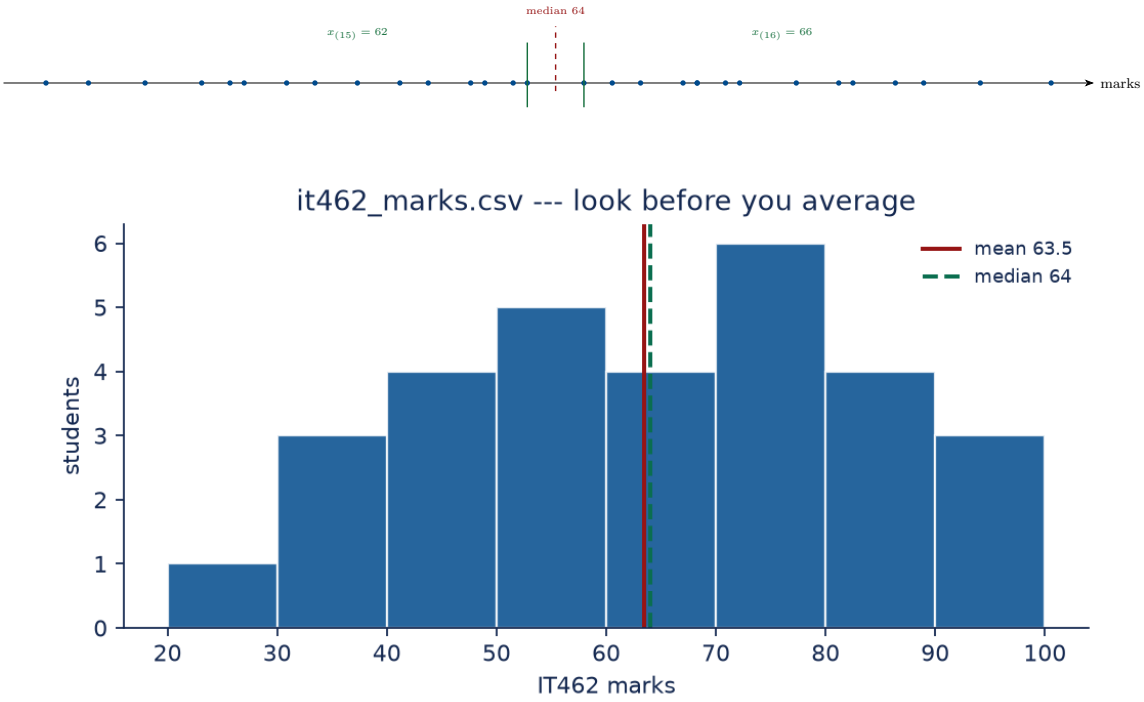


Figure — Thirty IT462 marks. Mean 63.5 and median 64 sit almost on top of each other — look, then average.

Derivation — step by step

Order statistics are not the original indices. $x_{(1)}$ is the smallest, whoever owned it. For odd n there is a single middle index. For even n there is a gap, and the board fills the gap by averaging the two values that hug it. That average is a convention (some texts report both centrals and refuse to split). IT462 splits. On thirty marks the split is $(62 + 66)/2 = 64$, an integer by luck of the file, not by law.

The median is the 50th percentile (next chapter). Half the ordered pile is $\leq$ it, half is $\geq$ it, up to the even-n handshake.

Another way to do it

A selection algorithm can return $x_{(k)}$ without fully sorting, in linear time. The *value* does not change. On $(1, 2, 4, 6)$ you still average 2 and 4. On the thirty marks you still average 62 and 66. Numpy's `median` sorts; pandas' `median` sorts. Assert against the handshake, not against a faster rumour.

How to write it in Python

```
tiny_odd = [1.0, 2, 4, 6, 8]
tiny_even = [1.0, 2, 4, 6]
s = sorted(tiny_even)
n = len(s)
med_even = 0.5 * (s[n // 2 - 1] + s[n // 2])

import numpy as np
import pandas as pd
assert abs(med_even - np.median(tiny_even)) < 1e-12
assert abs(np.median(tiny_odd) - 4.0) < 1e-12

marks = pd.read_csv("it462_marks.csv")["marks"].to_numpy()
sm = sorted(marks)
assert abs(0.5 * (sm[14] + sm[15]) - 64.0) < 1e-12
assert abs(np.median(marks) - 64.0) < 1e-12
```

9.2 Which value showed up most?

From the class board

Mode = the value with the highest frequency. Needs a sort (practically) or a count. Legal on every scale, including nominal.

Marks, gender, city

On $\{1, 2, 4, 6\}$ every value appears once. There is no useful mode. The pile is not voting. On `it462_marks.csv` the marks column contains two 74s (S01 and S30) and no other repeated score. The mode of marks is 74.
Gender: 15 F and 15 M. The vote is tied. There is no single mode; the column is balanced on purpose. Saying "the modal gender is female" is false. Saying "bimodal, both 15" is honest.
City counts: Ahmedabad 9, Surat 8, Vadodara 7, Rajkot 6. Ahmedabad wins. The mode of city is Ahmedabad. That sentence is the only central-tendency sentence the city column licenses.

column of `it462_marks.csv`	count that won	mode
marks	74 appears twice; every other score once	74
gender	F 15, M 15	none (tie)
city	Ahmedabad 9	Ahmedabad

Derivation — step by step

A frequency table is the whole definition. Let $f(v)$ be the number of rows equal to v. The mode is $\arg\max_v f(v)$, unique if one v strictly dominates, multiple if several v share the max, absent-as-a-tool if $f(v) = 1$ for every v. No algebra beyond counting. That poverty is the feature: nominal names have no algebra, and the mode does not invent one.

Another way to do it

`Series.mode()` returns *all* maximisers. On gender it returns both `F` and `M`. On marks it returns 74. On city it returns `Ahmedabad`. A classmate who prints `.mode().iloc[0]` and walks away will report F as the modal gender whenever pandas lists F first — a tie broken by sort order, not by the data. Read the whole Series.

How to write it in Python

```
from collections import Counter
def modes(values):
    c = Counter(values)
    m = max(c.values())
    return sorted(v for v, k in c.items() if k == m), m

df = pd.read_csv("it462_marks.csv")
got, k = modes(df["marks"].tolist())
assert got == [74] and k == 2
assert set(df["gender"].mode()) == {"F", "M"}
assert df["city"].mode().tolist() == ["Ahmedabad"]
assert int((df["city"] == "Ahmedabad").sum()) == 9
```

9.3 Which pin is legal for this column?

The board closed Lecture 7 with a map from measurement scale to centre. It is a contract, not a suggestion.

scale	licensed centre	why
nominal	mode only	names do not add or order
ordinal	median (mode allowed)	order exists; gaps may be a lie
interval / ratio	mean if symmetric; else median	addition is legal; outliers still exist

In Plain Words

City → Ahmedabad, never a "mean city". Likert → median of the five rungs, not because the mean is impossible to type, but because the gaps were drawn equal by us. Marks → 63.5 is legal *and* the histogram is almost symmetric, so the mean is not being kidnapped. If the histogram had a long right tail, the licensed headline would have been 64, not 63.5.

Watch Out

The mean of a zip-code column is the same crime as the mean of city names with the digits left on. The median of gender does not exist. The mode of marks exists (74) and is a terrible summary of a nearly symmetric exam — it happens to be the value that collided twice, not the centre of the pile. Licensed is not the same as wise. The table above tells you what you *may* compute. The histogram tells you what you *should* quote.

Analogy

A canteen queue: the **mode** is the snack that sold most. The **median** bill is the middle customer in a line sorted by spend. The **mean** bill is pulled by the one uncle buying for a wedding. Quote all three and you understand the evening. Quote only the mean and you overstock the wedding tray.

Studio — now you try

8.1 Odd list $\{28, 31, 35, 39, 41\}$: write the median and the index you used. Even list $\{28, 31, 35, 39\}$: write the two order statistics you averaged. Then, without opening a spreadsheet, name the two order statistics of the thirty marks that average to 64.

8.2 The city mode is Ahmedabad with 9 rows. Two new students from Rajkot join the file. What happens to the city mode? What — if anything — can you say about the mean of marks? (You have not been given their marks.)

8.3 A classmate reports the mean of the Likert column. Using the scale table, write three sentences: when that number is a white lie, when it is a crime, and which centre the table actually licenses for an ordinal five-point item.

Chapter 10

How Wide Is the Pile?

Right now we are: explore — after the pin, measure the scatter around it

A centre without a width is a dare. Lecture 7's second family is dispersion: range, variance (population and sample), standard deviation, then the cousins IQR, MAD, CV, and the board's quartile deviation. The ten-number list $2, 3, 5, 6, 8, 10, 12, 15, 18, 20$ is the arithmetic spine. The thirty marks give the range a second home: $99 - 28 = 71$.

In Plain Words

Spread answers "how far from the pin do the rows live?" Range uses two rows. Variance uses every row, squared so that left and right cannot cancel. SD unsquares the unit. IQR and MAD survive outliers that set the range on fire. CV is a unitless ratio, legal only when a true zero exists.

10.1 What do the two ends already tell you?

From the class board

Range $= x_H - x_L$ (max minus min). Equivalent: $x_{(n)} - x_{(1)}$.

Ten numbers, then thirty marks

Ten from the board, already sorted:

$$x_H - x_L \;=\; 20 - 2 \;=\; 18.$$

On `it462_marks.csv`:

$$99 - 28 \;=\; 71.$$

Four students fail (S05 39, S09 28, S18 35, S29 31); the 28 is one end of that 71. The 99 is the other. Two people set the range. The other twenty-eight are silent.

Derivation — step by step

Range is a function of two order statistics. Change $x_{(n)}$ and the range changes by the same amount. That is honesty and fragility in one subtraction. Use it as a first glance (feasible range, IDA) and never as a last word.

How to write it in Python

```python
ten = [2, 3, 5, 6, 8, 10, 12, 15, 18, 20]
rng = max(ten) - min(ten)
assert rng == 18
import pandas as pd
df = pd.read_csv("it462_marks.csv")
assert int(df["marks"].max() - df["marks"].min()) == 71
```

10.2 Why square the red sticks?

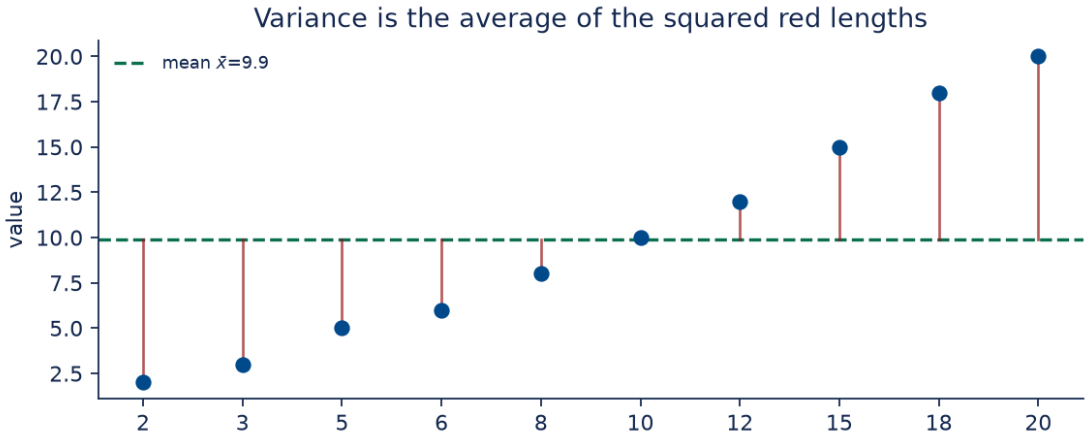


Figure — The ten board numbers. Teal line: mean 9.9. Each red stick is $x_i - \bar{x}$. Variance averages the squares of those sticks.

From the class board

Population variance $\sigma^2 = \frac{1}{N}\sum(x_i - \mu)^2$. Sample variance (ticked) $s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$. Standard deviation $= \sqrt{\text{variance}}$.

Every square of the ten numbers

$n = 10$, sum $= 2 + 3 + 5 + 6 + 8 + 10 + 12 + 15 + 18 + 20 = 99$, so $\bar{x} = 9.9$. Deviations and squares:

x_i	$x_i - 9.9$	$(x_i - 9.9)^2$	running sum of squares
2	−7.9	62.41	62.41
3	−6.9	47.61	110.02
5	−4.9	24.01	134.03
6	−3.9	15.21	149.24
8	−1.9	3.61	152.85
10	0.1	0.01	152.86
12	2.1	4.41	157.27
15	5.1	26.01	183.28
18	8.1	65.61	248.89
20	10.1	102.01	350.90

Sum of squares SS = 350.9.

$$\sigma^2 = \frac{350.9}{10} = 35.09, \qquad s^2 = \frac{350.9}{9} \approx 38.9889.$$

$$\sigma = \sqrt{35.09} \approx 5.924, \qquad s = \sqrt{38.9889} \approx 6.244.$$

Derivation — step by step

Raw deviations sum to zero — Chapter 7 proved it, and $-7.9 + \cdots + 10.1 = 0$ checks it on this list. Useless as a width. Squaring kills the sign: $(x_i - \bar{x})^2 \geq 0$. Averaging the squares is the definition.

If these ten numbers *are* the world, divide by $N = 10$: population variance. If they are a sample and $\bar{x}$ was estimated from them, divide by $n - 1 = 9$: sample variance. The board ticked the sample formula. SD is the square root, because 35.09 lives in squared units and nobody thinks in marks^2.

10.3 Why $n - 1$? One degree of freedom already spent

Derivation — step by step

Bessel's correction, slowly, on these ten numbers.

Step 1. The identity $\sum(x_i - \bar{x}) = 0$ is a constraint. If you know $\bar{x} = 9.9$ and nine of the ten values, the tenth is forced:

$$x_{10} = n\bar{x} - \sum_{i=1}^{9} x_i = 99 - (2 + 3 + 5 + 6 + 8 + 10 + 12 + 15 + 18) = 99 - 79 = 20.$$

You did not have ten free deviations. You had nine. The last stick is determined by the fulcrum you plugged in.

Step 2. $\bar{x}$ sits *inside* the sample. Deviations from $\bar{x}$ are a little too small compared with deviations from the unknown population mean μ. Dividing SS by n would therefore underestimate σ^2.

Step 3. For independent draws from a population with finite variance,

$$\mathbb{E}\left[\tfrac{1}{n-1}\sum(x_i - \bar{x})^2\right] = \sigma^2$$

and

$$\mathbb{E}\left[\tfrac{1}{n}\sum(x_i - \bar{x})^2\right] = \tfrac{n-1}{n}\sigma^2.$$

Dividing by $n - 1$ inflates s^2 just enough to cancel the bias. That is all "degrees of freedom $n - 1$" means here: one linear constraint, used up by replacing μ with $\bar{x}$.

On the ten numbers the inflation is visible: 35.09 versus 38.9889. Same SS, different honesty about whose mean you knew.

Another way to do it

`numpy.var` defaults to `ddof=0` (divide by n). `pandas.Series.var` defaults to `ddof=1` (divide by $n - 1$). On *this* list:

`np.var(ten)` = 35.09, `pd.Series(ten).var()` ≈ 38.9889.

Same squares. Different defaults. Always pass `ddof=` in mixed code. The board ticked pandas' default, not numpy's.

How to write it in Python

```
ten = [2.0, 3, 5, 6, 8, 10, 12, 15, 18, 20]
xbar = sum(ten) / len(ten)           # 9.9
ss = 0.0
for x in ten:
    d = x - xbar
    ss += d * d                      # every square, running to 350.9
assert abs(ss - 350.9) < 1e-9
pop = ss / 10
samp = ss / 9
import numpy as np, pandas as pd
assert abs(pop - np.var(ten, ddof=0)) < 1e-12
assert abs(samp - np.var(ten, ddof=1)) < 1e-12
assert abs(samp - pd.Series(ten).var()) < 1e-12
assert abs(samp ** 0.5 - 6.2441083341730135) < 1e-10
```

10.4 What width survives a wild row?

Concept

IQR. $Q_3 - Q_1$, the middle fifty percent (Chapter 10 for the recipe). On the ten numbers, class interpolation gives $Q_1 = 5.25$, $Q_3 = 14.25$, so IQR $= 9$.
MAD. Median absolute deviation from the median:

$$\text{MAD} \;=\; \text{median}_i\, |x_i - \tilde{x}|.$$

Ten numbers, $\tilde{x} = 9$ (average of 8 and 10). Absolute deviations: $7, 6, 4, 3, 1, 1, 3, 6, 9, 11$. Sorted: $1, 1, 3, 3, 4, 6, 6, 7, 9, 11$. Even n: average the 5th and 6th, $(4 + 6)/2 = 5$.
CV. For ratio-scale data with $\bar{x} \neq 0$,

$$\text{CV} \;=\; \frac{s}{\bar{x}}.$$

On the ten numbers $s \approx 6.244$, $\bar{x} = 9.9$, CV ≈ 0.631. Unitless, so you may compare spread of hours to spread of marks. Illegal for Celsius, because Celsius has no true zero and the ratio $s/\bar{x}$ would change if you renamed the zero.

From the class board

Quartile deviation (semi-interquartile range)

$$\text{QD} \;=\; \frac{Q_3 - Q_1}{2}.$$

QD on the ten numbers, class interpolation

We invert the board's percentile map $P = (i-1)/(n-1)$ (Chapter 10 owns the justification). For $p = 0.25$,

$$i \;=\; 1 + 0.25 \cdot 9 \;=\; 3.25.$$

$i = 3$ is 5, $i = 4$ is 6:

$$Q_1 \;=\; 5 + 0.25 \cdot (6 - 5) \;=\; 5.25.$$

For $p = 0.75$,

$$i \;=\; 1 + 0.75 \cdot 9 \;=\; 7.75.$$

$i = 7$ is 12, $i = 8$ is 15:

$$Q_3 \;=\; 12 + 0.75 \cdot (15 - 12) \;=\; 12 + 2.25 \;=\; 14.25.$$

$$\mathrm{QD} \;=\; \frac{14.25 - 5.25}{2} \;=\; \frac{9}{2} \;=\; 4.5.$$

If you ever see $Q_3 = 15.75$ and $\mathrm{QD} = 5.25$ on *this* list, someone used a $(n+1)$ recipe ($0.75 \cdot 11 = 8.25$, between 15 and 18) instead of $(n-1)$. Different school, different number. We stay with $(i-1)/(n-1)$.

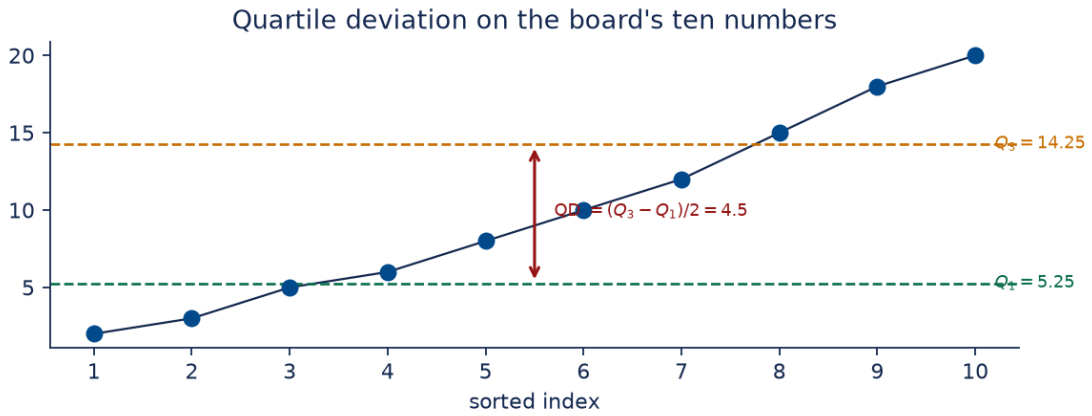


Figure — The ten board numbers. Teal $Q_1 = 5.25$, orange $Q_3 = 14.25$, red half-gap $\mathrm{QD} = 4.5$ — class interpolation, not the $(n+1)$ school.

How to write it in Python

```
q1 = 5 + 0.25 * (6 - 5)
q3 = 12 + 0.75 * (15 - 12)
qd = (q3 - q1) / 2
assert abs(q1 - np.quantile(ten, 0.25)) < 1e-12   # numpy default = type 7
assert abs(q3 - np.quantile(ten, 0.75)) < 1e-12
assert abs(qd - 4.5) < 1e-12
```

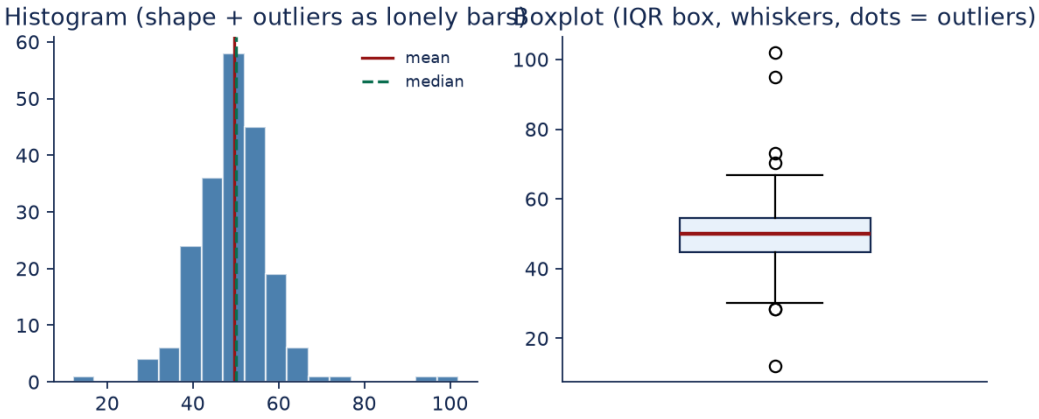


Figure — A histogram shows the shape variance cannot. A boxplot shows the IQR the range cannot. Use both on the same pile.

Watch Out

Ahmedabad pin codes run 380001 through 380061. Their "range" is 60. That subtraction treats a nominal (or at best ordinal) label as a ratio length. The digits are an addressing scheme, not a quantity. Range of zip codes is the same crime as the mean of zip codes. Legal dispersion for a code is: a count of distinct codes, a mode, a table. Not $x_H - x_L$.

Studio — now you try

9.1 Write the range of the ten board numbers and the range of the thirty marks. Why is "the class spans 71 marks" a poor description of a typical gap between two classmates?
9.2 On the short list $\{2, 3, 5, 6, 8\}$, write $\bar{x}$, then every $(x_i - \bar{x})^2$, then SS, then both variances and both SDs. Which `ddof` does `numpy.var` use if you forget to pass it, and which number on your page does that default match?
9.3 A file stores six-digit pin codes as integers. Compute their range anyway, then state the measurement-scale crime. Which of {range, s, IQR, mode-count} are still legal summaries of that column?

Chapter 11

Where Do You Sit in the Sorted Line?

Right now we are: explore — name a position, then invert it to a value

Lecture 7's third family is relative standing: percentiles, quantiles, quartiles, deciles, and the quartile deviation already computed in Chapter 9. The laboratory is the board's ten numbers $2, 3, 5, 6, 8, 10, 12, 15, 18, 20$ with $n = 10$. No mystery list.

In Plain Words

A percentile answers "what fraction of the ordered pile sits at or below this seat?" A quantile is the same idea on $[0, 1]$ instead of $[0, 100]$. Quartiles are the 25th, 50th, 75th percentiles. Deciles cut tenths. Inverting the map — "give me the 25th percentile as a *value*" — is interpolation when the seat is not an integer index. Different schools interpolate differently. We pick one, write it on the board, and then show what numpy does to the same ten numbers.

11.1 How does the board number a seat?

From the class board

Number line of length n. Percentile position of the ith ordered value:

$$P(x_i) = \frac{i-1}{n-1} \times 100 \qquad \text{and the alternative} \qquad \frac{i}{n} \times 100.$$

Data: $2, 3, 5, 6, 8, 10, 12, 15, 18, 20$, $n = 10$. Q1 (on the board): $P(12)$ with $i = 7 \to 67\%$. Q2: find the 25th percentile $\to i = 3.25$.

Label 12, then invert $P = 25$

Sorted, index i under the value:

i	1	2	3	4	5	6	7	8	9	10
$x_{(i)}$	2	3	5	6	8	10	12	15	18	20

$x = 12$ sits at $i = 7$:

$$P(12) = \frac{7-1}{10-1} \times 100 = \frac{6}{9} \times 100 = 66.\overline{6}\% \approx 67\%.$$

That is the board's Q1. Invert for $P = 25$:

$$i - 1 \;=\; 0.25 \cdot (10 - 1) \;=\; 2.25 \qquad \Rightarrow \qquad i \;=\; 3.25.$$

Seat 3.25 is a quarter of the way from $x_{(3)} = 5$ to $x_{(4)} = 6$:

$$P_{25} \;=\; 5 + 0.25 \cdot (6 - 5) \;=\; 5.25.$$

(The board stopped at $i = 3.25$. The interpolation is the step every computer then takes.) Same recipe at $P = 75$ gave $Q_3 = 14.25$ in Chapter 9.

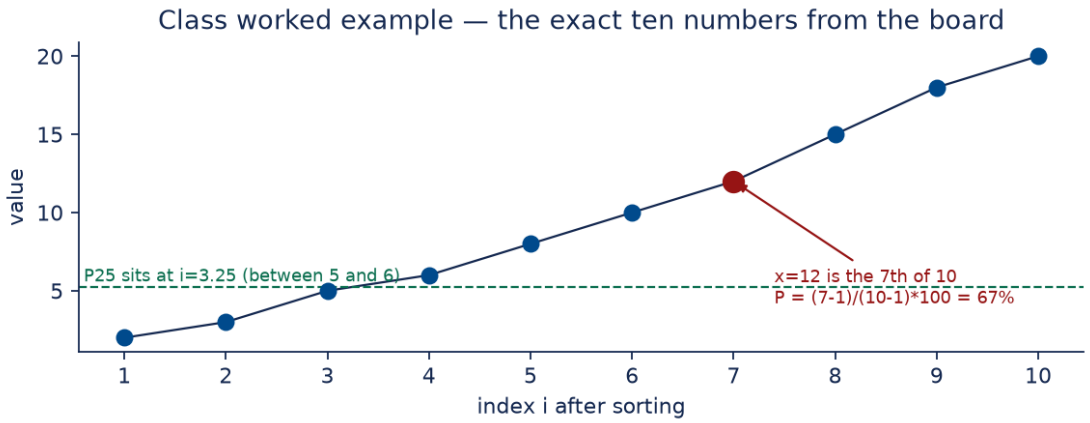


Figure — These exact ten numbers. Red: 12 is the 67th percentile. Teal: P_{25} lives between 5 and 6 at $i = 3.25$.

Derivation — step by step

The main formula sends the smallest value to 0% ($i = 1$) and the largest to 100% ($i = n$). It is the fraction of the way along the index range $[1, n]$. Inverting

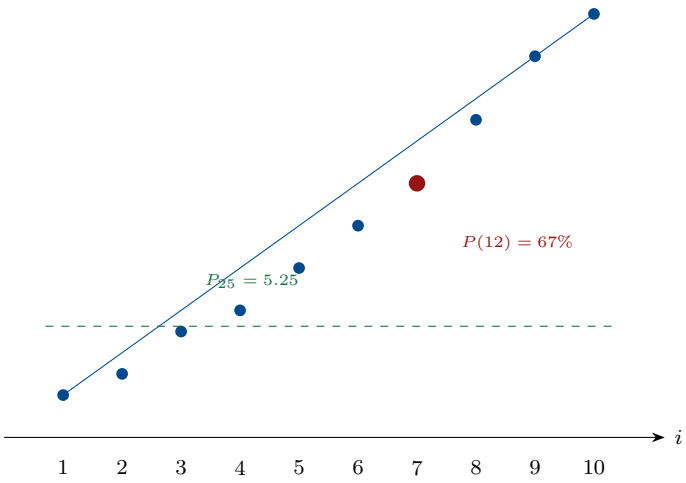
$$\frac{i - 1}{n - 1} = p \qquad \Rightarrow \qquad i = 1 + p(n - 1)$$

is Hyndman–Fan type 7's plotting position (named, not dumped, in the next section). If i is not an integer, linearly interpolate between the surrounding order statistics $x_{(\lfloor i \rfloor)}$ and $x_{(\lceil i \rceil)}$.

The alternative $i/n \times 100$ never sends the smallest to zero: on this list $i = 1$ would be 10%. For $x = 12$, $i = 7$,

$$\frac{7}{10} \times 100 \;=\; 70\%,$$

three points above the board. Same ten numbers. Different numbering of the seats.



11.2 What does numpy do to the same ten numbers?

Three recipes, one list

Let x be the ten numbers, already sorted.

recipe	what it answers	$x = 12$ or $p = 0.25$	number
board $(i-1)/(n-1)$	percentile of an observed $x_{(i)}$	$i = 7$	67%
alternative i/n	percentile of an observed $x_{(i)}$	$i = 7$	70%
`np.quantile` default	*value* at a given p	$p = 0.25$	5.25

Numpy's default quantile is not a third way of labelling 12. It is the inverse map: given p, return a value. At $p = 0.25$ it returns 5.25, which is exactly the board inversion plus interpolation. At $p = 2/3$ it returns 12 on the nose, because $i = 1 + (2/3) \cdot 9 = 7$. At $p = 0.67$ it returns 12.09, a hair past 12, because $i = 1 + 0.67 \cdot 9 = 7.03$. The board said 67% as a rounding of $66.\overline{6}$; numpy took 0.67 literally.

Hyndman–Fan, named not dumped

Hyndman & Fan (1996) catalogued nine sample-quantile definitions. They are *types* 1 through 9, not a menu you recite in a viva.

The one that matches the class board — $i = 1 + p(n-1)$, then linear interpolation — is **type 7**. Numpy and pandas default to type 7 (`method="linear"`). That is why `np.quantile(x, 0.25)` is 5.25 and `np.quantile(x, 0.75)` is 14.25 on this list.

Type 6 (Weibull, plotting position $p = i/(n+1)$) puts Q_3 at $0.75 \cdot 11 = 8.25$, between 15 and 18, and reports 15.75. Type 1 (inverse of the empirical CDF) reports $Q_1 = 5$, $Q_3 = 15$ with no interpolation. In a viva you name type 7, you compute 5.25 and 14.25, and you refuse to pretend the other eight types are the class formula.

How to write it in Python

```
x = [2.0, 3, 5, 6, 8, 10, 12, 15, 18, 20]
n, i = 10, 7
board = (i - 1) / (n - 1) * 100          # 66.66...  board said 67
alt = i / n * 100                        # 70.0

def quantile_type7(p):
```

```
    pos = 1 + p * (n - 1)                # i, 1-based
    lo = int(pos)                        # floor
    frac = pos - lo
    if frac == 0:
        return x[lo - 1]
    return x[lo - 1] + frac * (x[lo] - x[lo - 1])

import numpy as np
assert abs(board - 200 / 3) < 1e-12
assert abs(alt - 70.0) < 1e-12
assert abs(quantile_type7(0.25) - 5.25) < 1e-12
assert abs(quantile_type7(0.75) - 14.25) < 1e-12
assert abs(quantile_type7(0.25) - np.quantile(x, 0.25)) < 1e-12
assert abs(np.quantile(x, 2 / 3) - 12.0) < 1e-12
```

Watch Out

`np.percentile(x, 25)` and `np.quantile(x, 0.25)` are the same type 7 number. `np.percentile(x, 67)` is 12.09, not the board's "12 is at 67%". One call *labels a value*; the other *asks for a value at a percentage*. Mixing those two sentences is how two honest people get 67 versus 70 versus 12.09 and then argue. Write which map you used.

Studio — now you try

10.1 Using the class formula $(i-1)/(n-1)$, write the percentile of 8 in the ten-number list. Then invert: find the value of P_{40} by computing i and interpolating.
10.2 On the same ten numbers, a classmate reports "the percentile of 12 is 70". Another reports `np.quantile(x, 0.67) == 12.09`. You report 67%. Name the recipe each of you used, and say which two of you were answering different questions.
10.3 Name the Hyndman–Fan type that matches the class board. Compute Q_3 with that type. Then compute Q_3 with the $(n+1)$ (Weibull / type 6) plotting position. Why would two labs, both "correct", print 14.25 and 15.75 for the same list?

Chapter 12

When the Pile Leans, Peaks, or Goes Flat

Right now we are: explore — describe the shape the mean and SD cannot

Lecture 8 puts sketches on the board: a right tail, a bell with mean / median / mode stacked at the centre, a tent, a rectangle, and three kurtosis cartoons (lepto, meso, platy). The thirty marks in `it462_marks.csv` are the file we actually call `.skew()` and `.kurt()` on. A five-number toy $\{1, 2, 3, 4, 5\}$ is where Fisher and moment kurtosis are forced to show their working.

In Plain Words

Centre is a pin. Dispersion is a width. Shape is whether the pile is a mirror, whether it has a long tail, and whether the tails are heavier than a bell's. Skewness is the lean. Kurtosis is the tailedness (peakedness is a side-effect). A tent can be symmetric without being a bell. A uniform is symmetric and platykurtic. "Bell-shaped" is one fashion of symmetric, not a synonym for it.

12.1 Which way does the pile lean?

From the class board

Skewness (symmetry). Sketches: a right-tailed curve; a bell with Mean, Median, Mode stacked at the centre. Code: `df[''].skew()`.

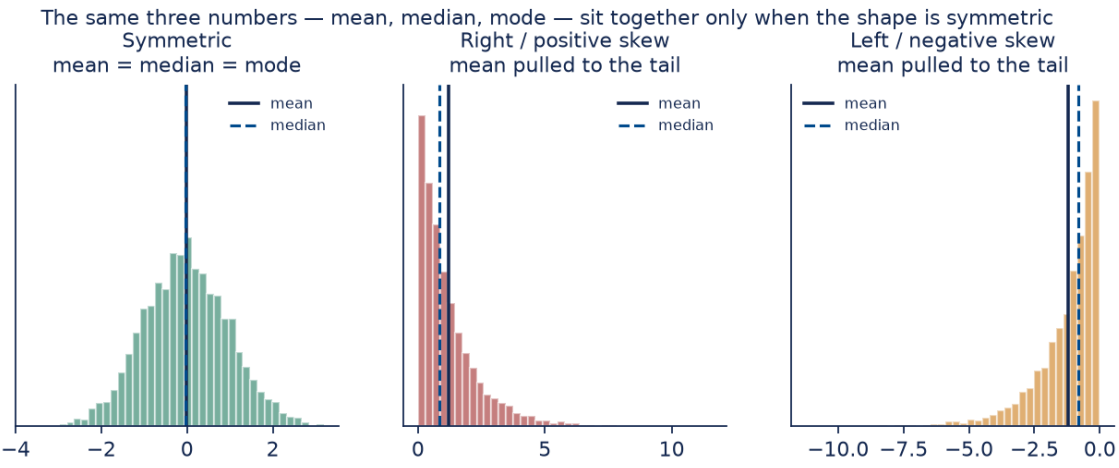


Figure — Symmetric: the three centres coincide. Right / positive skew: the mean is dragged into the long tail. Left / negative skew: the mirror image.

Pearson's median sketch, then the thirty marks

The cartoon on the board is a formula:

$$\frac{3(\bar{x} - \tilde{x})}{s}.$$

If the mean sits to the right of the median, the sign is positive — right skew. On `it462_marks.csv`, $\bar{x} = 63.5$, $\tilde{x} = 64$, sample $s \approx 19.595$:

$$\frac{3(63.5 - 64)}{19.595} \approx -0.077.$$

A hair of left skew, which is what "mean 63.5, median 64" already said in words. Pandas' Fisher–Pearson skew of the same column is

$$\texttt{df["marks"].skew()} \approx -0.075.$$

Two recipes, same verdict: almost symmetric, a slight lean toward the four fails (S05 39, S09 28, S18 35, S29 31).

Derivation — step by step

Cubes keep a sign. Pearson's moment coefficient is

$$g_1 = \frac{m_3}{s^3}, \qquad m_3 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3.$$

A long right tail makes $m_3 > 0$. Dividing by s^3 makes g_1 unitless, so the skew of kilograms can be compared to the skew of rupees. Fisher–Pearson, which is what `Series.skew()` returns, multiplies g_1 by $n/((n-1)(n-2))$ to adjust for sample size. On $n = 30$ the adjustment is small (-0.071 moment versus -0.075 Fisher). On $n = 5$ it would not be.

How to write it in Python

```python
import pandas as pd
import numpy as np
df = pd.read_csv("it462_marks.csv")
x = df["marks"].to_numpy(dtype=float)
xbar = sum(x) / len(x)
m3 = 0.0
for xi in x:
    d = xi - xbar
    m3 += d * d * d
m3 /= len(x)
s_pop = (sum((xi - xbar) ** 2 for xi in x) / len(x)) ** 0.5
g1 = m3 / (s_pop ** 3)
print(g1, df["marks"].skew())          # moment vs Fisher-Pearson
assert abs(df["marks"].skew() + 0.075) < 0.001
```

12.2 Is every symmetric pile a bell?

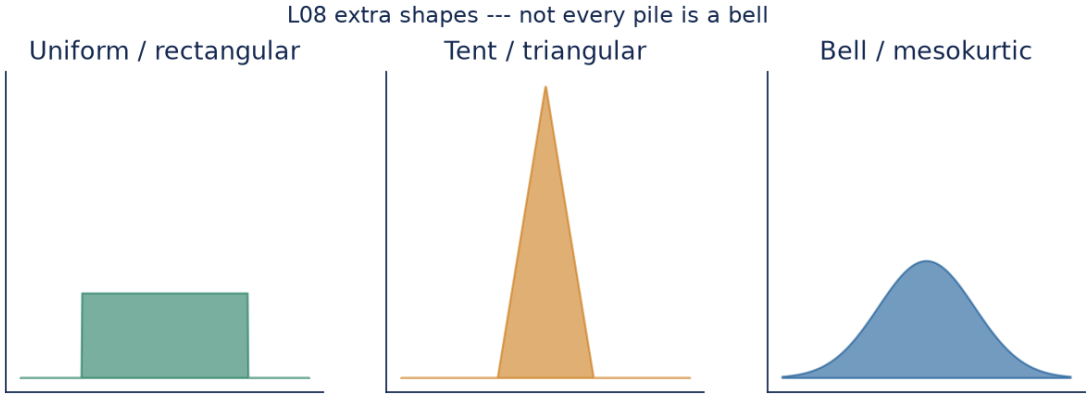


Figure — Lecture 8 extra shapes. Uniform / rectangular; tent / triangular; bell / mesokurtic. Only the right-hand panel is a Gaussian cartoon. The other two can still have skewness 0.

Concept

A **uniform** (rectangular pulse) on an interval puts equal mass on every value in that interval. Skewness 0, platykurtic (thin tails, the mass is the shoulders). A **tent** (triangular, symmetric) has a mode at the peak and straight sides down to zero; still skewness 0, still not a bell. A **bell** is mesokurtic: the Gaussian is the reference, not the universe. The board drew all three so that "symmetric" and "Gaussian" would stop being used as synonyms.

From the class board

If mean = median = mode, the distribution is symmetric and skewness = 0. **Positive (right) skew**: extreme values on the right of the x-axis, so the average increases. **Negative (left) skew**: extreme values on the left, so the average decreases. A normal distribution has skewness 0. *The converse is not true.*

Symmetric $\Rightarrow$ skew 0, but skew 0 $\not\Rightarrow$ symmetric

If a pile is a mirror image about its centre, every cube $(x_i - \bar{x})^3$ on the right is cancelled by an equal and opposite one on the left, so $m_3 = 0$ and the skewness is 0. The reverse fails: a lopsided pile can still have its cubes cancel *by accident*. A skewness of 0 is a necessary sign of symmetry, never a proof. Always look at the histogram too.

A lopsided column with skewness exactly 0

$x = [0, 0, 5, 5, 7, 10]$. Mean $\bar{x} = 27/6 = 4.5$, median = 5, modes 0 and 5 — the three centres disagree and the values are not a mirror image.

$x_i - \bar{x}$	−4.5	−4.5	0.5	0.5	2.5	5.5
$(x_i - \bar{x})^3$	−91.125	−91.125	0.125	0.125	15.625	166.375

$$\sum (x_i - \bar{x})^3 = 0.$$

So $m_3 = 0$ and `pd.Series(x).skew()` returns 0.0. Two large negative cubes on the left are balanced by one very large positive cube on the right: a short heavy left side against a long thin right side. Skewness 0, not symmetric. This is the student-notes sketch on the right of the page: "we show that the converse is not true".

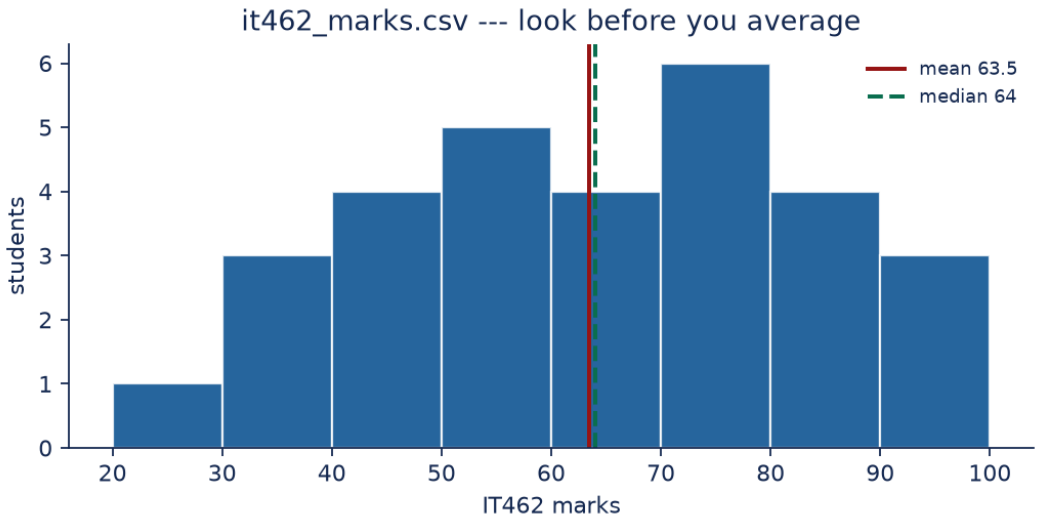


Figure — Thirty marks, ten-mark bins. Closer to a squat bell than to a tent or a rectangle — and the skew number above is why we are allowed to say that without squinting.

12.3 Are the tails heavier than a bell's?

From the class board

Kurtosis (tailedness / peakedness). Three bells: leptokurtic (tall), meso, platy (flat).

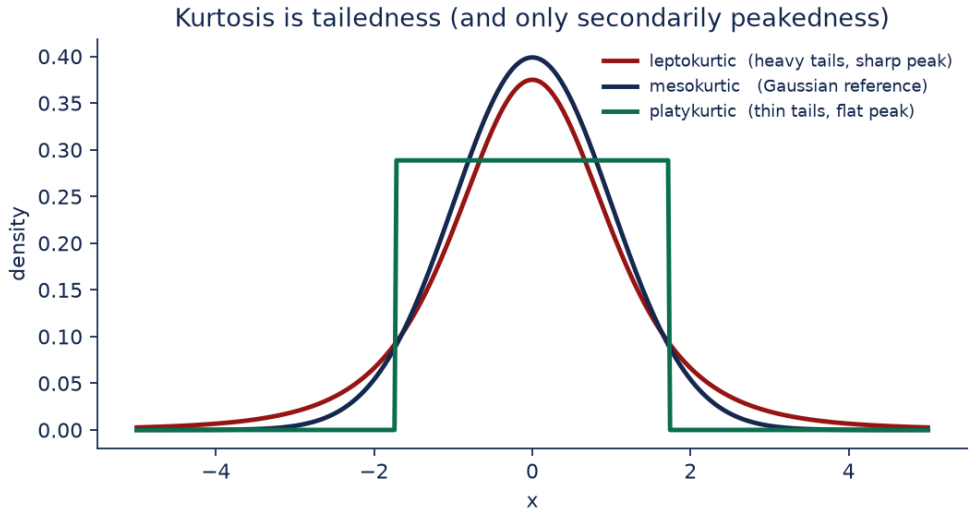


Figure — Lepto = heavy tails (and usually a sharp peak). Meso = Gaussian reference. Platy = thin tails, shoulder-y, like a uniform.

Derivation — step by step

The fourth standardised moment is

$$\text{Kurtosis} \;=\; \frac{m_4}{s^4}, \qquad m_4 \;=\; \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^4.$$

A Gaussian has kurtosis 3. Excess kurtosis subtracts that 3:

$$\kappa \;=\; \frac{m_4}{s^4} - 3,$$

so a perfect bell sits at 0. Signs, as the board named them:

- $\kappa > 0$ **leptokurtic** — outliers more common than a bell allows;
- $\kappa = 0$ **mesokurtic** — Gaussian-like tails;
- $\kappa < 0$ **platykurtic** — tails thinner than a bell.

Prefer the word *tails* when you speak. Peakedness comes along for the ride.

Marks file, then every fourth power of $\{1, 2, 3, 4, 5\}$

On `it462_marks.csv`,

$$\texttt{df["marks"].kurt()} \approx -0.943$$

(Fisher excess). Moment kurtosis of the same column is about 2.01, excess about -0.99. Platykurtic: the thirty marks do not have Gaussian tails. They live on $[0, 100]$ and they know it.
Now the tiny list $x = (1, 2, 3, 4, 5)$, $\bar{x} = 3$:

x_i	$x_i - 3$	$(x_i - 3)^2$	$(x_i - 3)^4$
1	-2	4	16
2	-1	1	1
3	0	0	0
4	1	1	1
5	2	4	16
sum	0	10	34

$$m_2 = \frac{10}{5} = 2, \qquad m_4 = \frac{34}{5} = 6.8, \qquad \frac{m_4}{m_2^2} = \frac{6.8}{4} = 1.7.$$

Moment excess $1.7 - 3 = -1.3$. Pandas Fisher excess on the same five numbers is -1.2. Two definitions, two numbers, one list — the next box.

Fisher versus moment, same five numbers

Moment kurtosis $m_4/m_2^2 = 1.7$ divides by n in every moment and does not subtract 3 until you ask for excess. Pandas `Series.kurt()` / `kurtosis()` returns **Fisher's** bias-corrected excess kurtosis, which for $n = 5$ is

$$\frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum \left(\frac{x_i - \bar{x}}{s}\right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)} = -1.2,$$

with $s^2 = \mathrm{SS}/(n-1) = 10/4 = 2.5$. Scipy's `kurtosis(..., fisher=False, bias=True)` returns the 1.7. Scipy's `fisher=True, bias=False` matches pandas. Assert all three against the table of fourth powers, not against a blog.

How to write it in Python

```
tiny = [1.0, 2, 3, 4, 5]
xbar = sum(tiny) / 5
ss = s4 = 0.0
for xi in tiny:
    d = xi - xbar
```

```
    ss += d * d
    s4 += d ** 4
m2, m4 = ss / 5, s4 / 5
moment_kurt = m4 / (m2 ** 2)                # 1.7
import pandas as pd
from scipy.stats import kurtosis
assert abs(moment_kurt - 1.7) < 1e-12
assert abs(kurtosis(tiny, fisher=False, bias=True) - 1.7) < 1e-12
assert abs(pd.Series(tiny).kurt() + 1.2) < 1e-12
print(pd.read_csv("it462_marks.csv")["marks"].kurt())
```

Watch Out

`Series.kurt()` is excess, Fisher, sample-adjusted. Printing it next to a textbook that says "Gaussian kurtosis equals 3" is how you invent a crisis of 3. Ask which 3 was subtracted. On marks, -0.94 already had the 3 removed; adding it back gets you near 2, not near a bell.

Studio — now you try

11.1 For the Lecture 1 scores $\{17, 41, 30, 99\}$, sketch where mean, median, and mode sit, then compute Pearson's median skewness $3(\bar{x} - \tilde{x})/s$. Which way does the pile lean, and which of the four numbers is doing the leaning?

11.2 Load `it462_marks.csv`, print `skew()` and `kurt()` of marks. Looking at `fig_marks_hist.png`, is the pile closer to tent, uniform, or bell? Support the claim with those two numbers, not with the word "looks".

11.3 On $\{1, 2, 3, 4, 5\}$ you now own every fourth power. Why is moment kurtosis 1.7 and pandas Fisher excess -1.2? Which of the two would you quote next to a textbook that says a Gaussian has kurtosis 3?

Part IV

Probability distributions

Chapter 13

Probability Distributions, and How to Compute Them in Python

So far every summary — mean, median, spread, shape — described the data *you have*. A **probability distribution** is the idealised shape the data is drawn *from*: the smooth curve a histogram is trying to be. Knowing the distribution lets you go beyond description and *compute*: the chance of a value above 115, the 97.5th percentile, how surprising an outlier really is. This chapter is a working catalogue — each distribution with its formula, its mean and spread, the exact `scipy.stats` calls, and a number worked by hand so the code is never a black box.

In Plain Words

A histogram is a distribution you *measured*; a probability distribution is a named, smooth shape you can *compute* with. Match your data to a named shape and you can answer "what fraction is below x?" or "what value sits at the 90th percentile?" without owning every future data point.

13.1 The four functions every distribution has

Whatever the distribution, `scipy.stats` gives it the same four buttons. Learn the buttons once and every distribution below is the same code with a different name.

What you want	Math	scipy call (e.g. `norm`)
Height of the curve at x	pmf / pdf $p(x)$	`norm.pdf(x)` (`.pmf` if discrete)
Probability up to x	cdf $F(x) = P(X \le x)$	`norm.cdf(x)`
Value at a given percentile	ppf $F^{-1}(q)$	`norm.ppf(0.9)`
Random samples	—	`norm.rvs(size=1000)`
Its mean / variance	$\mathbb{E}[X]$, $\mathrm{Var}(X)$	`norm.mean()`, `norm.var()`

pmf vs pdf, and why the cdf is the workhorse

Discrete variables (counts) have a **pmf** — $p(x)$ *is* the probability of exactly x. **Continuous** variables have a **pdf** — $p(x)$ is a *density*, and the probability of any exact value is 0; only *ranges* have probability. Either way, the **cdf** $F(x) = P(X \le x)$ answers real questions:

$$P(X > x) = 1 - F(x), \qquad P(a < X \le b) = F(b) - F(a).$$

The **ppf** is the cdf run backwards — feed it a percentile, get the value. That is exactly the percentiles of an earlier chapter, now for a *theoretical* curve.

13.2 Discrete distributions (counts)

13.2.1 Bernoulli — one yes/no

One trial with success probability p: $P(X{=}1) = p$, $P(X{=}0) = 1-p$. Mean p, variance $p(1-p)$. This is a single coin flip, a single "did the customer convert?"

13.2.2 Binomial — successes in n trials

Count of successes in n independent yes/no trials, each with probability p:

$$P(X{=}k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad \mathbb{E}[X] = np, \quad \mathrm{Var}(X) = np(1-p).$$

10 questions, guessing, $p = 0.3$

$n = 10$, $p = 0.3$, so $\mathbb{E}[X] = np = 3$ correct on average. Probability of exactly 3:

$$P(X{=}3) = \binom{10}{3}(0.3)^3(0.7)^7 = 120 \cdot 0.027 \cdot 0.0824 = \boxed{0.2668}.$$

Probability of at most 3: $P(X \le 3) = \sum_{k=0}^{3} P(X{=}k) = \boxed{0.6496}$ — add the four terms, or just call the cdf.

How to write it in Python

```
from scipy.stats import binom
binom.pmf(3, 10, 0.3)      # 0.2668  P(X=3)
binom.cdf(3, 10, 0.3)      # 0.6496  P(X<=3)
binom.mean(10, 0.3)        # 3.0
binom.rvs(10, 0.3, size=5)# e.g. array([2, 4, 3, 1, 3]) -- fake data
```

13.2.3 Poisson — counts of rare events

Number of events in a fixed window when they happen at a steady average rate λ: calls per hour, typos per page, defects per batch:

$$P(X{=}k) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad \mathbb{E}[X] = \mathrm{Var}(X) = \lambda.$$

$\lambda = 2$ arrivals per minute

$P(X{=}0) = e^{-2} = \boxed{0.1353}$ (a quiet minute); so the chance of *at least one* arrival is $1 - 0.1353 = \boxed{0.8647}$. Busiest single value: $P(X{=}3) = \frac{2^3 e^{-2}}{6} = 0.1804$. Telltale sign of a Poisson: the mean and variance are equal ($= \lambda = 2$).

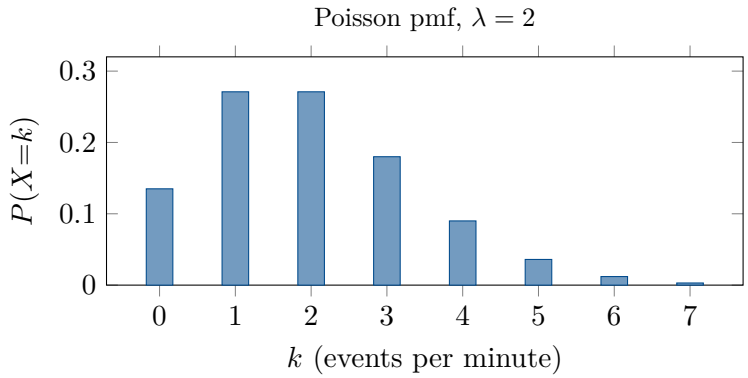


Figure — The Poisson piles its mass near $\lambda = 2$ and tails off. Counts can never be negative, so the shape is right-skewed for small λ.

How to write it in Python

```
from scipy.stats import poisson
poisson.pmf(0, 2)    # 0.1353
1 - poisson.pmf(0, 2)  # 0.8647  P(X>=1)
poisson.mean(2), poisson.var(2)    # (2.0, 2.0)  <- equal, the
    Poisson fingerprint
```

13.3 Continuous distributions (measurements)

13.3.1 Uniform — every value equally likely

$X \sim U(a, b)$: flat pdf $\frac{1}{b-a}$ on $[a, b]$, mean $\frac{a+b}{2}$, variance $\frac{(b-a)^2}{12}$. The "no information" distribution — a random spin of a dial.

13.3.2 Normal (Gaussian) — the bell, and the star of EDA

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(- \frac{(x-\mu)^2}{2\sigma^2} \Big), \qquad \mathbb{E}[X] = \mu, \quad \mathrm{Var}(X) = \sigma^2.$$

So many measurements (heights, exam scores, measurement errors) pile up in this symmetric bell that it earns its own rules of thumb.

From the class board

The 68–95–99.7 rule. For *any* normal, about 68% of values fall within 1σ of the mean, 95% within 2σ, and 99.7% within 3σ. A value beyond 3σ happens roughly 3 times in 1000 — which is why "3-sigma" is shorthand for "rare enough to investigate."

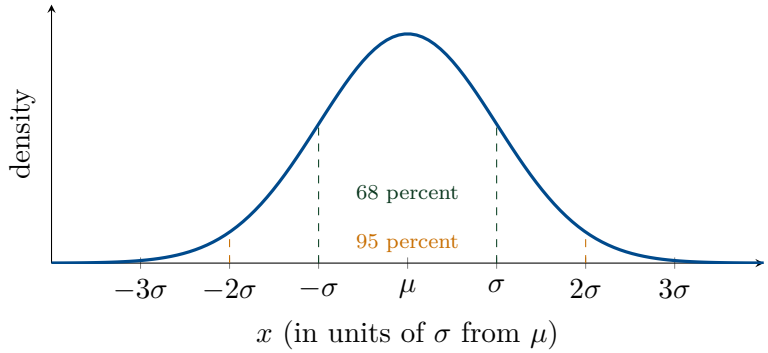


Figure — The standard normal. The $\pm 1\sigma$ band holds 68%, the $\pm 2\sigma$ band 95%. A z-score $z = (x - \mu)/\sigma$ just relabels the axis in these σ-units.

IQ-style scores, $\mu = 100$, $\sigma = 15$

Fraction below 115 ($= \mu + 1\sigma$): $P(X < 115) = \Phi(1) = \boxed{0.8413}$. Fraction between 85 and 115 ($\mu \pm 1\sigma$): $0.8413 - 0.1587 = \boxed{0.6827}$ — the 68% rule. The 97.5th percentile: $\mu + 1.96\sigma = 100 + 1.96(15) = \boxed{129.4}$ (this is where `ppf(0.975)` lands).

How to write it in Python

```
from scipy.stats import norm
norm.cdf(115, 100, 15)                    # 0.8413  P(X<115)
norm.cdf(115,100,15) - norm.cdf(85,100,15)  # 0.6827  the 68% band
norm.ppf(0.975, 100, 15)                  # 129.4   the 97.5th
    percentile
z = (115 - 100) / 15                      # 1.0     z-score: "1 sigma
    above"
```

13.3.3 Exponential — waiting times

Time until the next event when events arrive at rate λ (the continuous cousin of the Poisson): $p(x) = \lambda e^{-\lambda x}$ for $x \geq 0$, mean $1/\lambda$, variance $1/\lambda^2$. Right-skewed with a long tail — most waits short, a few long.

Buses average every 10 minutes

Mean wait $1/\lambda = 10$, so $\lambda = 0.1$. Chance of waiting under 5 minutes: $P(X < 5) = 1 - e^{-\lambda \cdot 5} = 1 - e^{-0.5} = \boxed{0.3935}$.

How to write it in Python

```
from scipy.stats import expon
expon.cdf(5, scale=10)      # 0.3935   scale = mean = 1/lambda
expon.mean(scale=10)        # 10.0
```

In Plain Words

Log-normal deserves a mention: incomes, city sizes, and file sizes are often log-normal — normal *after* taking logs. If a histogram is heavily right-skewed, try `np.log(x)` and see if the bell appears.

13.4 Fitting a named shape to real data

EDA rarely tells you the distribution — you *guess* it from the histogram and then check. `scipy.stats` fits parameters by maximum likelihood and lets you overlay the fitted curve on the histogram.

How to write it in Python

```python
import numpy as np, pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

x = pd.read_csv("it462_marks.csv")["marks"].dropna().values
mu, sigma = stats.norm.fit(x)          # MLE: mu = mean, sigma = std
grid = np.linspace(x.min(), x.max(), 200)
plt.hist(x, bins=20, density=True, alpha=0.5)   # empirical shape
plt.plot(grid, stats.norm.pdf(grid, mu, sigma)) # fitted bell on top
# quick sanity check that the fit is normal-ish:
stats.probplot(x, dist="norm", plot=plt)        # a Q-Q plot: points
    on the line = good fit
```

`density=True` rescales the histogram to area 1 so it is directly comparable to a pdf. A **Q–Q plot** (quantile–quantile) puts the data's quantiles against the theoretical ones; points hugging the diagonal mean the shape matches.

Watch Out

Fitting always "succeeds" — `norm.fit` returns a μ and σ even for data shaped nothing like a bell. Fitting a curve is not evidence the curve is right. Always look at the overlay or the Q–Q plot; the mean/variance rule of the Poisson and the log-trick for log-normal are cheap sanity checks.

13.5 One reference table

Distribution	Type	Mean	Variance	scipy name
Bernoulli(p)	discrete	p	$p(1-p)$	`bernoulli`
Binomial(n,p)	discrete	np	$np(1-p)$	`binom`
Poisson(λ)	discrete	λ	λ	`poisson`
Uniform(a,b)	continuous	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	`uniform`
Normal(μ,σ)	continuous	μ	σ^2	`norm`
Exponential(λ)	continuous	$1/\lambda$	$1/\lambda^2$	`expon`

Chai-Break

Every distribution in `scipy.stats` answers to the same four verbs — `pdf`, `cdf`, `ppf`, `rvs`. Learn them once and switching from `norm` to `poisson` is like switching cuisines but keeping the same spoon.

Studio — now you try

16.1 A quiz has 20 true/false questions answered by pure guessing ($p = 0.5$). Using `binom`, compute the expected score, $P(X{=}10)$, and $P(X \geq 15)$. Is scoring 15+ by luck common or rare?

16.2 A help desk gets $\lambda = 4$ calls per hour. By hand find $P(X{=}0)$ and $P(X \geq 1)$ in an hour; then use `poisson.cdf` for $P(X \leq 2)$. Confirm mean = variance.

16.3 Heights are Normal($\mu{=}170, \sigma{=}10$) cm. Find the fraction above 185, the fraction between 160 and 180, and the 90th-percentile height. Do it once with the 68–95–99.7 rule and once with `norm.cdf`/`norm.ppf`; do they agree?

16.4 Load `it462_marks.csv`, plot a density histogram of `marks`, fit a normal with `norm.fit`, and overlay its pdf. From the overlay and a Q–Q plot, argue in one sentence whether "marks are normal" is defensible — using the picture, not the word "looks".
16.5 Right-skewed data: take `expon.rvs(scale=10, size=1000)`, histogram it, then histogram `np.log` of it. Which one looks more bell-shaped, and what does that tell you about when to reach for a log transform?

Part V

Bivariate association

Chapter 14

When Two Columns Move Together

Right now we are: explore — name the association the scatter has earned

Lecture 8's last family is relationship: linear versus not, Pearson, Spearman, point-biserial, Cramér ϕ/V. Four mini-tables do all the arithmetic. The thirty-row gender×passed crosstab of `it462_marks.csv` is the fourth, balanced on purpose. Looking is not optional. Anscombe and a boomerang exist so that `df.corr()` cannot finish the chapter alone.

In Plain Words

Association asks whether Y tends to be large when X is large. Pearson asks that question of a straight line. Spearman asks it of ranks (monotone, not straight). Point-biserial is Pearson when one column is 0/1. Cramér starts from a crosstab of names. $r \approx 0$ means "no linear story". It does not mean "no story".

14.1 Four tiny tables, four different stories

linear +		linear −		parabola		gender × passed	
x	y	x	y	x	y		yes / no
1	2	1	8	−2	4	F	13 / 2
2	4	2	6	−1	1	M	13 / 2
3	6	3	4	0	0		
4	8	4	2	1	1		
				2	4		

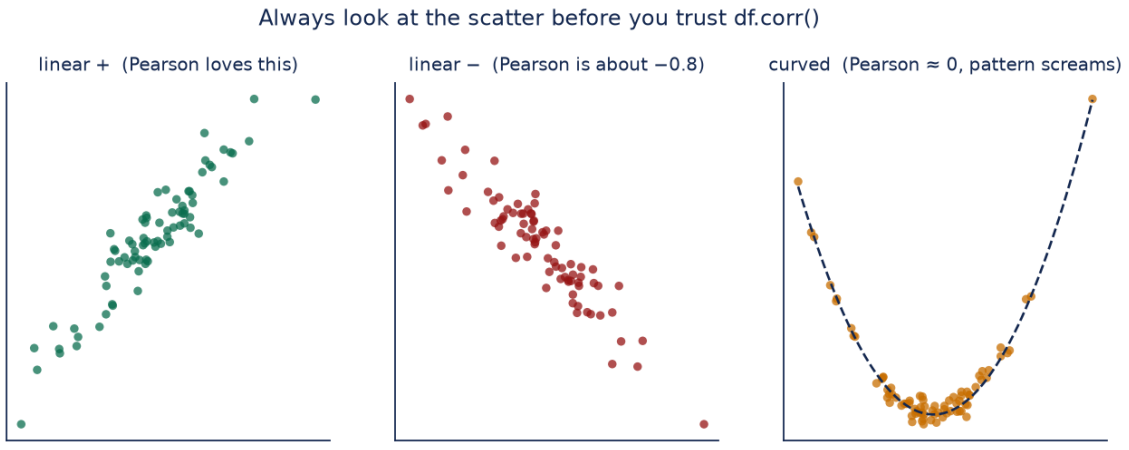


Figure — Left: Pearson's happy place. Middle: Pearson still works (negative). Right: Pearson takes a nap while the curve shouts.

14.2 How much straight line is in the cloud?

From the class board

Interval / ratio → Pearson r. Scatter with a positive ellipse; a negative ellipse; Pearson drawn on a boomerang, with `df.corr()`.

Every product on the rising line, then the parabola

Rising line $(x, y) = (1, 2), (2, 4), (3, 6), (4, 8)$. $\bar{x} = 2.5$, $\bar{y} = 5$.

x	y	$x - \bar{x}$	$y - \bar{y}$	product
1	2	-1.5	-3	4.5
2	4	-0.5	-1	0.5
3	6	0.5	1	0.5
4	8	1.5	3	4.5
		$\mathrm{SS}_x = 5$	$\mathrm{SS}_y = 20$	$\sum = 10$

$$r = \frac{10}{\sqrt{5 \cdot 20}} = \frac{10}{10} = 1.$$

Falling line $(1, 8), (2, 6), (3, 4), (4, 2)$: y-deviations flip sign, products sum to -10, $r = -1$. Parabola $(-2, 4), (-1, 1), (0, 0), (1, 1), (2, 4)$: $\bar{x} = 0$, $\bar{y} = 2$, products $(-2)(2)+(-1)(-1)+(0)(-2)+(1)(-1)+(2)(2) = -4+1+0-1+4 = 0$, so $r = 0$. A perfect U, a perfect zero. Pearson asked for a line. The U refused.

Derivation — step by step

Covariance is variance with a partner:

$$\mathrm{cov}(X, Y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y})$$

(sample; divide by n in population-mode). Pearson rescales to $[-1, 1]$:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}.$$

The $n - 1$ cancels. $r = 1$ is a rising line, $r = -1$ a falling line, $r = 0$ is no *linear* association. Cauchy–Schwarz forbids $|r| > 1$. The boomerang on the board is the $r \approx 0$ trap, and the parabola above is that trap with five points you can multiply by hand.

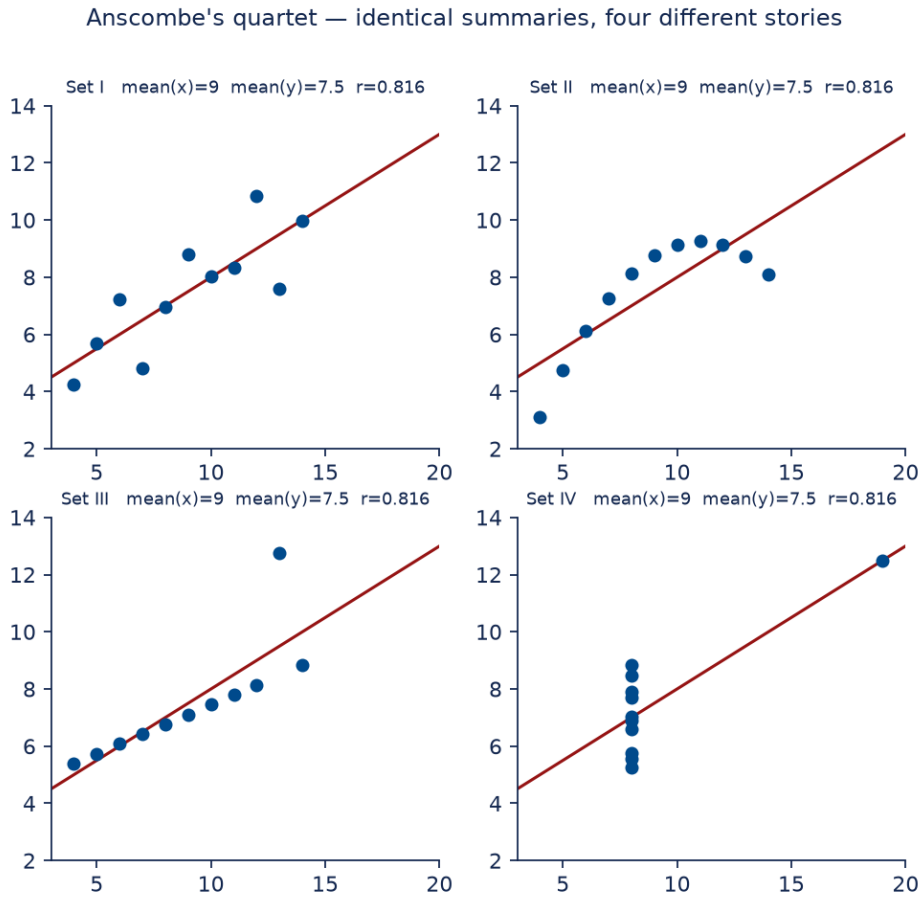


Figure — Anscombe 1973: four sets, identical means, variances, and $r \approx 0.816$. Four different pictures. The board's boomerang is the same sermon with one panel.

How to write it in Python

```
def pearson(xs, ys):
    n = len(xs)
    xb = sum(xs) / n
    yb = sum(ys) / n
    num = ssx = ssy = 0.0
    for x, y in zip(xs, ys):
        dx, dy = x - xb, y - yb
        num += dx * dy
        ssx += dx * dx
        ssy += dy * dy
    return num / (ssx * ssy) ** 0.5

xp, yp = [1, 2, 3, 4], [2, 4, 6, 8]
xn, yn = [1, 2, 3, 4], [8, 6, 4, 2]
xa, ya = [-2, -1, 0, 1, 2], [4, 1, 0, 1, 4]
import numpy as np
from scipy import stats as ss
assert abs(pearson(xp, yp) - 1) < 1e-12
assert abs(pearson(xn, yn) + 1) < 1e-12
assert abs(pearson(xa, ya) - 0) < 1e-12
assert abs(pearson(xp, yp) - np.corrcoef(xp, yp)[0, 1]) < 1e-12
assert abs(ss.pearsonr(xa, ya).statistic - 0) < 1e-12
```

14.3 What if we only believe the order?

From the class board

Ordinal → Spearman r_s. Ranks, then Pearson. No-tie shortcut

$$r_s = 1 - \frac{6\sum d_i^2}{n(n^2-1)}.$$

Code: `ss.spearmanr`.

Ranks of the falling line

$(x, y) = (1, 8), (2, 6), (3, 4), (4, 2)$. Ranks of x: $1, 2, 3, 4$. Ranks of y: $4, 3, 2, 1$. Differences $d_i = R(x_i) - R(y_i)$:

$$d = (-3, -1, 1, 3), \qquad \sum d_i^2 = 9 + 1 + 1 + 9 = 20.$$

No ties, so the shortcut is legal: $r_s = 1 - 6\sum d_i^2/(n(n^2-1)) = 1 - 6 \cdot 20/(4 \cdot 15) = 1 - 120/60 = -1$. Pearson on the ranks is the same -1. Rising line: ranks match, $r_s = 1$. Parabola: y-ranks $4.5, 2.5, 1, 2.5, 4.5$ against x-ranks $1, 2, 3, 4, 5$, and $r_s = 0$ — monotone-none, just as Pearson was linear-none.

Derivation — step by step

Spearman is Pearson after each column is replaced by its rank (average ranks for ties). It asks for a monotone relationship. A strictly increasing log-curve has $|r| < 1$ and $r_s = 1$. That is the whole point of walking through ranks.

How to write it in Python

```
d2 = 0.0
rx, ry = [1, 2, 3, 4], [4, 3, 2, 1]
for a, b in zip(rx, ry):
    d2 += (a - b) ** 2
rs = 1 - 6 * d2 / (4 * (16 - 1))
assert abs(rs + 1) < 1e-12
assert abs(rs - ss.spearmanr(xn, yn).statistic) < 1e-12
assert abs(ss.spearmanr(xa, ya).statistic - 0) < 1e-12
```

14.4 What if one column is a yes/no?

From the class board

Y continuous, X binomial → point-biserial. Code: `ss.pointbiserialr`.

$$r_{pb} = \frac{\bar{y}_1 - \bar{y}_0}{s_Y}\sqrt{pq}.$$

Four marks and a 0/1 pass, then the file

Toy: $Y = (2, 4, 6, 8)$, $X = (0, 0, 1, 1)$. Group means $\bar{y}_0 = 3$, $\bar{y}_1 = 7$, $p = q = 1/2$. Population SD of Y: $s_Y = \sqrt{((2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2)/4} = \sqrt{20/4} = \sqrt{5}$. Then $r_{pb} = (\bar{y}_1 - \bar{y}_0)/s_Y \cdot \sqrt{pq} = 4/\sqrt{5} \cdot 1/2 = 2/\sqrt{5} \approx 0.8944$. This is Pearson of the 0/1 column with Y (products sum to 4, $\mathrm{SS}_X = 1$, $\mathrm{SS}_Y = 20$, $4/\sqrt{20} = 2/\sqrt{5}$). Scipy returns the same 0.8944.

On `it462_marks.csv`, passed yes = 1 ($n_1 = 26$, $\bar{y}_1 \approx 68.15$) versus no ($n_0 = 4$ fails $39, 28, 35, 31$, $\bar{y}_0 = 33.25$) gives $r_{pb} \approx 0.616$. The binary column is a thresholded copy of marks, so a large r_{pb} is almost a tautology — still the right coefficient, still not a discovery.

How to write it in Python

```python
import pandas as pd
from scipy import stats as ss
Y, X = [2.0, 4, 6, 8], [0, 0, 1, 1]
r_pb, p_pb = ss.pointbiserialr(X, Y)
assert abs(r_pb - pearson(X, Y)) < 1e-12
assert abs(r_pb - 2 / 5 ** 0.5) < 1e-12
df = pd.read_csv("it462_marks.csv")
passed = (df["passed"] == "yes").astype(int)
print(ss.pointbiserialr(passed, df["marks"]))
```

14.5 What if both columns are names?

From the class board

Binomial × binomial → Cramér ϕ / V. Code: crosstab, then χ^2.

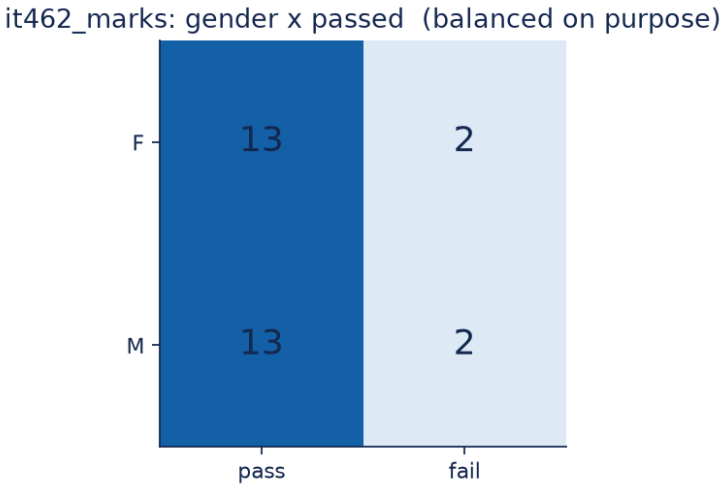


Figure — `it462_marks.csv`: gender × passed. 13 / 2 on both rows, by design.

Every expected count on the class 2×2

Observed:

	yes	no
F	13	2
M	13	2

$n = 30$, row totals $15, 15$, column totals $26, 4$. Expected $E_{rc} = (\text{row total} \times \text{col total})/n$:

$$E_{\text{F,yes}} = \frac{15 \cdot 26}{30} = 13, \quad E_{\text{F,no}} = \frac{15 \cdot 4}{30} = 2,$$

and the M row is identical. Every $O = E$, so

$$\chi^2 = \sum \frac{(O - E)^2}{E} = 0.$$

For a 2×2, $\phi = \sqrt{\chi^2/n} = 0$. For general $R \times C$,

$$V = \sqrt{\frac{\chi^2}{n \cdot \min(R - 1, C - 1)}},$$

which here is also 0. Gender and passing are independent in this file because we built them that way: two fails among the fifteen women (S05, S29) and two among the fifteen men (S09, S18).

Another way to do it

A 2×2 that is *not* balanced, so the formula has something to chew: $3, 1$ on the first row and $1, 3$ on the second. Every $E = 2$, $\chi^2 = 2$, $\phi = \sqrt{2/8} = 0.5$. Same definition, a nonzero number. Scipy's `ss.contingency.association(ct, method="cramer")` on the class table returns 0.0; on this toy it returns 0.5.

How to write it in Python

```
import pandas as pd
df = pd.read_csv("it462_marks.csv")
ct = pd.crosstab(df["gender"], df["passed"])
chi2, p, dof, exp = ss.chi2_contingency(ct, correction=False)
n = ct.to_numpy().sum()
phi = (chi2 / n) ** 0.5
V = (chi2 / (n * (min(ct.shape) - 1))) ** 0.5
assert abs(phi) < 1e-12 and abs(V) < 1e-12
assert abs(ss.contingency.association(ct, method="cramer") - 0) < 1e
    -12
print(ct)
print(exp)
```

Same test through a statsmodels contingency table

The board also mentioned *contingency table in statsmodels*. It wraps the crosstab and gives the χ^2 test, expected counts and (for 2×2) the odds ratio.

```python
import statsmodels.api as sm
toy = pd.DataFrame([[3, 1], [1, 3]], index=["F", "M"], columns=[0, 1])
table = sm.stats.Table(toy)
res = table.test_nominal_association()
print(res.statistic, res.pvalue, res.df)      # 2.0  0.1573  1   (same chi2 as scipy)
print(table.fittedvalues)                      # expected counts, all 2.0
print(sm.stats.Table2x2(toy.to_numpy()).oddsratio)   # (3*3)/(1*1) = 9.0
```

From the class board

Which coefficient? (student notes) interval / ratio × interval / ratio → Pearson r (`df.corr()`); ordinal (category) → Spearman (`ss.spearmanr(x, y)`); numeric y × binomial x → point-biserial (`ss.pointbiserialr(x, y)`); binomial × binomial (or nominal × nominal) → Cramér ϕ / V (crosstab, then χ^2).

Watch Out

The students' page groups *nominal* with ordinal under Spearman. Spearman needs an *order*, so it is legal for ordinal columns only. A nominal column (city, blood group) has no order to rank; use Cramér's V for nominal × nominal, and compare group means (or the correlation ratio η) for nominal × numeric.

Watch Out

`df.corr()` silently skips nominal columns and will Pearson anything stored as a number — Likert codes you did not mean to treat as interval, a 0/1 you should have handed to point-biserial by name, a parabola it will cheerfully call $r = 0$. Look at the scatter. Then pick the coefficient the board mapped. Correlation is not causation; it is not even "I looked".

Studio — now you try

12.1 Pearson by hand on $(1,2),(2,4),(3,6),(4,8)$: write $\bar{x}$, $\bar{y}$, every product, SS_x, SS_y, and r. Then on the parabola $(-2,4),(-1,1),(0,0),(1,1),(2,4)$. What did r miss the second time that a scatter would not?

12.2 Spearman by the no-tie shortcut on the falling line $(1,8),(2,6),(3,4),(4,2)$. Then confirm with `ss.spearmanr`. Why is $|r_s| = 1$ here even though a viva-neighbour might call the relationship "just Pearson with extra steps"?

12.3 From the gender × passed table, write every expected count, χ^2, and ϕ. In one sentence, what was "balanced on purpose" for? (If ϕ had been far from zero on this invented file, what would you have accused the author of?)

Chapter 15

Covariance and Correlation, in Depth

The last chapter measured association with r and its rank cousins. But `df.corr()` hides the engine underneath: **covariance**, the raw "do these two move together" number, and the **covariance matrix** that scales the whole idea to many columns at once. This chapter opens that engine — covariance by hand, why we *normalise* it into correlation, the covariance and correlation *matrices* you will actually read in a project, Kendall's τ as a third correlation, and the traps (causation, outliers, Simpson) that turn a strong r into a wrong story.

In Plain Words

Covariance answers "when X is above its mean, is Y usually above *its* mean too?" — in the raw units of the data. Correlation is that same answer divided by the two spreads, so it lands cleanly in $[-1, 1]$ and is comparable across any pair of columns.

15.1 Covariance from scratch

For two columns X, Y with means $\bar{x}, \bar{y}$, the **sample covariance** is

$$\mathrm{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}).$$

Read the product $(x_i - \bar{x})(y_i - \bar{y})$: it is *positive* when both deviations share a sign (both above, or both below, their means) and *negative* when they disagree. Sum them up and the sign of the total tells you the direction of the relationship; its size depends on the units.

Hours studied vs score, every product

X = hours $= [1, 2, 3, 4, 5]$, Y = score $= [2, 4, 5, 4, 5]$. Means $\bar{x} = 3$, $\bar{y} = 4$.

$x_i - \bar{x}$	$y_i - \bar{y}$	product
-2	-2	4
-1	0	0
0	1	0
1	0	0
2	1	2
		$\sum = 6$

$\mathrm{Cov}(X,Y) = \dfrac{6}{5-1} = \boxed{1.5}$. Positive — more hours goes with higher scores. (For later: $\mathrm{SS}_x = \sum(x_i - \bar{x})^2 = 10$, $\mathrm{SS}_y = 6$.)

Watch Out

Covariance has ugly units. Here it is "1.5 hour-marks". Measure the hours in *minutes* and every deviation is $\times 60$, so the covariance jumps to 90 — same relationship, sixty times the number. Covariance tells you the *direction* reliably but its *magnitude* is unreadable on its own. That is the whole reason correlation exists.

15.2 From covariance to correlation

Divide the covariance by both standard deviations and the units cancel, leaving a pure number in $[-1, 1]$ — **Pearson's** r:

$$r = \frac{\mathrm{Cov}(X,Y)}{s_x\, s_y} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\mathrm{SS}_x\, \mathrm{SS}_y}}.$$

The same two columns, normalised

$s_x = \sqrt{10/4} = 1.581$, $s_y = \sqrt{6/4} = 1.225$. Then

$$r = \frac{1.5}{1.581 \times 1.225} = \frac{1.5}{1.936} = \boxed{0.7746},$$

or straight from the raw sums $r = \dfrac{6}{\sqrt{10 \cdot 6}} = \dfrac{6}{\sqrt{60}} = 0.7746$. Now the minutes-vs-hours problem is gone: rescaling X multiplies the top and bottom by the same 60, so r does not move. That invariance is exactly what makes r comparable across columns.

How to write it in Python

```
import numpy as np, pandas as pd
X = [1,2,3,4,5]; Y = [2,4,5,4,5]
np.cov(X, Y, ddof=1)[0,1]      # 1.5      sample covariance
np.corrcoef(X, Y)[0,1]         # 0.7746   Pearson r
pd.Series(X).corr(pd.Series(Y))              # 0.7746   (pandas, Pearson by default)
pd.Series(X).corr(pd.Series(Y), method="spearman")   # 0.7379   rank correlation
```

15.3 Many columns at once: the covariance and correlation matrices

Real EDA has many numeric columns. Stack the pairwise covariances into a **covariance matrix** Σ: entry (j,k) is $\mathrm{Cov}(\mathrm{col}_j, \mathrm{col}_k)$. It is *symmetric* ($\mathrm{Cov}(X,Y) = \mathrm{Cov}(Y,X)$) and its *diagonal is the variances* ($\mathrm{Cov}(X,X) = \mathrm{Var}(X)$). Normalise every entry by the two standard deviations and you get the **correlation matrix**: symmetric, 1s on the diagonal, every entry in $[-1, 1]$.

Three columns: hours, score, sleep

hours $= [1,2,3,4,5]$, score $= [2,4,5,4,5]$, sleep $= [8,7,6,7,5]$. One entry by hand — hours vs sleep ($\bar{\text{sleep}} = 6.6$): deviations of sleep are $[1.4, 0.4, -0.6, 0.4, -1.6]$, products with hours' deviations $[-2,-1,0,1,2]$ are $[-2.8, -0.4, 0, 0.4, -3.2]$, summing to -6, so $\text{Cov} = -6/4 = -1.5$. The full matrices:

$$\Sigma = \begin{bmatrix} 2.5 & 1.5 & -1.5 \\ 1.5 & 1.5 & -1.25 \\ -1.5 & -1.25 & 1.3 \end{bmatrix}, \qquad R = \begin{bmatrix} 1 & 0.775 & -0.832 \\ 0.775 & 1 & -0.895 \\ -0.832 & -0.895 & 1 \end{bmatrix}.$$

Read R: more sleep goes with fewer hours studied (-0.83) and lower scores (-0.90) in this tiny toy set. The diagonal of Σ $(2.5, 1.5, 1.3)$ is just the three variances.

How to write it in Python

```
import pandas as pd, seaborn as sns
df = pd.DataFrame({"hours":[1,2,3,4,5], "score":[2,4,5,4,5], "sleep"
    :[8,7,6,7,5]})
df.cov()     # covariance matrix (diagonal = variances)
df.corr()    # correlation matrix (diagonal = 1, entries in [-1,1])
sns.heatmap(df.corr(), annot=True, vmin=-1, vmax=1, cmap="coolwarm")
      # the picture
```

A correlation **heatmap** is the fastest way to scan dozens of columns for the few strong pairs worth a scatter plot.

15.4 Kendall's τ — correlation by counting pairs

A third correlation, built purely from *order*. Take every pair of points; a pair is **concordant** if the one with the larger x also has the larger y, **discordant** otherwise. With C concordant and D discordant out of $\binom{n}{2}$ pairs,

$$\tau = \frac{C - D}{\binom{n}{2}} = \frac{C - D}{n(n-1)/2}.$$

Ranks $[1,2,3,4]$ vs $[1,2,4,3]$

The $\binom{4}{2} = 6$ pairs: five keep their order (concordant), only the last pair $(4,3)$ is flipped (discordant). So $C = 5$, $D = 1$ and

$$\tau = \frac{5-1}{6} = \boxed{0.667}.$$

Compared with Spearman on the same ranks, Kendall is more conservative and reads as "net fraction of pairs in the right order." Use it for small samples or when many ties make Spearman's rank-differences awkward.

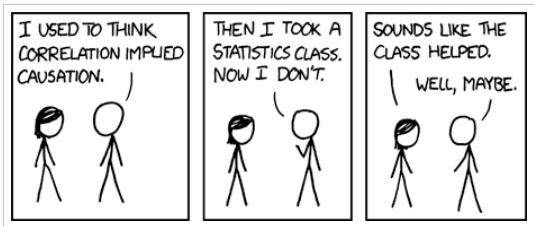
How to write it in Python

```
from scipy.stats import kendalltau
kendalltau([1,2,3,4], [1,2,4,3]).correlation    # 0.6667
df.corr(method="kendall")    # whole matrix in Kendall's tau
```

Three correlations, when to reach for each

Pearson r — strength of a *straight line*; interval/ratio data, no wild outliers. **Spearman** r_s — strength of a *monotone* trend (curved but consistent); ordinal data or when outliers would wreck Pearson. **Kendall** τ — also monotone, via pair-counting; robust, good for small n and heavy ties. All three live in $[-1, 1]$; disagreement between them is information — usually a sign of curvature or outliers.

15.5 The traps: correlation is not causation (and other lies of r)



xkcd #552 "Correlation" by Randall Munroe — `https://xkcd.com/552/` (CC BY-NC 2.5).

The whole discipline in one panel. A high r says two columns move together; it says *nothing* about which one moves the other, or whether a third thing moves both.

Watch Out

Four ways a strong r lies.

- **Causation.** Ice-cream sales and drownings correlate — because summer drives both. The lurking third variable is a *confounder*.
- **Outliers.** A single extreme point can drag Pearson from 0 to 0.9 or back. Always scatter-plot before trusting r; Spearman/Kendall resist this.
- **Non-linearity.** A perfect U-shape has $r \approx 0$ (the last chapter's parabola). Zero r means "no *straight* line," not "no relationship."
- **Simpson's paradox.** A correlation can flip sign once you split by a group — a trend inside every subgroup can reverse when the subgroups are pooled.

Chai-Break

"Correlation does not imply causation, but it does waggle its eyebrows suggestively and gesture furtively while mouthing 'look over there'." The scatter plot is how you decide whether to follow the finger — r alone will point you off a cliff.

Studio — now you try

17.1 By hand, compute $\mathrm{Cov}(X, Y)$ and Pearson r for $X = [2, 4, 6, 8]$, $Y = [1, 3, 2, 5]$: write $\bar{x}, \bar{y}$, every product, SS_x, SS_y, then r. Confirm with `np.cov` and `np.corrcoef`.
17.2 Show the units trap yourself: recompute the covariance of the hours/score example after converting hours to minutes ($\times 60$). What happens to the covariance, and what

happens to r? Explain in one sentence why only one of them moved.

17.3 For the three-column toy set (hours, score, sleep), reproduce `df.cov()` and `df.corr()` in pandas, then read off: which pair is most strongly associated, and is the diagonal of `df.cov()` what you expected?

17.4 Kendall by counting: for $X = [1, 2, 3, 4, 5]$ ranked against $Y = [1, 3, 2, 5, 4]$, list all $\binom{5}{2} = 10$ pairs, mark concordant/discordant, and compute τ. Check with `scipy.stats.kendalltau`.

17.5 Break Pearson with one point: take $X = [1, 2, 3, 4, 5]$, $Y = [1, 2, 3, 4, 5]$ ($r = 1$), then change the last point to $(5, -20)$. Recompute Pearson, Spearman, and Kendall. Which one collapses and which barely flinch — and what does that teach you about reporting a single correlation?

Part VI

From looking to designing

Chapter 16

Who Did We Observe? Sampling

Right now we are: explore — a statistic is a noisy photograph of a parameter

Lecture 9 begins with a blob labelled *target* and a smaller blob labelled *sample*. Everything in Chapters 7–12 compressed *the thirty rows we have*. This chapter asks the ruder question: why these thirty, and not thirty others?

16.1 N is a parameter. n is a statistic.

From the class board

Target population (e.g. Indian adults). Experiment: effect of vaccination → sampling. Population oval, size N → **parameter**. Sample oval inside it, size n → **statistic**. $N \gg n$. Question: "Who should I observe?" The number of possible samples is ${}_N C_n$.

In Plain Words

A **population** is the set of units you *wish* you could measure. Its size is N (finite, or "all future patients", which you may as well call infinite). A **parameter** is a number that lives on that whole set: a mean μ, a proportion p, a variance σ^2. It does not jitter when you redraw a sample. We write it in Greek.
A **sample** is the units you actually touch. Its size is n. A **statistic** is a number computed from those n rows: $\bar{x}$, $\hat{p}$, s^2. It jitters. We write it in Roman.
The thirty marks in `it462_marks.csv` have mean 63.5. If those thirty *are* the whole IT462 batch, 63.5 is a parameter. If they are a sample from a larger batch, 63.5 is a statistic. Same arithmetic. Different job.

The combination the board wrote

The number of ways to choose n units from N, order ignored, is

$$\binom{N}{n} = {}_N C_n = \frac{N!}{n!\,(N-n)!}.$$

You do not need a 120-student batch to feel the explosion. Four people, sample two:

$$\binom{4}{2} = \frac{4 \cdot 3}{2 \cdot 1} = 6.$$

Watch Out

N and n are not "big n and small n". N names the world you want; n names the world you have. Calling the thirty-row mean μ is a viva fail if those thirty are a sample.

16.2 Simple random sampling: list every pair

From the class board

Simple random sampling. Population $\{1, 2, 3, 4\}$, sample size $n = 2$. Sketch of pairs 12, 13, 14, then an ellipsis. You are expected to finish the list.

Under **simple random sampling** (SRS), every subset of size n is equally likely. For $N = 4$, $n = 2$ that means each of the six pairs has probability $1/6$. There is no "almost random" cousin in this definition: known equal probabilities, or it is not SRS.

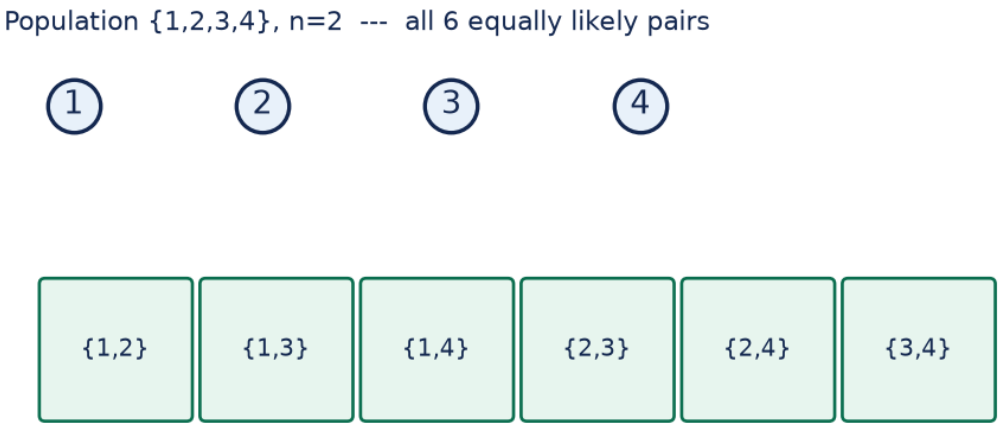


Figure — The sample space of SRS for $\{1, 2, 3, 4\}$ and $n = 2$. Six boxes, six equally likely tickets.

Derivation — step by step

Write every combination of four labels taken two at a time, in lexicographic order so you cannot "forget" one:

$$\{1, 2\}, \quad \{1, 3\}, \quad \{1, 4\}, \quad \{2, 3\}, \quad \{2, 4\}, \quad \{3, 4\}.$$

That is $\binom{4}{2} = 6$, and the list is the proof. Each pair is one sample. Because SRS puts equal mass on every subset,

$$P\big(\text{sample} = \{1, 4\}\big) \;=\; \frac{1}{6}.$$

The probability that a particular unit, say unit 1, appears in the sample is the fraction of pairs that contain a 1. Those pairs are $\{1, 2\}, \{1, 3\}, \{1, 4\}$ — three of six — so

$$P(1 \in \text{sample}) \;=\; \frac{3}{6} \;=\; \frac{1}{2}.$$

In general, under SRS without replacement,

$$P(\text{unit } i \text{ is drawn}) \;=\; \frac{n}{N}.$$

Here $n/N = 2/4 = 1/2$, which matches the enumeration. The formula is a compression of the list; the list is how you know the formula is not a rumour.

Another way to do it

You can also think sequentially: pick the first unit uniformly (4 choices), then the second from the remaining 3, then divide by 2 because order does not count.

$$\frac{4 \times 3}{2} \;=\; 6$$

again. Sequential language is useful in code (`rng.choice(..., replace=False)`); the combination ${}_N C_n$ is useful on the board.

How to write it in Python

```
from itertools import combinations
from math import comb
pop = [1, 2, 3, 4]
pairs = list(combinations(pop, 2))
assert len(pairs) == comb(4, 2) == 6
assert pairs == [(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)]
# each equally likely under SRS
print(1 / len(pairs))   # 0.1666... = 1/6
```

If that assert fires, you dropped a pair. The ellipsis on the board is not a licence to skip $\{3, 4\}$.

16.3 A 12-row toy: cluster is not stratified

The board drew two other pictures: a row of clumps with one clump circled (**cluster**), and a rectangle sliced into groups with a few dots in every slice (**stratified**). They look similar. They are opposites. A twelve-row table is enough to feel the difference in your hands.

From the class board

Three methods: (1) simple random, the pairs above; (2) cluster — pick a whole C_1; (3) stratified — dots in every group.

Twelve tutorial students, three hostels of four. Marks are constant-ish inside a hostel (that is the point of a clump) and different across hostels:

id	hostel	city-stratum	marks
T01–T04	H1	West	70, 72, 68, 74
T05–T08	H2	East	40, 42, 38, 44
T09–T12	H3	North	90, 88, 92, 86

Hostel sums: H1 $= 70+72+68+74 = 284$ (mean 71); H2 $= 40+42+38+44 = 164$ (mean 41); H3 $= 90+88+92+86 = 356$ (mean 89). The twelve-row total is $284+164+356 = 804$, so the population mean is

$$\mu = \frac{804}{12} = 67.$$

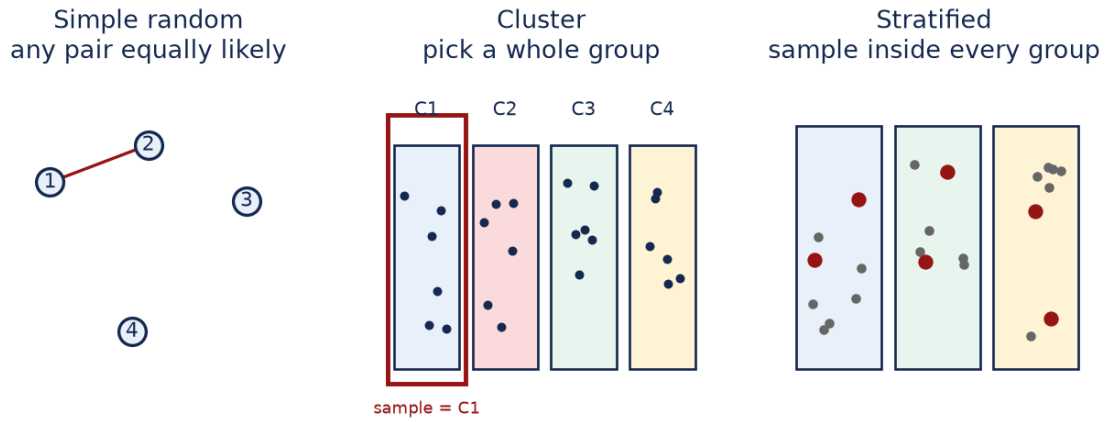


Figure — Left: SRS, any pair. Middle: cluster — take a whole clump. Right: stratified — take someone from every slice.

Cluster sample of one hostel

Cluster sampling here: toss a fair three-sided lot, pick one hostel, take all four students. Three possible samples, equally likely, with means

$$\bar{x}_{\mathrm{H1}} = 71, \qquad \bar{x}_{\mathrm{H2}} = 41, \qquad \bar{x}_{\mathrm{H3}} = 89.$$

Cheap (one building). Noisy: two of the three photographs miss $\mu = 67$ by 26 marks or more. People in a clump are alike, so four bodies are not four independent stories.

Stratified sample of one from each hostel

Stratified sampling here: treat the three hostels as strata, and SRS *inside* each. One draw from H1, one from H2, one from H3. Every sample has size $n = 3$ and contains West, East, and North. One equally-likely example: $\{\mathrm{T01} = 70,\ \mathrm{T05} = 40,\ \mathrm{T09} = 90\}$,

$$\bar{x} = \frac{70+40+90}{3} = \frac{200}{3} \approx 66.67,$$

already sitting on $\mu = 67$. You can write down all $4 \times 4 \times 4 = 64$ stratified samples; every one mixes a ~ 71, a ~ 41, and a ~ 89. Representation is guaranteed. Travel is not cheap: you visit every hostel.

In Plain Words

Cluster *samples clumps.* Stratified *forces every clump-type to appear.* You cluster for logistics (one village, one ward, one hostel). You stratify for representation (gender, city, hostel — whatever you refuse to leave out). Using the same three groups both ways is the whole lesson: the grouping is not the method; what you do with the grouping is.

Another way to do it

On the same twelve rows you can still draw an SRS of size 4: $\binom{12}{4} = 495$ equally likely subsets. SRS does not *forbid* an all-H2 sample; it just makes that sample as likely as any other four-set, namely 1/495. Cluster makes "all of H2" one of three tickets. Stratified makes "all of H2" impossible.

How to write it in Python

```
import pandas as pd
toy = pd.DataFrame({
    "id": [f"T{i:02d}" for i in range(1, 13)],
    "hostel": ["H1"]*4 + ["H2"]*4 + ["H3"]*4,
    "marks": [70,72,68,74, 40,42,38,44, 90,88,92,86],
})
assert toy.marks.sum() == 804 and toy.marks.mean() == 67
cluster = toy[toy.hostel == "H2"]            # one whole clump
print(cluster.marks.mean())                  # 41
strat = toy.groupby("hostel", group_keys=False).sample(n=1,
    random_state=0)
print(strat.marks.mean())                    # near 67, always mixed
```

Watch Out

"Random" in English means haphazard. "Random" in this course means *known probabilities*. Grabbing the first four students at the door is not SRS. Neither is "I picked H1 because I like that hostel" a cluster sample — a cluster sample still uses a chance process to choose which clump.

Studio — now you try

13.1 Population $\{1, 2, 3, 4\}$, SRS of size $n = 2$. Write all six samples. What is $P(\text{sample} = \{2, 4\})$? What is $P(4 \in \text{sample})$, by listing and by the n/N formula?
13.2 On the twelve-row hostel table, a cluster sample of one hostel is drawn. List the three possible sample means. A stratified sample draws one student from each hostel; compute $\bar{x}$ for $\{\text{T04}, \text{T06}, \text{T12}\}$. Why can this stratified $\bar{x}$ not equal 41?
13.3 Suppose `it462_marks.csv` is an SRS of $n = 30$ from a batch of $N = 120$. Is 63.5 a parameter or a statistic? Write $\binom{120}{30}$ as a factorial ratio (do not expand it). What would 63.5 become if those thirty rows *were* the whole batch?

Chapter 17

Experiments, Bias, and the Two Questions

Right now we are: collect — design the observation so the photograph is not a selfie

Sampling answers "who do I observe?" Experimental design answers "what do I *do* to them, and what do I measure?" Lecture 9 put both on the same board because a beautiful sample of the wrong experiment is still the wrong experiment.

17.1 A vaccination story with four labels

From the class board

Experiment: effect of vaccination. Centre of the design: problem definition / research question / target. Branches: experimental unit, factors / levels, response variable → choose design.

In Plain Words

Write the four labels *before* anyone rolls up a sleeve. If you cannot fill them, you do not have a design. You have a vibe.

One sentence, four names

Research question: does vaccine A reduce fever on day 7 in adults over 60, compared with a saline placebo?

- **Experimental unit.** The smallest thing that independently receives a treatment. Here: one adult. (If a whole clinic is assigned the same vial-lot, the unit is the clinic, and the people inside it are sub-measurements.)
- **Factor.** The knob you turn. Here: which injection.
- **Levels.** The values of that knob. Here: vaccine A, or saline placebo. Two levels, one factor. A second factor (dose: low / high) would make a two-way design.
- **Response.** What you measure afterwards. Here: fever in °C on day 7 (interval), or "fever yes/no" (nominal). Marks, yield, and "passed" are responses in other stories; they are not responses here.

The **target** is the world you will generalise to — adults over 60 in India, say — which is

exactly the population oval of Chapter 13. Sampling is how units enter the experiment. Design is what happens after they enter.

Watch Out

A unit is not "a row in Excel". If thirty students in one classroom all watch the same tutorial video, you have *one* experimental unit (the class) and thirty correlated marks. Treating $n = 30$ as thirty independent vaccines is how people invent significance.

17.2 The three Rs

From the class board

Three headings above Collect data: **Randomization**, **Replication**, **Control / Block**. Then collect. Then IDA — "Is the data reliable?" Then EDA — "What patterns are there?"

Derivation — step by step

Fisher's three Rs, in the vaccination story, without a p-value yet.

Randomization. Assign vaccine vs placebo with a known chance process — a coin, a shuffled list, `rng.permutation`. Two jobs, not one: (i) lurking variables (the gym-goers, the already-infected) land in both arms *on average*; (ii) the probability model behind a later test is licensed. Without randomization you have an observational study. Observational studies are legal. Causal claims from them are a different sport.

Replication. More than one unit per level. $n = 1$ is an anecdote ("my uncle took the vaccine and was fine"). Replication also means: a stranger can rerun the protocol on a new draw and land near your number. Thirty IT462 students are a replication of "a student", not a replication of "a college" — one college is still $n = 1$ at the college level.

Control / blocking. Control is a comparison arm. Vaccine-without- placebo cannot tell fever-from-vaccine apart from fever-from-flu-season. **Blocking** is stratified *assignment*: randomize inside gender, or inside hospital, so those factors do not swamp the treatment. A randomised complete block design is stratified sampling wearing a lab coat.

Another way to do it

Observational cousin, using `it462_marks.csv`. You did not assign `hours_studied`. Students chose it. Randomization of the factor never happened, so a scatter of hours against marks (Chapter 12) is association, not "studying caused the mark". You can still *block* in the analysis (look at Ahmedabad separately from Rajkot) and you can still *replicate* (thirty students, not one). The missing R is the one that buys causal language.

How to write it in Python

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(0)
units = pd.DataFrame({
    "id": [f"U{i:02d}" for i in range(1, 21)],
    "block": ["M"]*10 + ["F"]*10,
```

```
})
# randomize *inside* each block: 5 vaccine, 5 placebo
units["arm"] = (
    units.groupby("block")["id"]
         .transform(lambda s: rng.permutation(["vaccine","placebo"]*(len(s)//2)))
)
print(units.groupby(["block","arm"]).size())
```

The print should show 5 and 5 in every block. If it shows 10 vaccine in one gender, you randomized the whole list and ignored the block.

17.3 Sampling bias is a design failure, not a rude mean

From the class board

Sampling bias, arrow into problem definition. If the sample is the wrong costume, no later formula puts the right clothes on it.

In Plain Words

Sampling bias means the chance of entering the sample depends on the thing you care about, or on a lurking cousin of it. The terrace that faces the yellow-kite shop will report "most kites are yellow". The vaccination survey run in one private clinic will report "India".

Three costumes that are not the IT462 batch

If the target is "all thirty students in `it462_marks.csv`":

- Google form posted at 1 am: you over-sample the night-owls, who may also be the high-`hours_studied` rows.
- Survey only Ahmedabad (9 of 30 in the file). City-mean marks will not represent Rajkot.
- Voluntary "tell us your marks if you like": the four fails (S05 39, S09 28, S18 35, S29 31) are the least likely to volunteer. The sample mean drifts up from 63.5 toward the classmate's prettier 71.

None of these is fixed by taking a median instead of a mean. Bias is about *who is missing*, not about which pin you drop on the people who showed up.

Watch Out

Convenience is not a sampling method with a funny name. It is a different target. If you only measured whoever walked into the lab, your parameter is "lab walk-ins", and you should say so. Lying about the target is worse than a small n.

17.4 IDA asks "reliable?" EDA asks "patterns?"

The board did not stop at Collect data. It wrote two questions in order.



Figure — IDA, then EDA, then CDA. Randomization does not skip the first box. A randomised column can still be coded −99 for missing.

In Plain Words

IDA (Chapter 3, and the L10 checklist in Chapter 15): is this table usable — valid, reliable, authentic, error-hunted? After a vaccination trial that question is still "did we record fever in °C or in a made-up code; are two IDs the same person; did anyone enter age 0?"
EDA: given a usable table, what is the shape? Two peaks in fever? A gender × arm table that is *not* 5–5? A boomerang in the scatter that Pearson will miss?
CDA (not this course): a model, a test, a confidence interval, a claim that should survive a fair hold-out. You do not get to CDA by skipping IDA because "we randomized".



Figure — Five IDA questions that still apply after a perfect randomization: structure, granularity, scope, temporality, faithfulness. Scope is sampling. Faithfulness is whether the recorded fever is the real fever.

The same thirty rows, two different jobs

On `it462_marks.csv`, after a hypothetical "new tutorial video" trial:

- IDA: `df.shape` is $(30, 8)$; `student_id` unique; marks in $[28, 99]$, feasible for a 0–100 paper; four `passed = no` match marks < 40; no missing cells. Reliable enough to look.
- EDA: marks mean 63.5, median 64, almost symmetric; hours and marks move together; gender × passed is 13–2 / 13–2. Patterns.

If IDA had found duplicate IDs and a mark of −20, EDA on the raw file would be a short story about a typing accident, not about learning.

Studio — now you try

14.1 A college tests whether a new canteen snack changes afternoon alertness. They randomly give snack A or snack B to each of 40 students and record Likert sleepiness (1–5) two hours later. Name the experimental unit, the factor, the levels, and the response. What measurement scale is the response *before* anyone "Likerts it into interval"?

14.2 Rewrite the three Rs as three sentences for that snack experiment. Then name the R that is missing if the canteen instead lets students *choose* A or B.

14.3 You want the mean IT462 mark of all DA-IICT first-years. You WhatsApp a form to your hostel floor (mostly Ahmedabad, mostly people who have already passed). (a) Name the sampling bias in one sentence. (b) After the CSVs arrive, which question is IDA and which is EDA: "are there duplicate roll numbers?" vs "do hours and marks move together?"

Chapter 18

The Import Lab: A One-Page EDA Report

Right now we are: manage — after `read_csv`, walk the board, then look

This is Lecture 10, executed. No new formula. The board is a checklist; `it462_marks.csv` is the patient; the output is one page you could tape above a lab bench. Chapter 0 told you the target/feature split. Chapters 3–6 told you IDA and NOIR. Chapters 7–12 told you which summaries are legal. Chapters 13–14 told you the thirty rows are a sample. Now put it in one place.

18.1 Where this sits in the course, and how data gets in

From the class board

Randomization, replication, control/block → collect data → **IDA** (is the data reliable?) → **EDA** (what patterns are there?).
Part 1: Data → scales, tools, design. **Part 2**: project / working space → import data with **pandas**.
Data import: online or local. **CSV files**: (i) load the packages (install if not done), `import pandas as pd`; (ii) `PATH = "..."`; (iii) `df = pd.read_csv(PATH)`; (iv) look at the table. Understand the **metadata**: the description of the dataset.

Local file vs online file

```
# (i) install once in a terminal / notebook cell, then import every session
# !pip install pandas
import pandas as pd

# (ii)+(iii) LOCAL: a path on your own disk (or in the project / working folder)
PATH = "data/it462_marks.csv"                    # relative to the notebook's folder
df = pd.read_csv(PATH)

# (ii)+(iii) ONLINE: read_csv also accepts a URL directly
URL = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(URL)

```

```
# other formats, same idea
# pd.read_excel("file.xlsx"), pd.read_json("file.json"), pd.read_csv(PATH, sep=";")

# (iv) first look + metadata
df.head(); df.shape; df.columns; df.dtypes; df.info()
```

For an online file, use the **raw** link (on GitHub, the "Raw" button), not the web page that displays it — the web page is HTML, not CSV. If the source has a data dictionary (Kaggle "Data" tab, UCI "Variables" table), read it before anything else: it is the metadata that tells you units, codes for missing values, and what one row means.

The column-level functions from the notes

```
df.info()                          # dtype + non-null count for every column
df.dtypes                          # int64 / float64 = numeric, object = text (category-like)
df.columns                         # all feature names (an attribute, no brackets)
df.shape                           # (m rows, n columns)
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")   # naming convention
df["roll"] = df["roll"].astype(str)          # change type, e.g. int64 -> string
df["likert"] = df["likert"].astype("category")
df["city"].value_counts()          # count the values
df["city"].unique()                # the distinct values
df["city"].nunique()               # how many distinct values
```

18.2 Open the file. Do not model.

How to write it in Python

```
import pandas as pd
df = pd.read_csv("it462_marks.csv")
assert df.shape == (30, 8)
assert df["marks"].sum() == 1905
assert df["marks"].mean() == 63.5
```

If an assert fires, you opened the classmate file from studio 0.2. Stop.

From the class board

After Data Import (`.csv`):

1. **features** vs **target feature**
2. dimension — `df.shape`
3. type (measurement scales)
4. naming convention
5. units, date-time (values)

6. duplication
7. feature engineering (add-on)

Then, on **values / observations**: feasible range (min, max); unitless; duplicates; missing; outlier.

18.3 Half 1 — columns

Features vs target

Question in play: "did the student pass?" Then

$$\text{target} = \texttt{passed},$$

and the features are `student_id`, `marks`, `age`, `gender`, `likert`, `city`, `hours_studied`. If the question were "what mark did they get?", the target would be `marks` and `passed` would become a redundant encoding of `marks` ≥ 40, not a feature you should feed a model. Name the question first. The split follows.

`df.shape`, types, names, units, duplicates, engineering

`df.shape` $= (30, 8)$: thirty observations, eight columns.
NOIR, column by column (machine type in parentheses):

column	machine type	NOIR	notes
`student_id`	string	nominal	label, never a mean
`marks`	int	ratio	0 means no marks
`age`	int	ratio	years; true 0 exists
`gender`	string	nominal	F / M
`likert`	int	ordinal	5-point, not interval yet
`city`	string	nominal	four Gujarat cities
`hours_studied`	int	ratio	hours, not "o'clock"
`passed`	string	nominal	binary yes/no

Naming. Snake-case, ASCII, no units stuffed into the header. Good. `Age (yrs)` would have been a later headache.
Units / datetime. Marks are marks-out-of-100. Age is years. `hours_studied` is hours (a duration), not a timestamp. There is no datetime column in this file; write that down so nobody "parses" `age`.
Duplication. `student_id` is unique (30 distinct). No entire-row copies. Duplicate IDs with different payloads would have been a conflict table, not a `drop_duplicates()` reflex.
Feature engineering (add-on, after looking). Legal later adds: `passed` from `marks >= 40` (already present — check it matches); a z-score of marks; a one-hot of `city`; a missingness flag if a future file has holes. Do not one-hot a unique `student_id`. Do not average `city`.

How to write it in Python

```
print(df.shape)                    # (30, 8)
print(df.dtypes)
```

```
print(df.columns.tolist())
print(df["student_id"].is_unique, df.duplicated().sum())
# codebook, not in the CSV:
# marks: 0-100 paper; age: years; hours_studied: hours; no datetime
```

18.4 Half 2 — cells

Feasible range, unitless, dupes, missing, outlier

Numeric min / max:

column	min	max	feasible?
`marks`	28	99	yes, inside $[0, 100]$
`age`	18	23	yes, inside $[0, 120]$
`likert`	1	5	yes, 5-point
`hours_studied`	1	11	yes, non-negative duration

Gender $\in \{\text{F}, \text{M}\}$, city $\in \{\text{Ahmedabad}, \text{Surat}, \text{Vadodara}, \text{Rajkot}\}$, passed $\in \{\text{yes}, \text{no}\}$. All legal.

Unitless. Nothing in this file is a z-score or a correlation. The later Pearson r between marks and hours will be unitless; it is not a column yet.

Duplicates (values). Two students scored 74 (S01 and S30). That is a repeated *value*, not a duplicated *row*. Mode of marks is 74; do not delete one of them.

Missing. Zero empty cells. `df.isna().sum()` is a row of zeros. (Contrast `it462_dirty.csv`, next studio.)

Outliers. Sorted marks start at 28 (S09) and end at 99 (S22). Tukey fences on marks, with $Q_1 = 47.75$, $Q_3 = 76.75$, IQR $= 29$:

$$47.75 - 1.5 \times 29 = 4.25, \qquad 76.75 + 1.5 \times 29 = 120.25.$$

No mark sits outside (4.25, 120.25). The four fails are low, not illegal. A 1.5 IQR ritual that deleted S09 would be vandalism of a real fail.

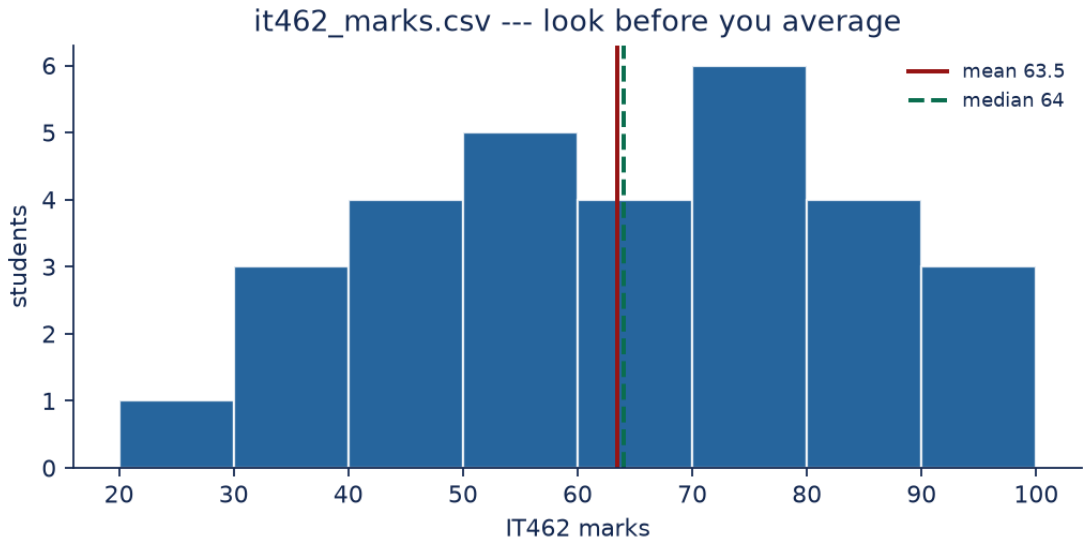


Figure — Look: thirty marks, mean 63.5, median 64, no cartoon tail. This is why the file is a playground, not a gotcha.

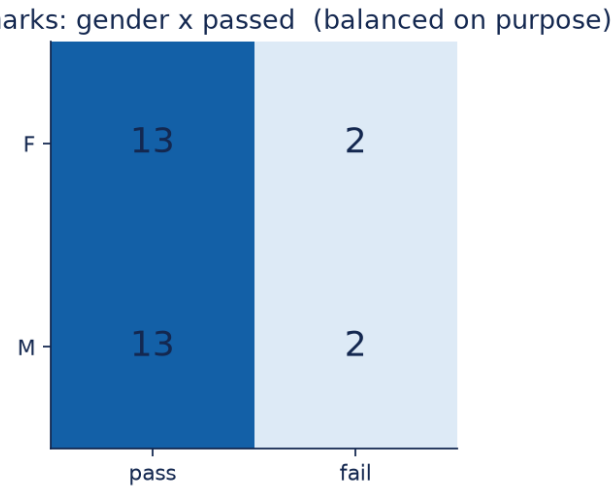


Figure — Gender × passed is 13–2 / 13–2 on purpose. A checklist that never makes this table has not looked.

Watch Out

`df.describe()` will happily average a numeric-looking `likert` and a student id that pandas parsed as a float. NOIR is your job. The checklist on the board is the reminder.

18.5 The one-page report (filled in for the thirty)

Tape this. Change the numbers only when the file changes.

EDA report — `it462_marks.csv`

File. `it462_marks.csv`, $n = 30$ rows, $p = 8$ columns. Secondary, invented for IT462. Provenance: Chapter 0.
Question / target. "Did they pass?" → `passed`. Features: the other seven columns. (If predicting marks, swap.)
Usable? Unique IDs, no missing cells, marks in $[28, 99]$, ages in $[18, 23]$, genders balanced 15/15, cities Ahmedabad 9, Surat 8, Vadodara 7, Rajkot 6. Four fails: S05 39, S09 28, S18 35, S29 31, matching `marks < 40`. IDA answer: reliable enough to explore.
Univariate. Marks: sum 1905, mean 63.5, median 64, min 28, max 99, sample SD ≈ 19.60. Almost symmetric (mean $\approx$ median). Age mean exactly 20. Likert mode 3 (nine students). Hours mean ≈ 5.37.
Bivariate. Marks and hours move together (both ratio: Pearson is legal). Gender × passed is balanced 13 yes / 2 no in each gender — no gender-pass story in this file. Likert vs marks: Spearman, not Pearson, unless you argue the interval transform.
Scope / sampling. These thirty are the running table, not "India". A mean of 63.5 is a parameter of *this* file and a statistic of any larger batch you pretend to speak for.
Not yet. No model. No dropped "outliers". No one-hot of unique IDs. Feature engineering waits until the question is locked.

How to write it in Python

```
num = df.select_dtypes("number")
print(num.agg(["min", "max", "mean", "median"]))
print(df.isna().sum())
print(df["gender"].value_counts())
print(pd.crosstab(df["gender"], df["passed"]))
fails = df.loc[df["passed"].eq("no"), ["student_id", "marks"]]
print(fails)
assert set(fails["student_id"]) == {"S05", "S09", "S18", "S29"}
```

Studio — now you try

15.1 Execute the *column* half of the board on `it462_marks.csv`: name the target if the question is "did they pass?", report `df.shape`, and assign NOIR to every column. Which column is a duration, and which columns have no datetime meaning at all?
15.2 Execute the *value* half: feasible min/max for marks and age; count missing cells; decide whether S09's 28 is an outlier you should delete. Show the 1.5 IQR fences that say no.
15.3 Write the one-page EDA report for `it462_dirty.csv` (five rows from Chapter 0). IDA will dominate. Do not invent a mean of IT462 that pretends −20 is a mark.

Part VII

Cleaning and transforming (Insem 2)

Chapter 19

Missing Data: What It Is and Why It Went Missing

Right now we are: managing — the import lab told you to count blanks. Now we ask *why* each blank is there, because the answer decides every fix that follows.

The board called this topic **Missing Value Analysis** and gave it four headings: *definition, causes, classification, detection, treatment.* This chapter does the first three. The next two chapters do detection and treatment.

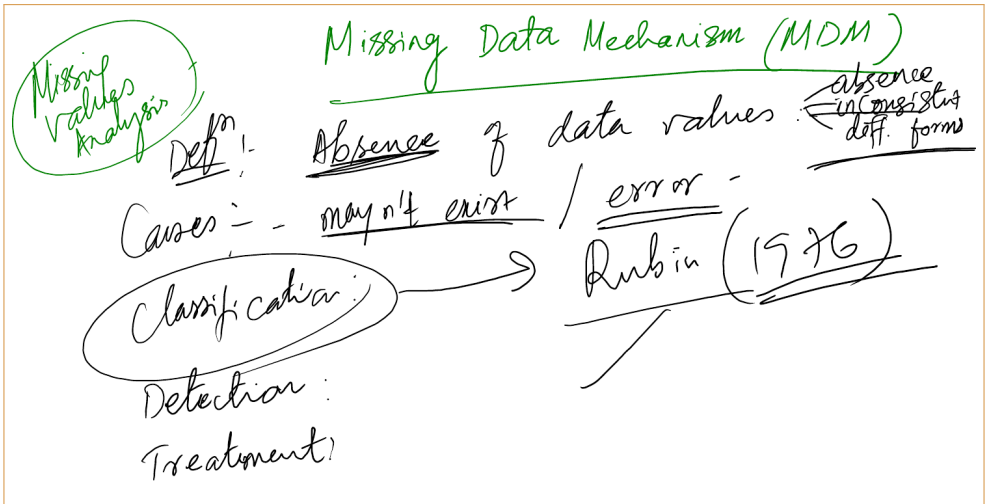
In Plain Words

A blank cell is not just "no number". It is a clue. If the blank fell there by pure chance, you can mostly ignore it. If it fell there *because* of something in the data, filling it in carelessly will bend every mean, correlation and model you build afterwards.

19.1 Definition and causes

From the class board

Missing data: absence of data values, or values present in inconsistent / different forms.
Causes: the value may not exist, or there was an error.
Classification: Rubin (1976) — MCAR, MAR, MNAR.



Sir's board, Insem 2 notes, page 04 — definition, causes, Rubin (1976), and the four headings of missing value analysis.

Missing value

A cell is **missing** when the value that *should* describe that observation for that feature is not available for analysis. It shows up in two disguises:

1. **Absent**: an empty cell, `NaN`, `None`, `NULL`.
2. **Present but inconsistent**: a stand-in that *looks* like data — `"NA"`, `"-"`, `"?"`, `"unknown"`, `999`, `-1`, `0` for an age. These are called **sentinel values**. pandas does not know they mean "missing" until you tell it.

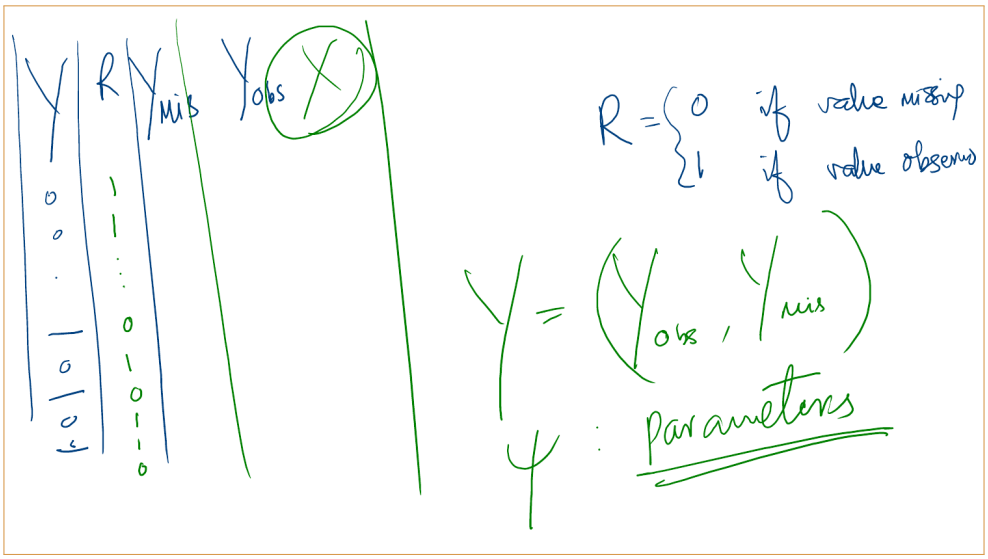
Why values go missing. Two families, matching the board.

- **The value does not exist.** "Spouse's name" for an unmarried person, "marks in lab 4" for a student who joined after lab 4. These are *structural* blanks; often the right fix is a new category such as `"not applicable"`, not a guess.
- **The value exists but we lost it.** Sensor failure, typing error, a respondent who skipped or refused a question, a student who dropped out, two files merged on a key that did not match, a column renamed between two years of exports.

Watch Out

Sentinels hide from `isnull()`. A column of ages with `999` for "unknown" reports zero missing values and a mean that is far too high. Convert sentinels first: `pd.read_csv(path, na_values=["NA","-","?",999])` or `df["age"] = df["age"].replace(999, np.nan)`.

19.2 The notation on the board



Sir's board, Insem 2 notes, page 12 — the response indicator R, $Y = (Y_{obs}, Y_{mis})$ and the parameters ψ.

Rubin's idea is to treat *missingness itself* as a variable you can study.

From the class board

$R = \begin{cases} 0 & \text{if the value is missing} \\ 1 & \text{if the value is observed} \end{cases}$ $Y = (Y_{\text{obs}}, Y_{\text{mis}})$ ψ: parameters of the missingness process.

- Y is the *complete* data you wish you had.
- Y_{obs} is the part you actually see; Y_{mis} is the part hidden behind the blanks. Other fully observed columns (the board's X) belong to Y_{obs}.
- R is the **response indicator**: one 0/1 per cell, the same shape as the table.
- ψ describes *how* values go missing (for example "each cell has a 10% chance").

Building R for a tiny table

student	hours	score
1	2	55
2	4	NaN
3	6	71
4	8	NaN

$\Longrightarrow \quad R_{\text{score}} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$

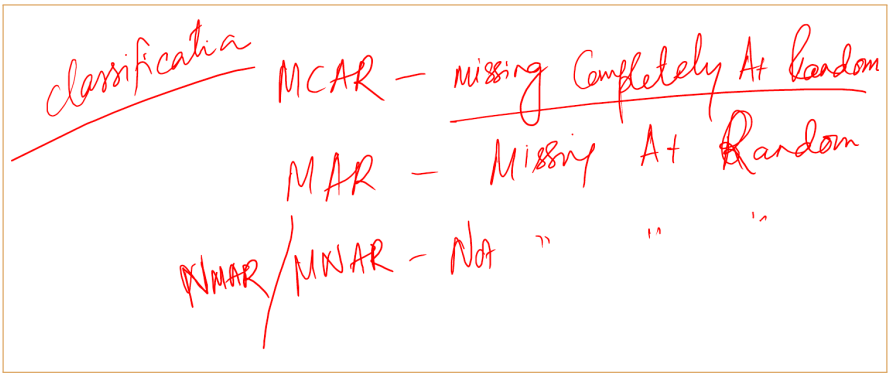
$Y_{\text{obs}} = \{\text{all hours}, 55, 71\}, \qquad Y_{\text{mis}} = \{\text{score}_2, \text{score}_4\}.$

The question Rubin asks: *does the pattern in R depend on anything?*

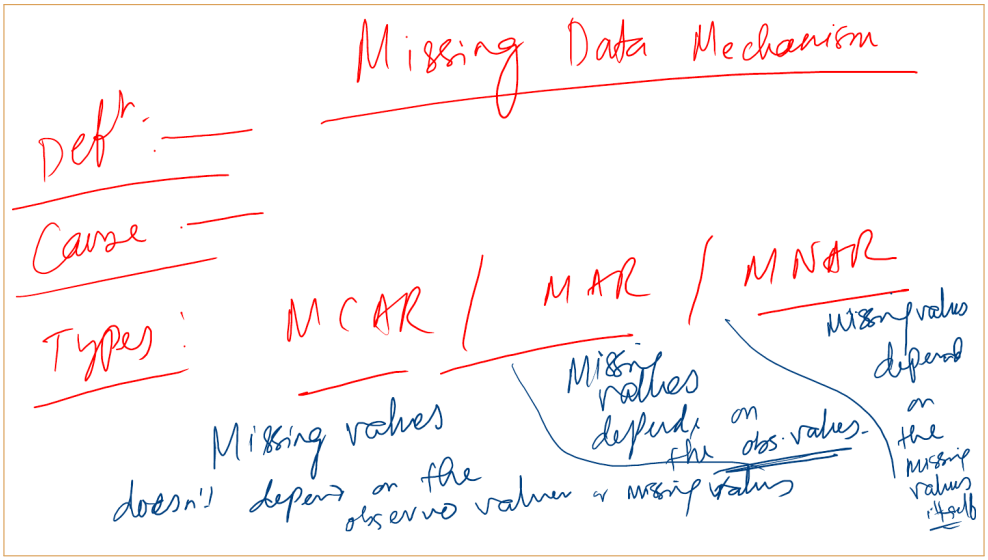
Watch Out

Opposite conventions. The board uses $R = 0$ for missing. pandas' `df.isnull()` returns `True` (i.e. 1) for missing. So $R = 1 - $ `df.isnull()`. Same information, flipped coding. Read every slide and every library doc for which way round it is.

19.3 The three mechanisms



Sir's board, Insem 2 notes, page 05 — the classification.



Sir's board, Insem 2 notes, page 07 — one line per mechanism.

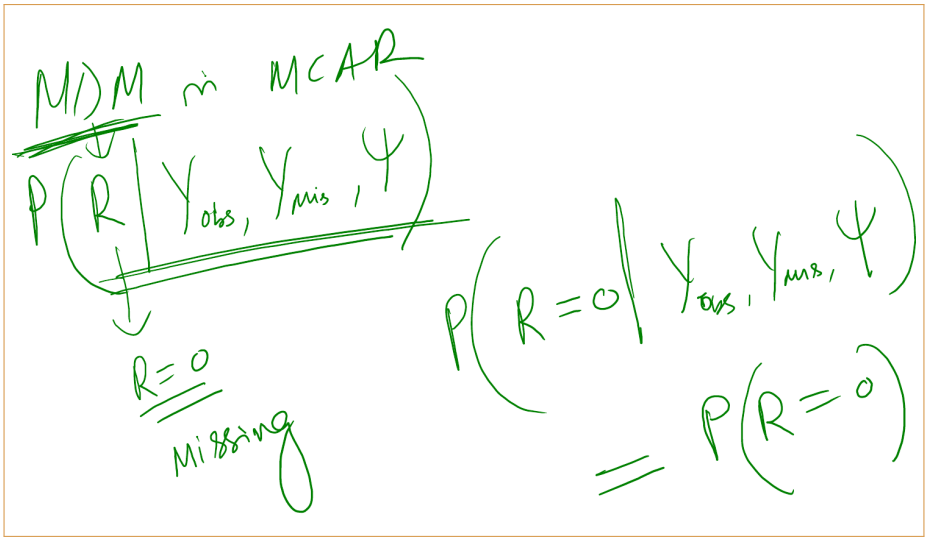
From the class board

MCAR: missing values do not depend on the observed values nor on the missing values.
MAR: missing values depend on the observed values.
MNAR (also NMAR): missing values depend on the missing values themselves.

19.3.1 MCAR — Missing Completely At Random

$$P(R = 0 \mid Y_{\text{obs}}, Y_{\text{mis}}, \psi) \;=\; P(R = 0 \mid \psi).$$

The chance that a cell is blank is the same for every row. Nothing you can see and nothing you cannot see predicts it. Y_{mis} and Y_{obs} are independent of the missingness.



Sir's board, Insem 2 notes, page 13 — the MCAR formula: the probability of a blank equals $P(R = 0)$, whatever the data say.

Examples. A lab assistant spills tea on a random page of the register. A survey server drops 2% of submissions at random. Blood samples for some patients are lost in transit, for reasons unrelated to the patients.

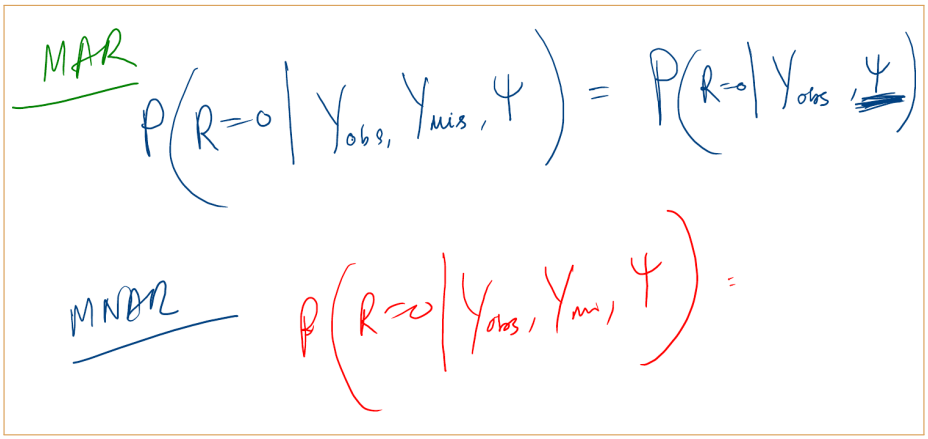
In Plain Words

MCAR means the rows you still have are a fair, smaller random sample of the rows you started with. Means and correlations computed on them are unbiased; you only lose precision.

19.3.2 MAR — Missing At Random

$$P(R = 0 \mid Y_{\text{obs}}, Y_{\text{mis}}, \psi) \;=\; P(R = 0 \mid Y_{\text{obs}}, \psi).$$

The chance of a blank depends on things you *did* record, but, once you account for those, not on the hidden value itself. The board's short form: *in MAR, Y_{mis} depends on Y_{obs}.*

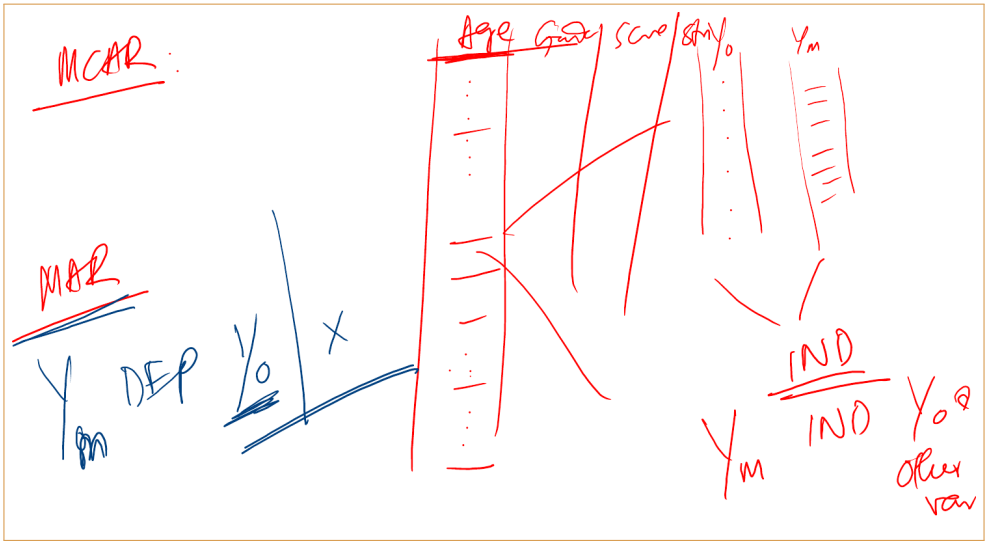


Sir's board, Insem 2 notes, page 14 — MAR drops Y_{mis} from the condition; MNAR cannot drop anything.

Examples. Younger respondents skip the income question more often (age is recorded). Students with fewer study hours leave the feedback score blank more often (hours is recorded). A blood pressure reading is taken only when the recorded BMI is above a threshold.

Watch Out

The name is misleading. MAR is **not** "random" in everyday English. It means "random *after* conditioning on what you observed". The rows you keep are biased, but the bias is *explainable* with columns you have, which is exactly why it can be fixed.



Sir's board, Insem 2 notes, page 06 — a table with Age, Gender, Score: under MCAR the missing scores Y_m are independent of Y_o and every other variable; under MAR Y_m depends on the observed Y_o / X.

Reading the board's Age / Gender / Score table

Suppose `score` has blanks. Ask two questions of the *other* columns. (1) Do the rows with a blank score have a different age or gender mix from the rows with a score? If not, nothing observed predicts the blank: consistent with **MCAR** (Y_m independent of Y_o and other variables). (2) If, say, most blanks are among students over 25, the blanks depend on an observed column: **MAR** (Y_m depends on Y_o). Neither question can reveal whether the blank scores were the low ones — that is the MNAR question, which the table cannot answer.

19.3.3 MNAR — Missing Not At Random

$$P(R = 0 \mid Y_{\text{obs}}, Y_{\text{mis}}, \psi) \quad \text{depends on } Y_{\text{mis}} \text{ and cannot be simplified.}$$

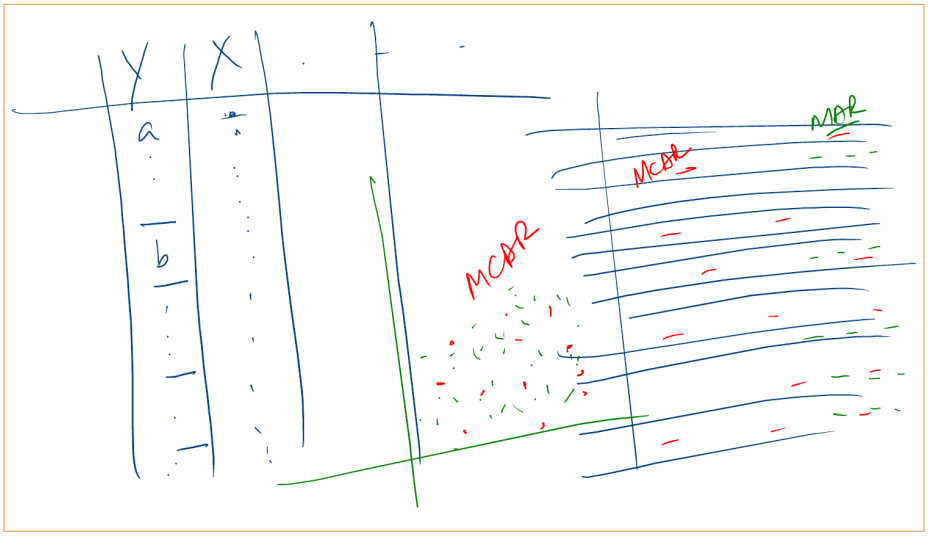
The value is missing *because of what the value is.*

Examples. People with very high incomes refuse to state income. Students who failed do not report their marks. A weighing scale that maxes out at 150 kg leaves heavier people blank. A depression survey is skipped by the most depressed patients.

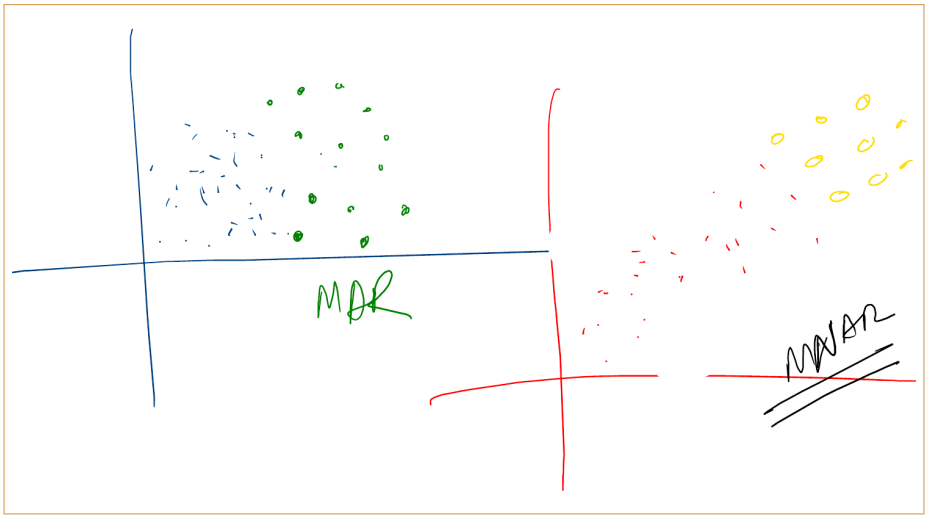
Analogy

Think of a class WhatsApp group where the teacher asks everyone to post their mock-test score. **MCAR**: a few phones were on silent, so some never saw the message. **MAR**: hostel students reply less because the hostel Wi-Fi is bad, and you know who lives in the hostel. **MNAR**: students with low scores quietly do not post. The average of the posted scores looks great, and nothing in the chat tells you why.

19.4 Seeing the three mechanisms on one dataset



Sir's board, Insem 2 notes, page 08 — a Y, X table and the MCAR / MAR scatter of holes.



Sir's board, Insem 2 notes, page 09 — MAR: holes in one region of the observed axis; MNAR: holes where the hidden value itself is extreme.

Take eight students with *true* (complete) data:

$$\text{hours} = [1, 2, 3, 4, 5, 6, 7, 8], \qquad \text{score} = [40, 48, 55, 60, 66, 71, 78, 84].$$

True mean score = 62.75. Now hide two scores in two different ways.

MCAR vs MAR: what happens to the mean

MCAR — hide rows 3 and 6 (picked at random). Observed scores $[40, 48, 60, 66, 78, 84]$, mean $= 376/6 = \boxed{62.67}$. Almost exactly the truth.

MAR — hide the two students with the *lowest hours* (rows 1 and 2). Observed scores $[55, 60, 66, 71, 78, 84]$, mean $= 414/6 = \boxed{69.0}$. Biased upward by more than six marks.

Same number of blanks, same column, completely different damage. The count of missing values (`isnull().sum()` = 2 both times) cannot tell these two situations apart. Only the *pattern* can.

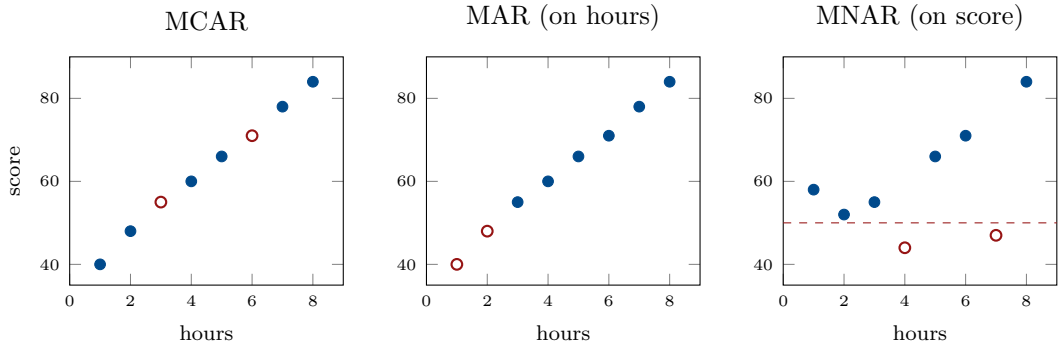


Figure — Blue = observed, red ring = value that went missing. MCAR: holes scattered anywhere. MAR: holes line up with an observed column (low hours). MNAR: holes line up with the hidden value itself (scores below 50 are not reported), wherever the hours are.

Watch Out

You cannot prove MNAR from the data alone. In the MAR example above, the two hidden scores happened to be the two lowest scores too. From the observed

table, "missing because hours were low" (MAR) and "missing because the score was low" (MNAR) can look identical. Tests can reject MCAR; telling MAR from MNAR needs *domain knowledge*: who collected the data, how, and why someone would leave it blank.

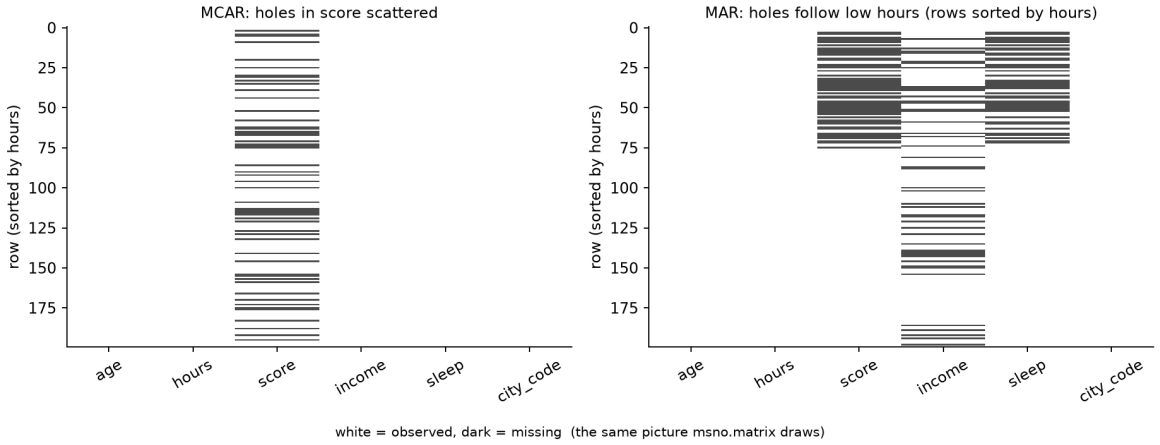


Figure — 200 simulated students. Left: 25% of scores removed by coin flip. Right: scores removed mostly when hours < 4.5; with rows sorted by hours the holes crowd at the top. In the MAR table `sleep` *goes missing together with* `score`*, and* `income` *is missing mostly for age > 45.*

19.5 Ignorable vs non-ignorable

Rubin's result, in one line: if the data are MAR (or MCAR) *and* the parameters of the missingness process ψ are unrelated to the parameters of the data, then the missingness mechanism is **ignorable** for likelihood-based inference. You can model Y_{obs} directly (maximum likelihood, EM, multiple imputation) and get valid answers without modelling R. MNAR is **non-ignorable**: you must model *why* values are missing, or run a sensitivity analysis.

	MCAR	**MAR**	**MNAR**
Missingness depends on	nothing	observed data Y_{obs}	the missing value Y_{mis}
Formula	$P(R \mid \psi)$	$P(R \mid Y_{\text{obs}}, \psi)$	$P(R \mid Y_{\text{obs}}, Y_{\text{mis}}, \psi)$
Complete rows are	a random subsample	biased, but explainable	biased, unexplainable from data
Deleting rows gives	unbiased, less precise	biased estimates	biased estimates
Testable?	can be rejected (Little)	not separable from MNAR	needs domain knowledge
Ignorable?	yes	yes	no

How to write it in Python

Simulating the three mechanisms is the fastest way to feel the difference.

```
import numpy as np, pandas as pd
rng = np.random.default_rng(0)
n = 1000
hours = rng.uniform(1, 10, n)
```

```
score = 30 + 6*hours + rng.normal(0, 5, n)
df = pd.DataFrame({"hours": hours, "score": score})

mcar = df.copy(); mcar.loc[rng.random(n) < 0.3, "score"] = np.nan
               # coin flip
mar  = df.copy(); mar.loc[(hours < 4) & (rng.random(n) < 0.8), "
    score"] = np.nan  # low hours
mnar = df.copy(); mnar.loc[(score < 55) & (rng.random(n) < 0.8), "
    score"] = np.nan # low score

for name, d in [("true", df), ("MCAR", mcar), ("MAR", mar), ("MNAR",
     mnar)]:
     print(name, round(d["score"].mean(), 2), d["score"].isnull().
         mean().round(3))
```

The MCAR mean stays near the truth; MAR and MNAR means drift upward.

Questions & Variations

Q. Is "a sensor fails when the temperature it measures goes above 50°C" MAR or MNAR?
A. MNAR: the reading is missing because of the value the sensor would have recorded. If a second, working thermometer in the same room is in the table, the missingness becomes explainable through it and is closer to MAR.

Q. Why does the mean of an MCAR column stay unbiased after deleting rows?
A. Because the deleted rows are a random subset, the remaining rows have the same distribution as the full column. Only n shrinks, so the standard error grows.

Q. Who classified missing-data mechanisms, and when?
A. Donald Rubin, *Biometrika*, 1976.

Studio — now you try

18.1 Classify each as MCAR, MAR or MNAR and give a one-line reason. (a) A hospital weighs patients only if the recorded age is over 60. (b) Wealthy households leave the income field blank. (c) A random 5% of survey pages were lost in the post. (d) Night-shift workers skip the "hours of sleep" question, and shift is recorded.
18.2 For the eight-student table, hide instead the two *highest* scores (rows 7 and 8). Compute the observed mean. Which mechanism did you just simulate if the reason was "top students did not bother to submit"? Which if the reason was "students with 7+ hours were in a different lab that lost its sheets"?
18.3 A CSV has an `age` column with values `[21, 999, 23, -1, 22, "NA"]`. Write the one line of pandas that reads it with all three sentinels as `NaN`, and report the correct mean age.

Chapter 20

Detecting Missingness: Counts, Pictures and MCAR Tests

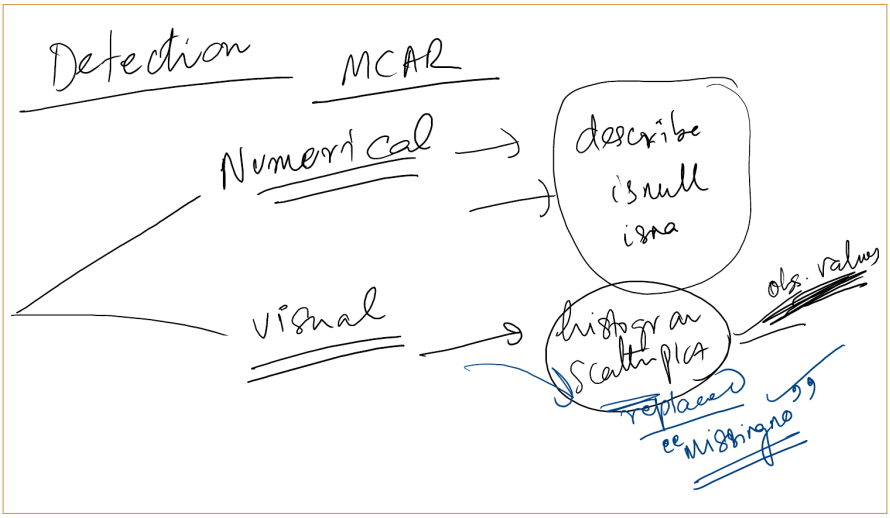
Right now we are: exploring the *holes* — before you touch a single blank, you measure how many there are, where they sit, and whether they look random.

From the class board

Detection → **Numerical**: `describe`, `isnull`, `isna` **Visual**: histogram / scatter of the observed values; the `missingno` package.
`missingno` (`msno`) → bar, matrix, heatmap, dendrogram: a **non-confirmatory signal**.
Confirmatory tests for MCAR: Little's (overall), Dixon's (univariate), Hawkins' (multivariate).

The order matters. Numbers tell you *how much* is missing. Pictures tell you *where* and *together with what*. Only a test lets you *state* "not MCAR" with a known error rate.

20.1 Numerical detection



Sir's board, Insem 2 notes, page 10 — numerical vs visual detection.

A six-row table with one hole per column

	A	B	C
0	1	2	5
1	2	NaN	3
2	NaN	6	4
3	4	8	NaN
4	5	10	1
5	6	12	0

`df.isnull().sum()` → A:1, B:1, C:1. Percentage missing = $1/6 = 16.67\%$ per column. Total cells missing = $3/18 = 16.7\%$. But rows with *at least one* hole = $3/6 = 50\%$. That last number is the one that hurts when you delete rows (next chapter).

How to write it in Python

```
df.info()                          # non-null count per column (n - missing)
df.isnull().sum()                  # missing count per column   (isna() is identical)
df.isnull().mean() * 100           # missing percentage per column
df.isnull().any(axis=1).sum()      # rows with at least one hole
df.describe()                      # 'count' row = non-missing count for numeric columns
df.isnull().value_counts()         # missingness PATTERNS and how often each occurs
```

`isnull()` and `isna()` are aliases — same function, two names.

Watch Out

`describe()` shows `count`, not `missing`. If a column has $n = 6$ rows and `count` = 5, one value is missing. Students regularly read `count` as the row count and miss this.

20.2 Visual detection: compare observed vs missing groups

The board's "histogram / scatter plot" idea: split the rows by whether column B is missing, then plot another column A for both groups. If the two histograms of A look the same, the holes in B do not care about A (consistent with MCAR). If they differ, the holes in B depend on A (MAR on A).



Figure — Hours of students whose score is observed (blue) vs missing (red). MCAR: the two shapes match (mean hours 5.19 vs 5.09). MAR: the missing group sits at low hours (5.77 vs 3.45).

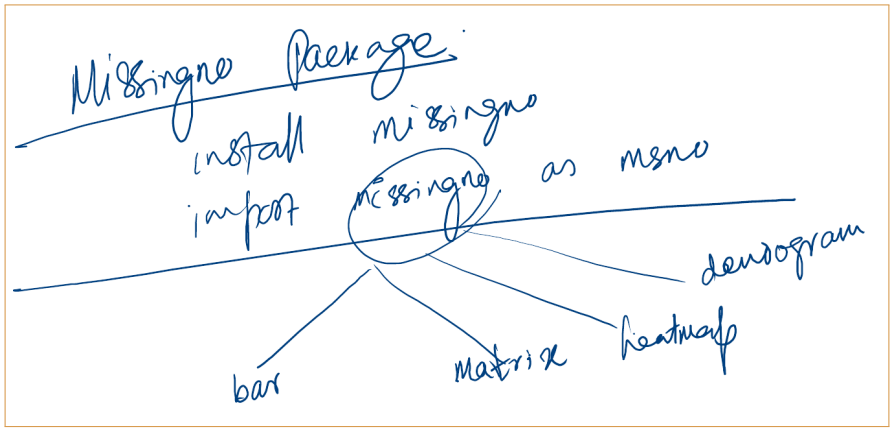


Sir's board, Insem 2 notes, page 18 — scatter sketch: two point clouds. Plot observed values coloured by missingness and look for a cloud that only one group occupies.

How to write it in Python

```python
import matplotlib.pyplot as plt
miss = df["score"].isnull()
plt.hist(df.loc[~miss, "hours"], alpha=.6, label="score observed")
plt.hist(df.loc[ miss, "hours"], alpha=.6, label="score missing")
plt.legend(); plt.xlabel("hours"); plt.show()
```

20.3 The `missingno` package



Sir's board, Insem 2 notes, page 11 — install, import as `msno`, four plots.

How to write it in Python

```python
# pip install missingno
import missingno as msno
msno.bar(df)          # non-null count per column (bars)
msno.matrix(df)       # one row per observation, white gaps = missing
    ; sparkline on the right
```

```
msno.heatmap(df)      # nullity correlation between pairs of columns
msno.dendrogram(df)   # clusters columns by how similarly they go missing
```

Plot	What it shows	What to look for
`bar`	non-missing count per column	which columns are worst; is any column almost empty?
`matrix`	every cell, white = missing, rows in file order	stripes (blocks of rows missing together, e.g. one bad day of data entry); sorted data showing holes only at one end
`heatmap`	correlation of the 0/1 *nullity* indicators between columns	values near +1: two columns go missing together; near −1: one is present exactly when the other is missing; near 0: unrelated
`dendrogram`	hierarchical clustering of columns by nullity	columns joined at a low height share a missingness pattern (same form page, same sensor)

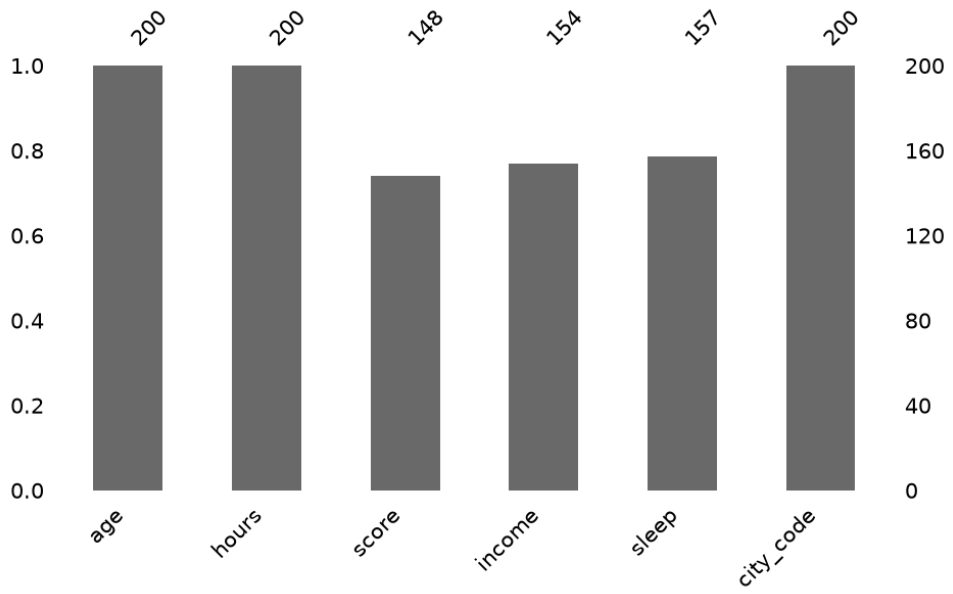


Figure — `msno.bar` on the simulated MAR table: `score` 26%, `income` 23%, `sleep` 21.5% missing; three columns complete.

Nullity correlation

Replace each column by its missing indicator $M_j = \mathbf{1}[\text{cell is missing}]$ (that is, $1 - R_j$). The `msno.heatmap` entry for columns j, k is Pearson's r between M_j and M_k. It answers "when j is missing, is k also missing?" — a question about *missingness*, not about values.

Nullity correlation of the six-row table

$M_A = [0,0,1,0,0,0]$, $M_B = [0,1,0,0,0,0]$, $M_C = [0,0,0,1,0,0]$. Each pair never misses on the same row, so every off-diagonal entry is negative: `df.isnull().astype(int).corr()` gives -0.2 for every pair. Small in size — no strong "missing together" structure.

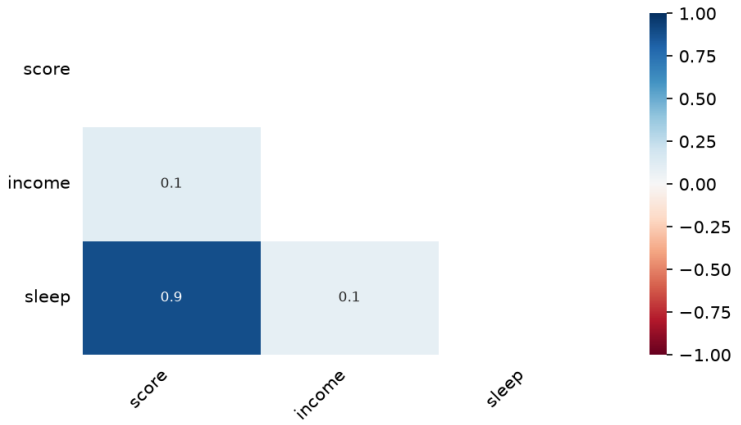
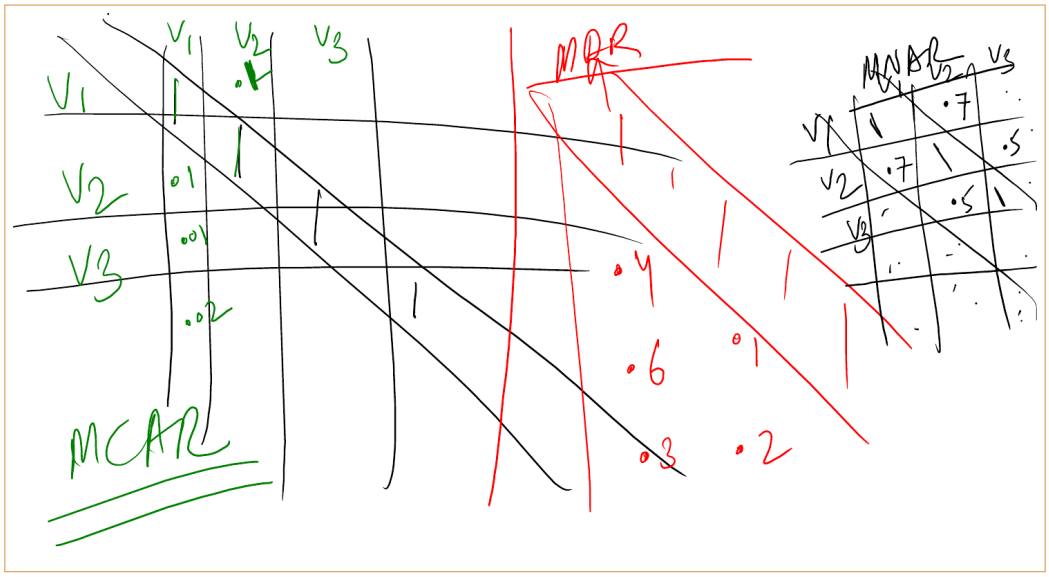


Figure — `msno.heatmap` on the same table. `sleep` and `score` have nullity correlation 0.9 (they go missing on the same rows); `income` is near 0.1 with both (its holes have another cause).



Sir's board, Insem 2 notes, page 15 — three nullity-correlation matrices: near-zero entries (MCAR), moderate (MAR), larger (MNAR label on the board).

Reading the board's three heatmaps. On the board, *sir* drew three nullity matrices: one with tiny entries (0.1, 0.01, 0.02) labelled MCAR, one with moderate entries (0.4, 0.6, 0.3) labelled MAR, and one with larger entries (0.7, 0.5) labelled MNAR. The lesson: *near-zero* nullity correlations are what random holes produce; *structured* correlations say the holes are linked to something, and you must investigate.

Reading the dendrogram. `msno.dendrogram` clusters columns by how similar their nullity patterns are. Columns that join near the bottom (height near 0) go missing on almost the same rows; columns that join only at the top are unrelated. Complete columns all join at height 0 because they are identical (never missing).

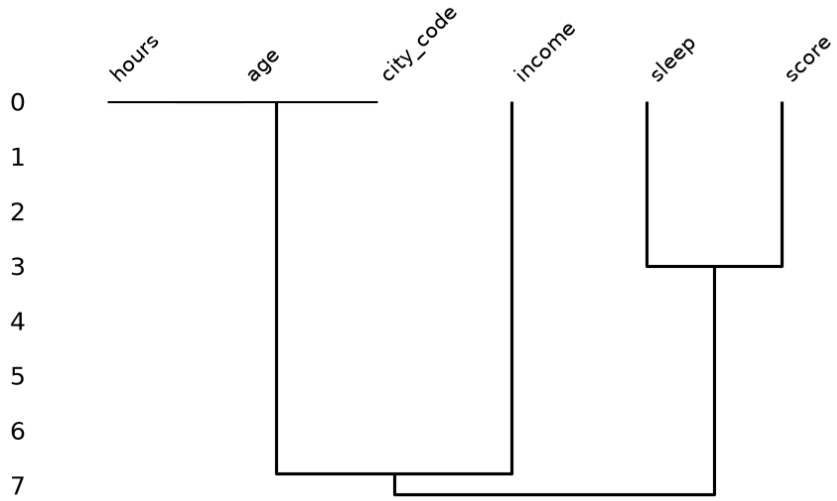
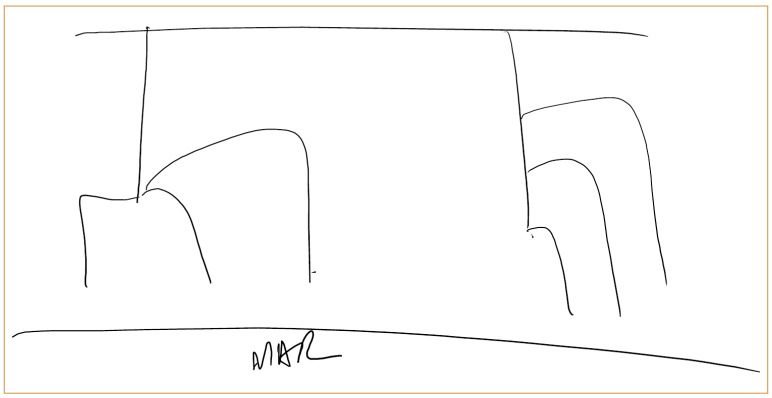
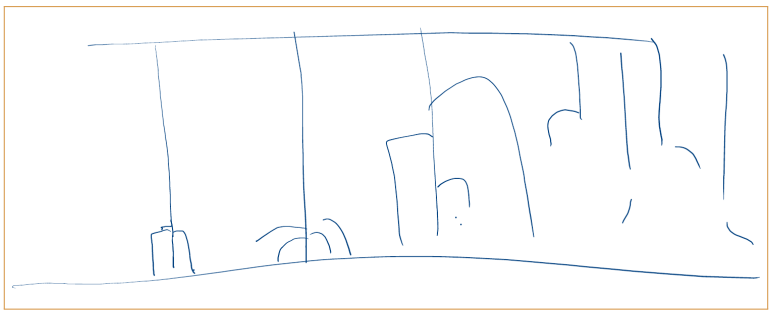


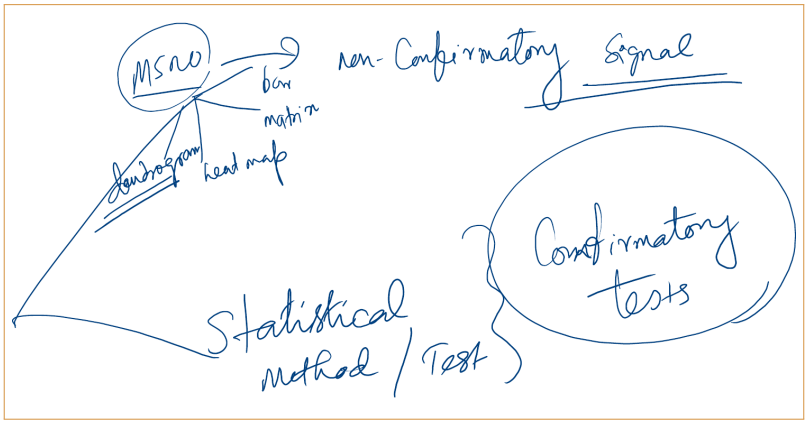
Figure — `msno.dendrogram` on the simulated table: `sleep` and `score` join low (shared missingness, a MAR-type signal), `income` joins later, and the three complete columns sit together at 0.



Sir's board, Insem 2 notes, page 16 — dendrogram sketch labelled MAR: two groups of columns joining at different heights.



Sir's board, Insem 2 notes, page 17 — a wider dendrogram: many columns, several clusters of shared missingness.

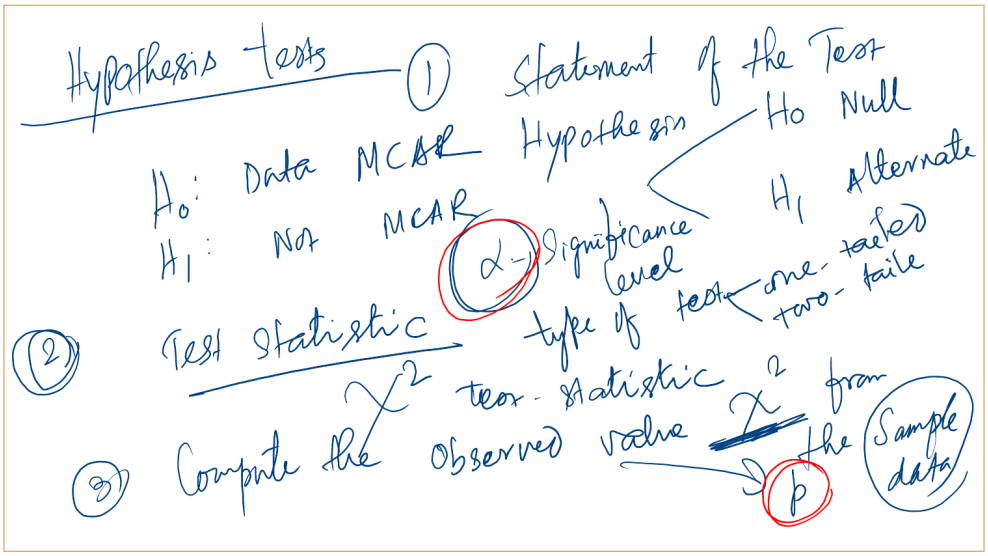


Sir's board, Insem 2 notes, page 19 — `msno` plots are a non-confirmatory signal; statistical tests are confirmatory.

Watch Out

`missingno` is a **non-confirmatory signal**. A heatmap of nullity can show that two columns go missing together (a strong hint against MCAR), but it can never *prove* MCAR, and it can never see MNAR, because the thing MNAR depends on — the hidden value — is not in the plot. To make a claim, run a test.

20.4 A two-minute primer on hypothesis tests



Sir's board, Insem 2 notes, page 21 — statement of the test, significance level α, test statistic, observed value and p.

Every confirmatory test in this chapter follows the same four steps from the board.

From the class board

1. **Statement of the test.** H_0 (null): data are MCAR. H_1 (alternate): data are not MCAR. Choose α, the significance level (usually 0.05).
2. **Test statistic.** For Little's test, a χ^2 statistic.
3. **Compute the observed value** χ^2_0 from the sample data, and its p-value.
4. **Decide.** $p > \alpha$: fail to reject H_0. $p < \alpha$: reject H_0 (not MCAR).

The vocabulary

- α **(significance level)**: the false-alarm rate you accept — the probability of rejecting H_0 when it is actually true. $\alpha = 0.05$ means "wrong 5% of the time when the data really are MCAR".
- **Test statistic**: one number that measures how far the sample is from what H_0 predicts. Big means far.
- **Critical value** χ^2_α: the cut-off with probability α to its right under H_0. Reject if $\chi^2_0 > \chi^2_\alpha$.
- p**-value**: the probability, *if* H_0 *were true*, of a statistic at least as extreme as the one you got. Small p = your data would be surprising under MCAR.

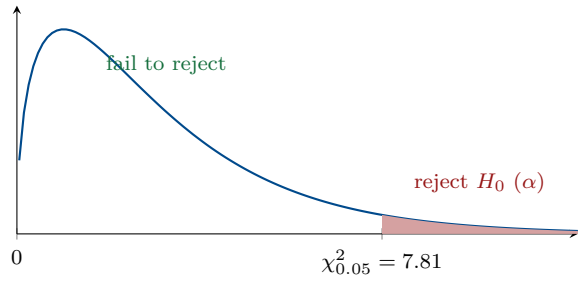
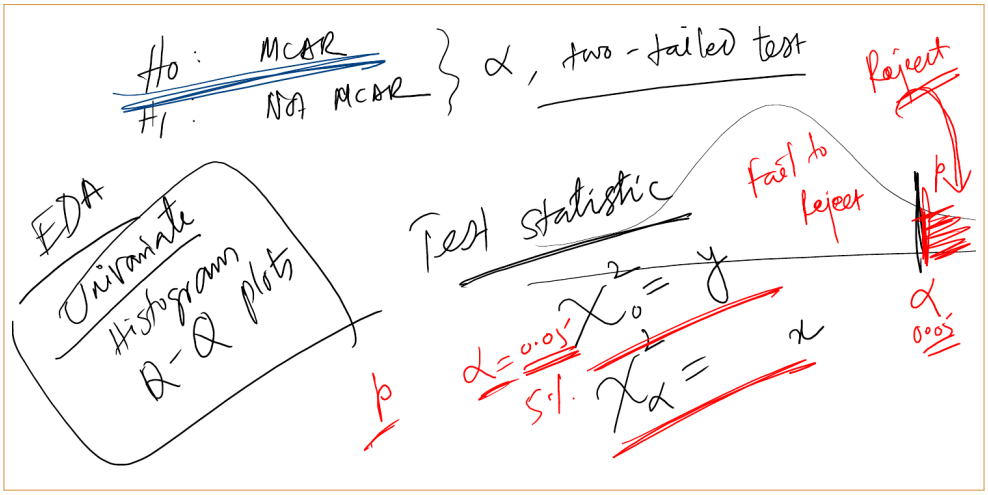


Figure — A χ^2 *test rejects only in the right tail: a large statistic means the groups differ more than chance allows. The shaded area is* α*.*

Watch Out

One tail, not two. The board note says "two-tailed", but the picture *sir* drew shades only the right tail, and that is correct for Little's test: χ^2 is a sum of squares, so only *large* values are evidence against MCAR. A small χ^2 is not suspicious. Also: "fail to reject" is not "accept". A large p means the test found no evidence against MCAR, not that MCAR is proven.



Sir's board, Insem 2 notes, page 23 — H_0 *MCAR vs* H_1 *not MCAR;* χ^2_0 *vs* χ^2_α*; the rejection region; and, boxed, univariate EDA (histogram, Q-Q plots) first.*

Why the board boxed "histogram, Q-Q plots" next to the test. Little's test (and Hawkins') assume the columns are roughly (multivariate) **normal**. Before trusting the p-value, look at each numeric column: a histogram for the shape and a **Q-Q plot** against the normal. Points on the straight line mean the normal assumption is reasonable; a curve bending away at one end means skew, and a log or Box–Cox transform (Chapter 23) may be needed first.

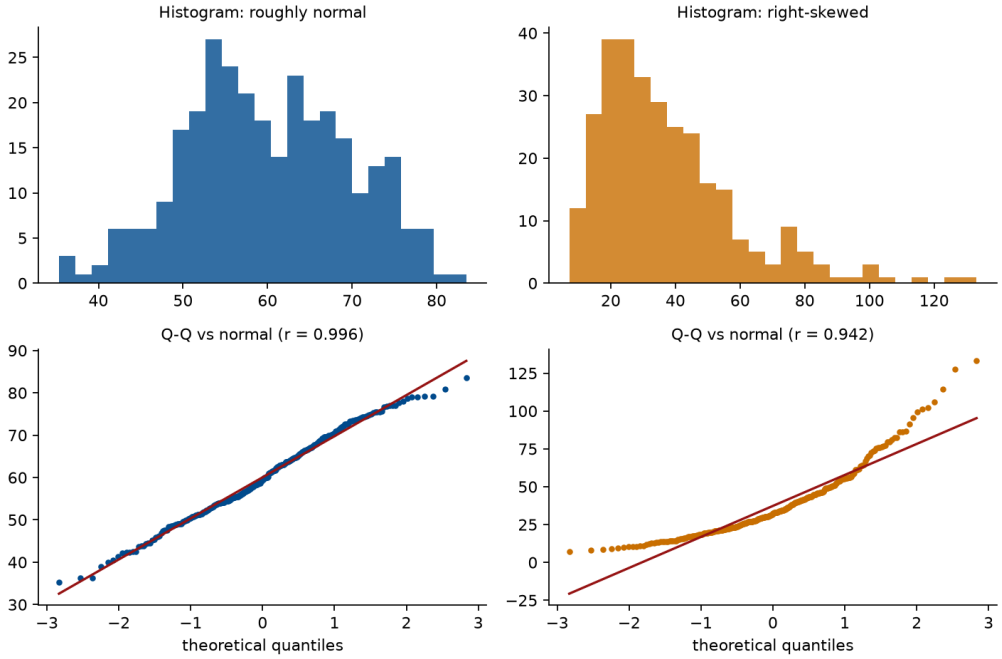
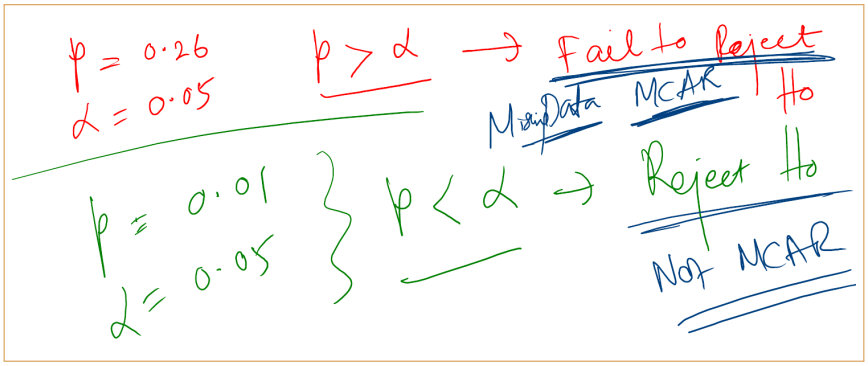


Figure — Left: a roughly normal column, Q-Q points on the line. Right: a right-skewed column, Q-Q points curve upward at the right end. Code: `scipy.stats.probplot(x, dist="norm", plot=plt)`.

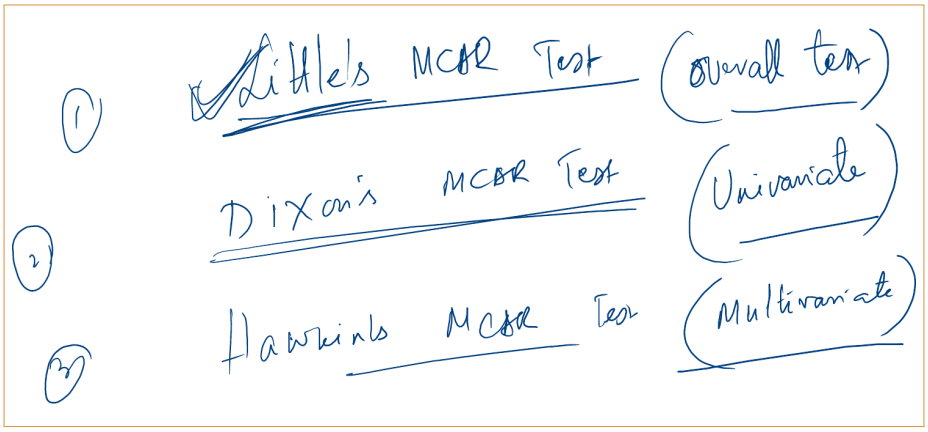


Sir's board, Insem 2 notes, page 24 — $p = 0.26 > 0.05$: fail to reject (missing data MCAR); $p = 0.01 < 0.05$: reject, not MCAR.

The board's two decisions

$p = 0.26$, $\alpha = 0.05$: $p > \alpha \Rightarrow$ **fail to reject** H_0; the data are consistent with MCAR.
$p = 0.01$, $\alpha = 0.05$: $p < \alpha \Rightarrow$ **reject** H_0; the data are **not MCAR** (so MAR or MNAR).

20.5 Little's MCAR test (overall test)



Sir's board, Insem 2 notes, page 20 — the three confirmatory tests: Little (overall), Dixon (univariate), Hawkins (multivariate).

Roderick Little (1988) proposed one global test for the whole table. It is the one ticked on the board.

Idea

Group the rows by their **missingness pattern** (which columns are blank). If the data are MCAR, every group is a random slice of the same population, so the mean of each observed column should be about the same in every group. Little's statistic adds up how far each group's means sit from the overall means, scaled by the covariance:

$$d^2 = \sum_{j=1}^{J} n_j \, (\bar{\mathbf{y}}_j - \hat{\boldsymbol{\mu}}_j)^\top \, \hat{\Sigma}_j^{-1} \, (\bar{\mathbf{y}}_j - \hat{\boldsymbol{\mu}}_j) \;\sim\; \chi^2_{\sum_j p_j - p} \quad \text{under } H_0.$$

J = number of patterns, n_j = rows in pattern j, p_j = observed columns in that pattern, p = total columns, $\bar{\mathbf{y}}_j$ = observed means in pattern j, and $\hat{\boldsymbol{\mu}}_j, \hat{\Sigma}_j$ = the overall mean and covariance (estimated by maximum likelihood with the EM algorithm) restricted to those columns.

In Plain Words

Little's test asks: "do the people with blanks look different, on the things we *can* see, from the people without blanks?" If yes, the blanks are not completely random.

Two simulations, same code

300 rows, two correlated columns $x_1 \sim N(50, 10^2)$ and $x_2 = 0.8x_1 + \text{noise}$; 20% of x_2 removed.
MCAR (coin flip): $d^2 = 0.99$, df $= 1$, $p = 0.32 > 0.05 \Rightarrow$ fail to reject.
MAR (remove x_2 mostly when $x_1 > 55$): $d^2 = 102.4$, df $= 1$, $p \approx 4 \times 10^{-24} \Rightarrow$ reject, not MCAR.
The critical value for df $= 1$ at $\alpha = 0.05$ is $\chi^2_{0.05} = 3.84$.

How to write it in Python

Little's test is not in pandas or scikit-learn. Here is a compact, readable version (EM for the mean and covariance, then the statistic).

```python
import numpy as np, pandas as pd, scipy.stats as ss

def em_mvn(X, iters=500, tol=1e-8):
    """ML mean and covariance of a multivariate normal with NaNs, via EM."""
    n, p = X.shape
    mu, S = np.nanmean(X, 0), np.diag(np.nanvar(X, 0))
    for _ in range(iters):
        T1, T2 = np.zeros(p), np.zeros((p, p))
        for x in X:
            m = np.isnan(x); o = ~m; xh = x.copy(); C = np.zeros((p, p))
            if m.any():                                   # E-step: fill with conditional mean
                Soo, Smo = S[np.ix_(o, o)], S[np.ix_(m, o)]
                xh[m] = mu[m] + Smo @ np.linalg.solve(Soo, x[o] - mu[o])
                C[np.ix_(m, m)] = S[np.ix_(m, m)] - Smo @ np.linalg.solve(Soo, Smo.T)
            T1 += xh; T2 += np.outer(xh, xh) + C
        mu2 = T1 / n; S2 = T2 / n - np.outer(mu2, mu2)    # M-step
        done = np.abs(mu2 - mu).max() < tol
        mu, S = mu2, S2
        if done: break
    return mu, S

def little_mcar(df):
    X = df.to_numpy(float); n, p = X.shape
    mu, S = em_mvn(X)
    patterns = df.isnull().astype(int).astype(str).agg("".join, axis=1)
    d2, dof = 0.0, 0
    for _, idx in patterns.groupby(patterns).groups.items():
        o = ~np.isnan(X[idx[0]])                          # observed columns in this pattern
        if not o.any(): continue
        diff = X[np.ix_(idx, o)].mean(0) - mu[o]
        d2 += len(idx) * diff @ np.linalg.solve(S[np.ix_(o, o)], diff)
        dof += o.sum()
    dof -= p
    return d2, dof, ss.chi2.sf(d2, dof)                   # statistic, df, p-value

d2, dof, p = little_mcar(df[["x1", "x2"]])
print("not MCAR" if p < 0.05 else "consistent with MCAR", round(p, 4))
```

In R the same test is `naniar::mcar_test()`; the `pyampute` Python package also ships an `MCARTest`.

20.6 Dixon's test (univariate)

The univariate version, popularised in the BMDP software by Dixon (1988): for each column B with holes, split the rows into "B missing" and "B observed", then run a two-sample t-test

on every other column A. A significant difference means the holes in B depend on A.

Dixon-style t-test on the eight students

Compare `hours` for students whose score is missing vs observed.

MCAR case (rows 3, 6 missing): missing-group hours $[3, 6]$, mean 4.5; observed-group hours $[1, 2, 4, 5, 7, 8]$, mean 4.5. Difference 0, $t = 0$, $p = 1$. No evidence against MCAR.

MAR case (rows 1, 2 missing): missing-group hours $[1, 2]$, mean 1.5, variance 0.5; observed-group hours $[3, \ldots, 8]$, mean 5.5, variance 3.5. Welch's statistic

$$t = \frac{1.5 - 5.5}{\sqrt{0.5/2 + 3.5/6}} = \frac{-4}{\sqrt{0.8333}} = \frac{-4}{0.9129} = \boxed{-4.38}, \qquad p = 0.006 < 0.05.$$

Reject: score is missing more often for low-hours students. Not MCAR.

How to write it in Python

```
import scipy.stats as ss
miss = df["score"].isnull()
for col in df.columns.drop("score"):
    t, p = ss.ttest_ind(df.loc[miss, col], df.loc[~miss, col],
        equal_var=False, nan_policy="omit")
    print(col, round(t, 3), round(p, 4))
```

Watch Out

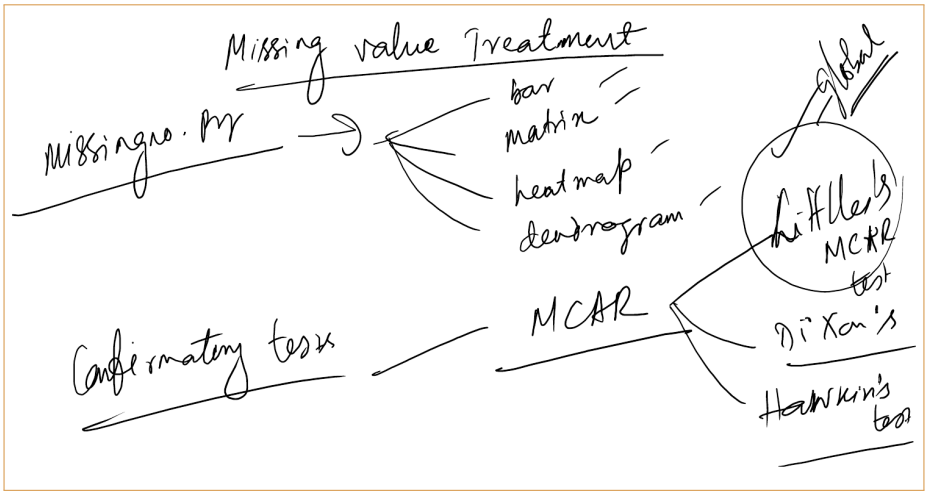
Many columns means many t-tests, and with $\alpha = 0.05$ about one in twenty will be "significant" by chance. That is why Little's single global test is preferred as the headline answer, with Dixon-style tests used to find *which* column drives the pattern.

20.7 Hawkins' test (multivariate)

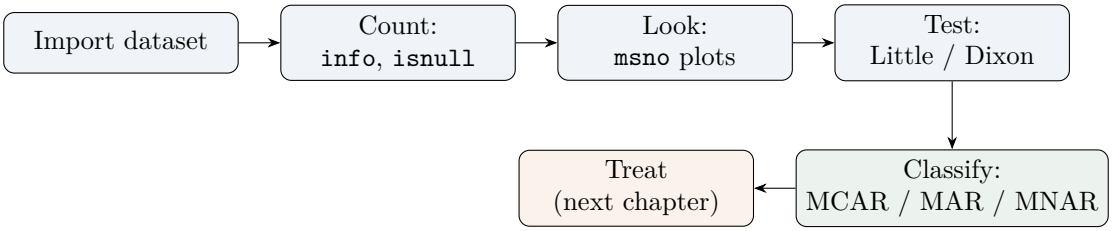
Hawkins (1981) built a test for whether several groups share the same covariance matrix and are multivariate normal. Jamshidian and Jalal (2010) turned it into an MCAR test: impute the data, group rows by missingness pattern, and check that all groups have *equal covariances* (homoscedasticity). Under MCAR, every pattern group is a random slice of one population, so their covariances must agree. If they disagree, reject MCAR. It complements Little's test, which looks only at *means*. It is implemented in the R package `MissMech` (`TestMCARNormality`); there is no standard Python port.

Test	Scope	Compares across missingness groups
Little (1988)	overall / global	means (one χ^2 for the whole table)
Dixon (1988)	univariate	mean of one column, t-test per column pair
Hawkins (1981), via Jamshidian–Jalal (2010)	multivariate	covariance matrices (and normality)

20.8 The detection workflow



Sir's board, Insem 2 notes, page 22 — missing value treatment starts with missingno (bar, matrix, heatmap, dendrogram) and confirmatory tests.



Questions & Variations

Q. Can `msno.heatmap` detect MNAR?
A. No. It only correlates missing indicators with each other. MNAR depends on the hidden values, which no plot of the observed table can show.

Q. Little's test gives $p = 0.40$. Are the data MCAR?
A. We fail to reject MCAR at $\alpha = 0.05$. The data are *consistent* with MCAR; it is not a proof, and a small sample has little power to detect departures.

Q. What are the degrees of freedom of Little's test?
A. $\sum_j p_j - p$: the observed columns summed over all patterns, minus the total number of columns.

Studio — now you try

19.1 For the six-row table, list the missingness patterns (as strings like `"010"` for A, B, C) and how many rows have each. How many patterns are there? What would the degrees of freedom of Little's test be?
19.2 Little's test on a survey gives $\chi^2_0 = 11.2$ with df $= 4$. The critical value $\chi^2_{0.05}(4) = 9.49$. State H_0, H_1, the decision, and the conclusion in plain words. Would the decision change at $\alpha = 0.01$, where $\chi^2_{0.01}(4) = 13.28$?
19.3 Run the Dixon-style comparison yourself for the MAR case if instead rows 7 and 8 are missing (high hours). Compute both group means, both variances, and Welch's t.

Chapter 21

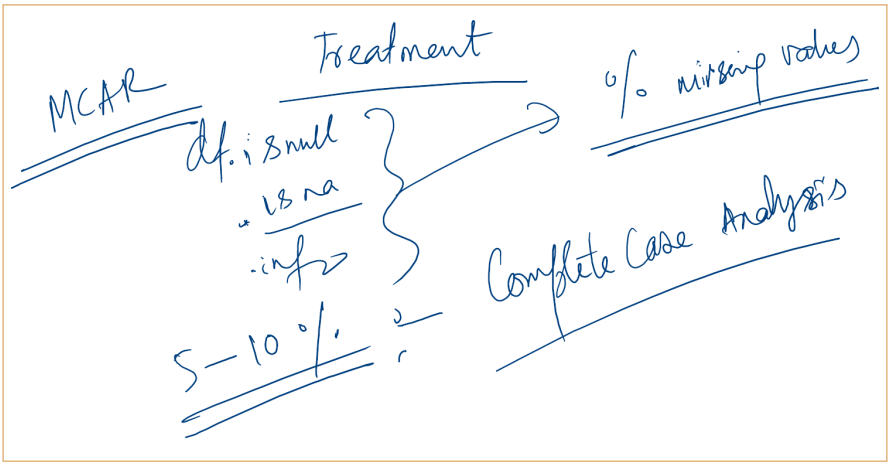
Treating Missing Values: Delete, Impute, or Model

Right now we are: managing — you know how many holes there are and roughly why. Now you choose a fix, and the fix must match the mechanism.

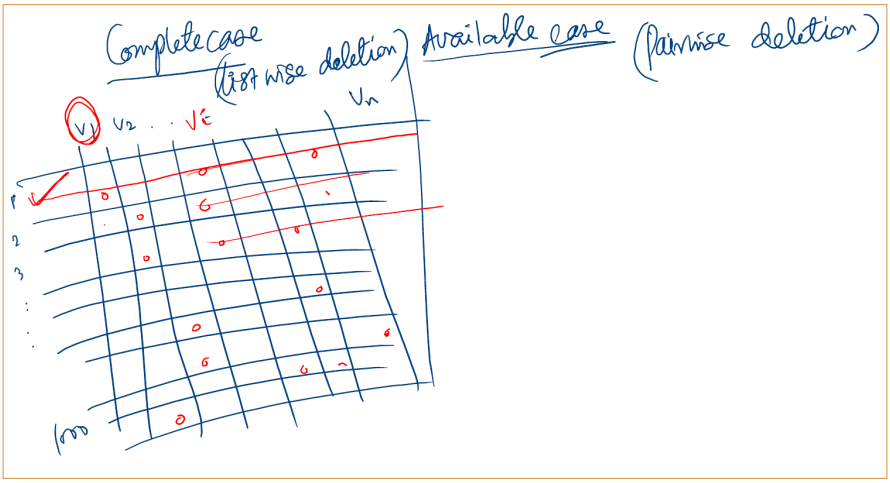
From the class board

Treatment (MCAR). Check % missing with `df.isnull`, `.isna`, `.info`. If small (about 5–10%) → complete case analysis.
Complete case (listwise deletion) **Available case** (pairwise deletion)
Imputation: mean, median (numeric data); function → regression, e.g. $V_1 = c_0 + c_1 V_2 + c_3 V_3 + \varepsilon$.
Workflow: import → check for missing → classify (MCAR / MAR / MNAR) → treat (deletion, `SimpleImputer` / `IterativeImputer` in scikit-learn, maximum likelihood).

21.1 Deletion



Sir's board, Insem 2 notes, page 25 — under MCAR, check the percentage missing; if about 5–10%, complete case analysis.



Sir's board, Insem 2 notes, page 26 — complete case (listwise) vs available case (pairwise) on a grid of rows and variables.

21.1.1 Complete case analysis (listwise deletion)

Drop every row that has *any* missing value, then analyse what is left. `df.dropna()`.

21.1.2 Available case analysis (pairwise deletion)

Keep every row. For each calculation, use all rows that have the values *that calculation needs*. The mean of A uses all rows where A is present; the correlation of A and B uses all rows where both are present. pandas does this by default: `df.mean()` and `df.corr()` skip `NaN` pair by pair.

Same six rows, two deletion rules

Table from the detection chapter: one hole in each of A, B, C, on three different rows.

Listwise: rows 0, 4, 5 survive. $n = 3$ out of 6 — half the data gone for three holes. Mean of $A = (1 + 5 + 6)/3 = 4.0$.

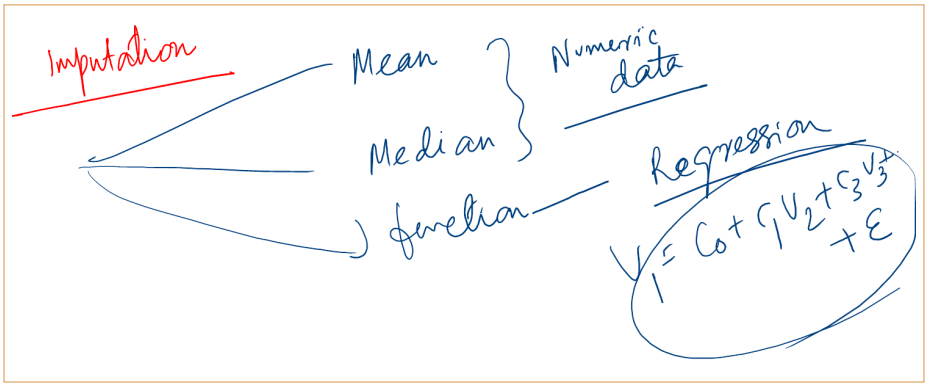
Pairwise: mean of A uses its five present values, $(1 + 2 + 4 + 5 + 6)/5 = 3.6$. Each correlation uses 4 rows (every pair loses two rows). $r(A, C)$: listwise -1.000 (the three surviving rows happen to lie exactly on the line $C = 6 - A$), pairwise -0.979. Same table, different answers, purely from the deletion rule.

	Listwise (complete case)	**Pairwise (available case)**
Rows used	only fully complete rows	different rows for each statistic
Loss of data	large; grows fast with many columns	small
Under MCAR	unbiased, less precise	unbiased, uses more data
Under MAR / MNAR	biased	biased
Side effect	simple, consistent n	each statistic has its own n; a correlation matrix can become impossible (not positive semi-definite)

Watch Out

Small per column is not small per row. With 20 columns each missing 5% at random, the chance a row is complete is $0.95^{20} = 0.358$. Listwise deletion throws away about 64% of rows. The board's "5–10%" rule is about the share of *rows* you would lose, and it only applies when the data are MCAR.

21.2 Simple imputation



Sir's board, Insem 2 notes, page 27 — imputation by mean, median (numeric data), or a function such as a regression.

Imputation means filling each blank with a plausible value so that every row can be used.

Single-value imputation

- **Mean**: numeric, roughly symmetric columns.
- **Median**: numeric, skewed columns or columns with outliers.
- **Mode (most frequent)**: categorical columns.
- **Constant**: a flag value such as `"Unknown"` or 0 when "missing" is itself meaningful (for example "no second phone number").
- **Forward / backward fill**: time series, carry the last observation forward (`ffill`).

Mean imputation shrinks the spread

Eight students, true scores $[40, 48, 55, 60, 66, 71, 78, 84]$, true SD $= 14.95$.
MCAR (rows 3, 6 blank): observed mean 62.67. Fill both blanks with 62.67. SD drops to $\boxed{14.33}$.
MAR (rows 1, 2 blank, low hours): observed mean 69.0. Fill both with 69.0. Mean stays at the biased 69.0 and SD collapses to $\boxed{9.23}$.

Mean imputation puts every filled value at the centre. The column looks more certain than it is: variance shrinks, correlations with other columns weaken, and under MAR the mean stays biased.

Watch Out

Mean imputation is fine for a quick look, dangerous for inference. It keeps the mean (under MCAR), but it understates the variance and weakens every correlation. Add a **missing indicator** column (`SimpleImputer(add_indicator=True)`) if the fact

that a value was missing might carry information.

21.3 Imputation by a function: regression

Use the other columns to *predict* the missing one. The board's form:

$$V_1 = c_0 + c_1 V_2 + c_3 V_3 + \varepsilon.$$

Fit the model on rows where V_1 is observed, then predict V_1 for the rows where it is missing.

Regression imputation fixes the MAR bias

MAR case: scores of students 1 and 2 (hours 1, 2) are missing. Fit score $= c_0 + c_1$ hours on the six observed rows (hours 3–8):

$$c_1 = \frac{\sum(h-\bar{h})(s-\bar{s})}{\sum(h-\bar{h})^2} = 5.829, \qquad c_0 = \bar{s} - c_1\bar{h} = 69.0 - 5.829 \times 5.5 = 36.94.$$

Predictions: hours 1 → 42.77, hours 2 → 48.60 (truth: 40 and 48). Imputed column mean = $\boxed{63.17}$ vs true 62.75 (mean imputation gave 69.0). Imputed SD = 14.28 vs true 14.95 (mean imputation gave 9.23).

It works because the missingness depends on `hours`, and `hours` is in the model. That is exactly the MAR situation.

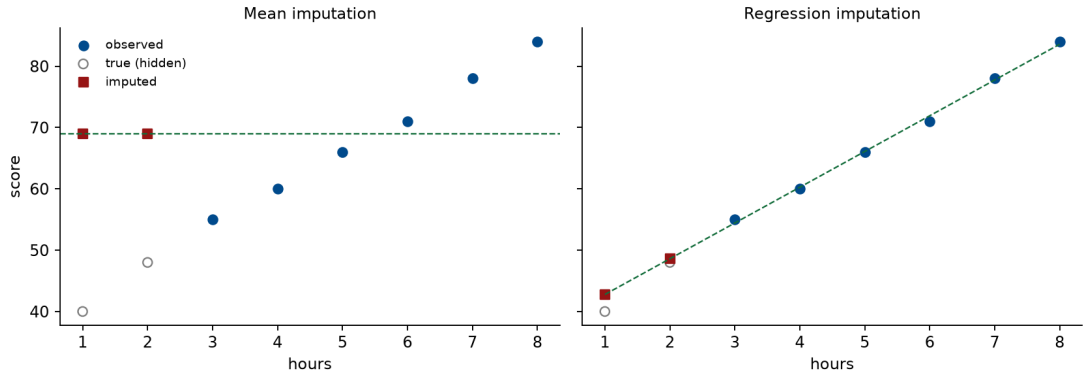


Figure — The MAR case (two low-hours scores hidden). Mean imputation drops both fills on a flat line at 69, far above the hidden truth. Regression imputation puts them on the trend, close to the true 40 and 48.

Better relatives of regression imputation

- **Stochastic regression**: prediction + a random residual drawn from the fitted noise, so filled values are not all on the line (keeps the variance honest).
- **k-nearest neighbours** (`KNNImputer`): average the column over the k most similar complete rows.
- **Iterative / MICE** (Multivariate Imputation by Chained Equations, `IterativeImputer`): impute every column from all the others with a regression, cycle round the columns several times until the fills settle.
- **Multiple imputation**: create m different imputed datasets (with randomness), anal-

yse each, and pool the results (Rubin's rules). The spread between the m answers measures the extra uncertainty caused by the missing data.

21.4 Model-based: maximum likelihood (EM)

Instead of filling holes and pretending they were data, estimate the parameters directly from the incomplete table. The **EM algorithm** alternates: **E-step** — given current $\boldsymbol{\mu}, \Sigma$, compute the expected value of each missing cell from the observed cells in its row; **M-step** — re-estimate $\boldsymbol{\mu}, \Sigma$ from the completed sufficient statistics. Repeat until nothing changes. The detection chapter's `em_mvn` function is exactly this. Under MAR, ML estimates are unbiased; it is the "use specific ML algorithms" branch on the board.

Another way to do it

Some models handle `NaN` natively and need no imputation at all: gradient-boosted trees in scikit-learn (`HistGradientBoostingRegressor`), XGBoost and LightGBM learn which branch a missing value should go down.

21.5 Matching the fix to the mechanism

Mechanism	Safe choices	Avoid
MCAR, few rows lost	listwise deletion; mean/median for a quick look	nothing in particular
MCAR, many rows lost	pairwise; regression, KNN, MICE	listwise (wastes data)
MAR	regression, MICE, multiple imputation, ML/EM — using the columns that explain the missingness	listwise and mean imputation (biased)
MNAR	model the missingness (selection / pattern-mixture models), sensitivity analysis, collect more data, add a "was missing" indicator	pretending it is MAR without saying so

How to write it in Python

```
import numpy as np, pandas as pd
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer   # noqa: needed once
from sklearn.impute import IterativeImputer

df.dropna()                                          # listwise deletion
df.dropna(subset=["score"])                          # drop rows only if 'score' is missing
df.dropna(thresh=3)                                  # keep rows with at least 3 non-missing values
df["score"].fillna(df["score"].median())             # median imputation, pandas
```

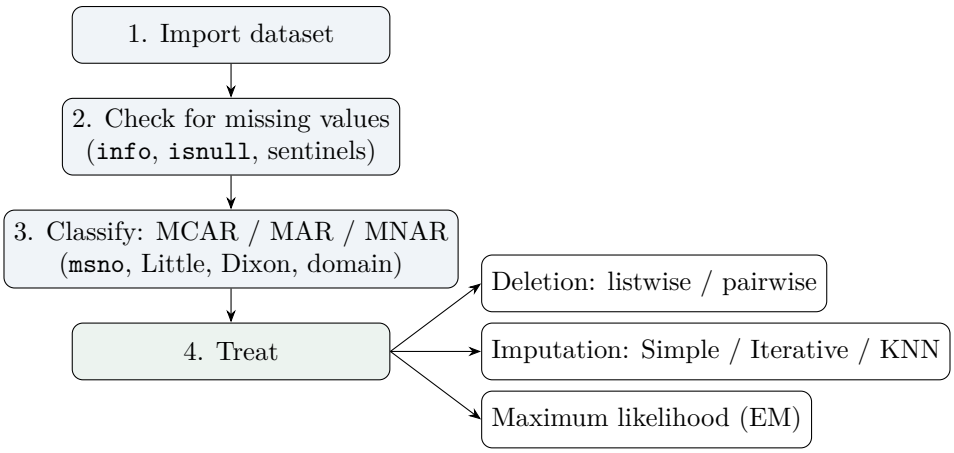
```
df["city"].fillna(df["city"].mode()[0])        # mode imputation for a category

num = ["hours", "score"]
SimpleImputer(strategy="mean").fit_transform(df[num])            # mean | median | most_frequent | constant
KNNImputer(n_neighbors=3).fit_transform(df[num])
IterativeImputer(max_iter=10, random_state=0).fit_transform(df[num])        # MICE-style
```

Watch Out

Fit imputers on the training data only. Compute the mean (or fit the regression) on the training split, then apply the same numbers to the test split. Computing the mean on all data leaks test information into training. scikit-learn's `fit` / `transform` split exists for exactly this.

21.6 The full workflow, from the students' notes



Questions & Variations

Q. Why does listwise deletion stay unbiased under MCAR but not under MAR?
A. Under MCAR the complete rows are a random subsample. Under MAR they over-represent the rows whose observed values make missingness unlikely, so their mean and spread are shifted.

Q. Mean or median imputation for a column of incomes?
A. Median: incomes are right-skewed and a few large values drag the mean up.

Q. What extra information does multiple imputation give that single imputation does not?
A. The between-imputation variability, i.e. how much the answer changes because of the missing values. Single imputation treats filled values as known and underestimates standard errors.

Studio — now you try

20.1 A table has 12 columns, each independently missing 3% at random. What fraction of rows survives listwise deletion? Would the board's "5–10%" rule allow deletion here?

20.2 For the eight-student MCAR case (rows 3, 6 blank), the regression fit on the six observed rows is score $= 34.83 + 6.187$ hours. Compute the two imputed values and the imputed mean. Compare with the true values 55 and 71.

20.3 A column `income` is missing for 30% of rows, and Little's test rejects MCAR. The survey team tells you high earners often refused to answer. Name the mechanism, explain why `SimpleImputer(strategy="median")` would understate average income, and suggest two better actions.

Chapter 22

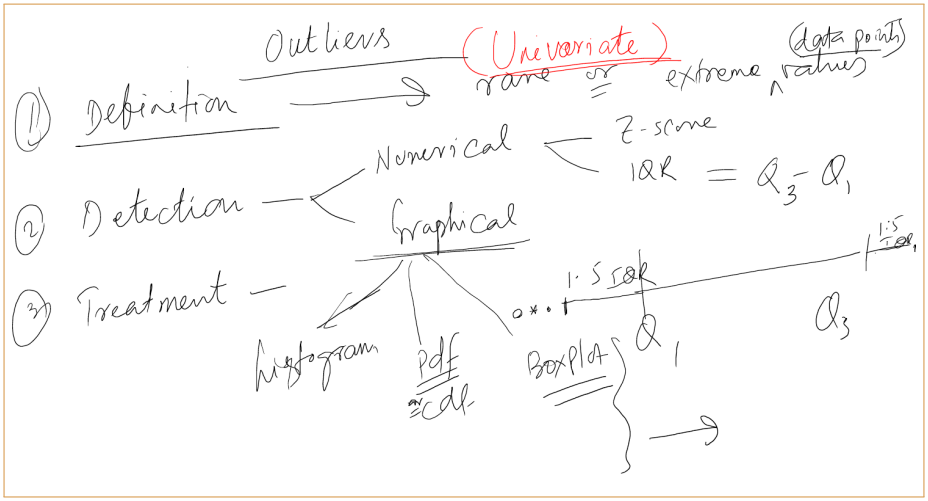
Outliers: Detection and Treatment

Right now we are: exploring, then managing — find the values that sit far from the rest, decide whether they are errors or real, and only then decide what to do.

From the class board

Outliers (univariate). 1. Definition: rare or extreme values (data points). 2. Detection: numerical (z-score, IQR $= Q_3 - Q_1$) and graphical (histogram, pdf/cdf, boxplot with 1.5 IQR whiskers). 3. Treatment: remove; replace (mean, median, percentile, range-random; winsorization, trimming); imputation using specific ML algorithms.
Multivariate. Extend IQR to the correlation matrix; graphical: heatmap, scatter plots, pair plots; complex / high-dimensional data: PCA, LOF, PyOD.

22.1 Definition



Sir's board, Insem 2 notes, page 01 — univariate outliers: definition, detection (numerical, graphical), treatment.

Outlier

An **outlier** is an observation that lies far from most of the other observations. From the students' notes, three facts worth memorising:

1. Outliers are defined for **numeric** (interval or ratio) data; very rarely for ordinal data. "Far" needs a distance, and nominal categories have none.

2. Outliers are always a **minority** (typically well under 5% of the data). If 30% of points are "outliers", you have two populations, not outliers.
3. In a normal distribution, ±3 standard deviations cover 99.7% of the data; the remaining 0.3% are the classic outlier zone.

Where outliers come from. **Errors**: typing 950 for 95, a unit mix-up (grams vs kilograms), a sensor glitch, a sentinel like 999. **Genuine extremes**: a billionaire in an income survey, a 7-foot basketball player, a real fraud transaction. Errors should be fixed or removed. Genuine extremes are often the most interesting rows in the file — fraud detection *is* outlier detection.

Watch Out

Never delete an outlier just because it is far away. First ask: is it **possible** (feasible range), is it **plausible** (domain knowledge), and does it **matter** (does removing it change the conclusion)? Write the decision down.

22.2 Univariate detection: numerical rules

We use one running example: delivery times (minutes) for ten orders,

$$x = [12,\ 13,\ 13,\ 14,\ 15,\ 15,\ 16,\ 17,\ 18,\ 95].$$

The 95 is obviously suspicious. Watch which rules catch it.

22.2.1 The z-score rule

$$z_i = \frac{x_i - \bar{x}}{s}, \qquad \text{flag } x_i \text{ if } |z_i| > 3.$$

z-score on the delivery times

$\bar{x} = 228/10 = 22.8$. Population SD $\sigma = 24.13$ (sample SD $s = 25.44$).

$$z_{95} = \frac{95 - 22.8}{24.13} = \boxed{2.99} \quad (\text{with } s:\ 2.84).$$

Not flagged, even though 95 is five times the typical value. The outlier dragged the mean from about 14.8 up to 22.8 and inflated the SD from about 2 to 24 — it hid itself. This is called **masking**.

Derivation — step by step

Why $|z| > 3$ is impossible in small samples. With the population SD, one can show $\max_i |z_i| \le \sqrt{n-1}$; with the sample SD, $\max_i |z_i| \le (n-1)/\sqrt{n}$. For $n = 10$ these bounds are 3.00 and 2.85. So with ten points, the "$|z| > 3$" rule can *never* fire, no matter how extreme the value. The z-rule needs a reasonably large sample and a roughly normal column.

22.2.2 The IQR rule (Tukey's fences)

$$\text{IQR} = Q_3 - Q_1, \qquad \text{flag } x \text{ if } x < Q_1 - 1.5\,\text{IQR} \ \text{ or } \ x > Q_3 + 1.5\,\text{IQR}.$$

IQR fences on the delivery times

Sorted data already. With numpy's default (linear) percentiles: $Q_1 = 13.25$, $Q_3 = 16.75$, IQR $= 3.5$, $1.5\,\text{IQR} = 5.25$.

$$\text{lower fence} = 13.25 - 5.25 = 8.0, \qquad \text{upper fence} = 16.75 + 5.25 = 22.0.$$

$95 > 22 \Rightarrow$ **flagged**. Everything else lies inside $[8, 22]$. Quartiles barely move when one extreme value is added, so the IQR rule does not suffer from masking.

Derivation — step by step

Why 1.5? For a normal distribution, $Q_1 = \mu - 0.6745\sigma$ and $Q_3 = \mu + 0.6745\sigma$, so IQR $= 1.349\sigma$. The upper fence is $\mu + 0.6745\sigma + 1.5(1.349\sigma) = \mu + 2.698\sigma$. The share of a normal beyond $\pm 2.698\sigma$ is about 0.70%. So Tukey's fences flag roughly 0.7% of clean normal data — close to, but a bit stricter than, the 0.3% of the 3σ rule. A multiplier of 3 instead of 1.5 marks "extreme" outliers. The students' note is right that 1.4 or 2 can be used; 1.5 is the standard convention.

22.2.3 A robust z-score (modified z)

Replace mean and SD by median and **MAD** (median absolute deviation), which an outlier cannot drag:

$$M_i = \frac{0.6745\,(x_i - \tilde{x})}{\text{MAD}}, \qquad \text{MAD} = \text{median}\,|x_i - \tilde{x}|, \qquad \text{flag if } |M_i| > 3.5.$$

Modified z on the delivery times

Median $\tilde{x} = 15$. Absolute deviations $[3, 2, 2, 1, 0, 0, 1, 2, 3, 80]$, median MAD $= 2$. $M_{95} = 0.6745 \times 80/2 = \boxed{26.98}$ — flagged by a mile. $M_{18} = 0.6745 \times 3/2 = 1.01$, fine. This is the rule from Iglewicz and Hoaglin, recommended by the NIST handbook.

How to write it in Python

```
import numpy as np, pandas as pd
x = pd.Series([12,13,13,14,15,15,16,17,18,95])

z = (x - x.mean()) / x.std(ddof=0)                    # z-score
print(x[z.abs() > 3])                                  # empty:
    masking!

q1, q3 = x.quantile([.25, .75]); iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
print(lo, hi, x[(x < lo) | (x > hi)].tolist())         # 8.0 22.0 [95]

med = x.median(); mad = (x - med).abs().median()
mz = 0.6745 * (x - med) / mad
print(x[mz.abs() > 3.5].tolist())                      # [95]
```

22.3 Univariate detection: graphical

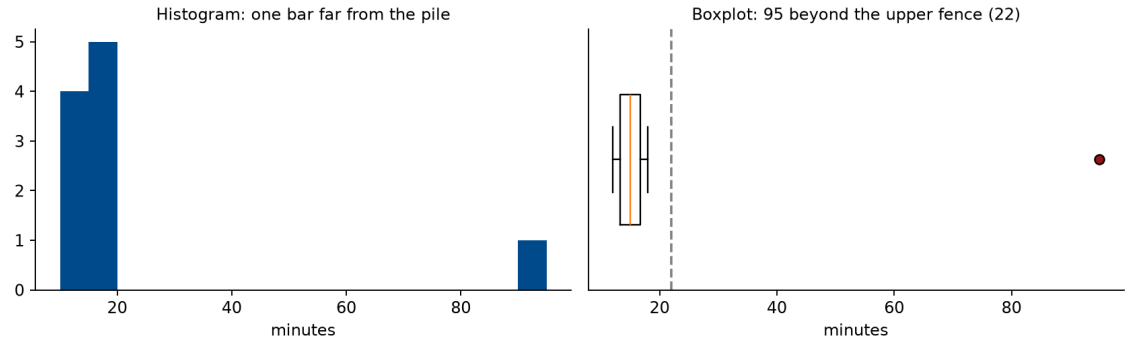


Figure — The ten delivery times. The histogram has one isolated bar; the boxplot draws 95 as a lone point beyond the upper fence.

- **Boxplot**: the IQR rule drawn as a picture. Points beyond the whiskers are the flagged outliers. `sns.boxplot(x=df["col"])`.
- **Histogram**: an isolated bar far from the main pile (the board's sketch).
- **pdf / cdf**: a KDE with a long thin tail, or an ECDF that reaches 0.9 quickly and then crawls to 1.

22.4 Multivariate outliers

Some points are ordinary in every single column but strange as a *combination*: a person 165 cm tall is normal, 78 kg is normal, but 165 cm *and* 78 kg in a group where weight tracks height closely is unusual.

Normal in each column, outlier together

Twelve people. Eleven follow weight = height −105 exactly: $(155, 50), (158, 53), \ldots, (180, 75)$. The twelfth is $(165, 78)$.
Univariate IQR: height fences [146.5, 188.5], weight fences [37.0, 91.0]. Both 165 and 78 are inside. *Not flagged.*
Correlation: $r = 1.00$ without the point, $r = 0.82$ with it — one row cuts the correlation sharply.
Mahalanobis distance (distance that accounts for the correlation):

$$d^2(\mathbf{x}) = (\mathbf{x} - \bar{\mathbf{x}})^\top S^{-1} (\mathbf{x} - \bar{\mathbf{x}}) \quad \Rightarrow \quad d^2 = 10.08 \text{ for } (165, 78),$$

while all other points have $d^2 \leq 2.64$. Compare with χ^2_2: the 97.5% cut-off is 7.38 and the 99% cut-off is 9.21. **Flagged.**

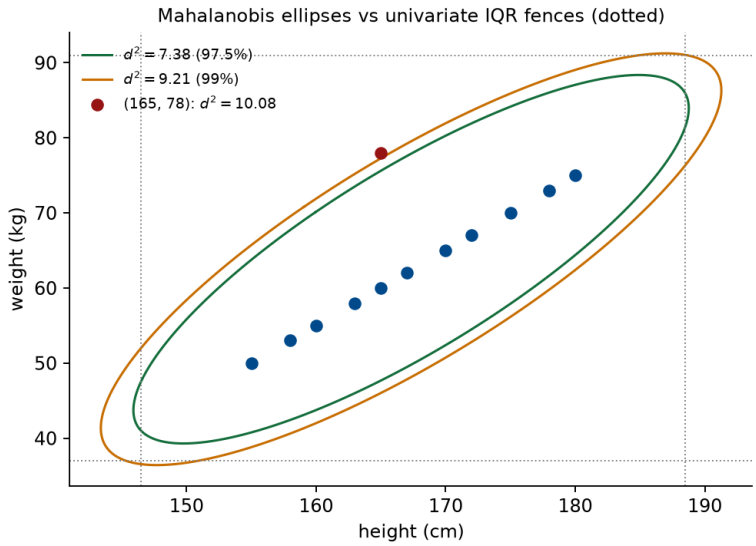
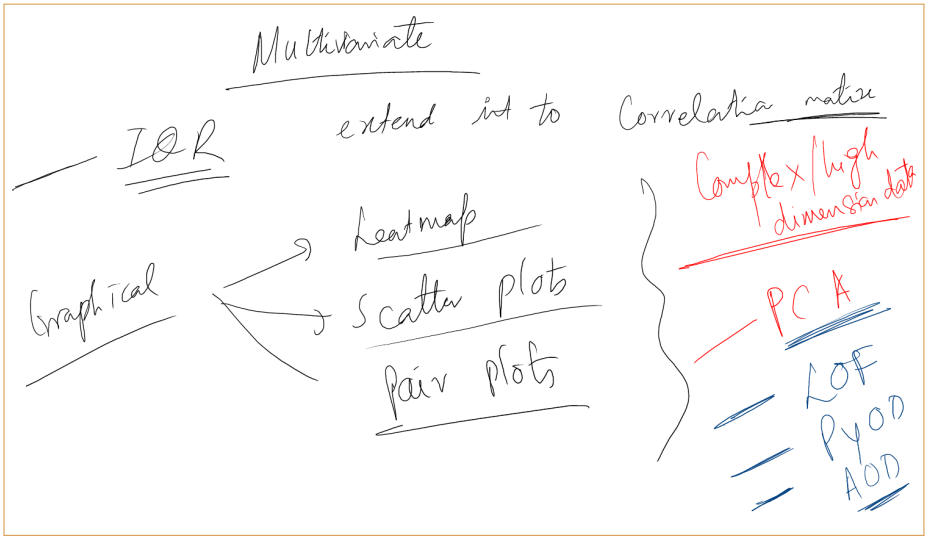


Figure — Mahalanobis distance draws ellipses along the trend. $(165, 78)$ *is inside both dotted IQR fences but outside the 99% ellipse.*



Sir's board, Insem 2 notes, page 02 — multivariate: correlation matrix, heatmap, scatter and pair plots; PCA, LOF, PyOD, AOD for complex high-dimensional data.

Multivariate toolkit, from the board

- **Graphical**: correlation heatmap (which pairs are strongly related and worth a look), scatter plots, pair plots (`sns.pairplot`) — every pair at once.
- **Mahalanobis distance**: the multivariate z-score, compared with a χ^2_p cut-off.
- **PCA**: project to a few principal components; points far out along a component, or with a large reconstruction error, are outliers.
- **LOF (Local Outlier Factor)**: compares a point's local density with that of its k neighbours. LOF ≈ 1 means as dense as its neighbours; LOF $\gg 1$ means isolated.
- **PyOD**: a Python library with 40+ outlier / anomaly detection (the board's "AOD") algorithms behind one `fit` / `predict` interface (Isolation Forest, LOF, kNN, ECOD, autoencoders, …).

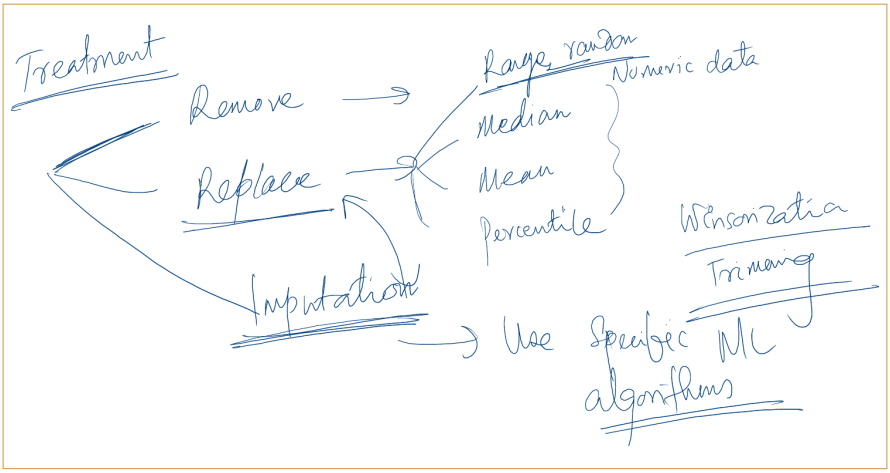
How to write it in Python

```python
import numpy as np, scipy.stats as ss
from sklearn.neighbors import LocalOutlierFactor
X = df[["height", "weight"]].to_numpy()
d = X - X.mean(0); Si = np.linalg.inv(np.cov(X.T))
d2 = np.einsum("ij,jk,ik->i", d, Si, d)              # Mahalanobis squared, one per row
print(df[d2 > ss.chi2.ppf(0.99, df=X.shape[1])])

lof = LocalOutlierFactor(n_neighbors=5)
labels = lof.fit_predict(X)                           # -1 = outlier, 1 = inlier

# PyOD (pip install pyod): same idea, many algorithms
# from pyod.models.iforest import IForest
# IForest().fit(X).labels_                            # 1 = outlier
```

22.5 Treatment



Sir's board, Insem 2 notes, page 03 — remove, replace, impute; winsorization and trimming.

From the class board

Treatment → **Remove**; **Replace** (median, mean, percentile, range-random: numeric data; winsorization, trimming); **Imputation** (use specific ML algorithms).

Five treatments of the 95, side by side

Original mean = 22.8. Without the outlier, the other nine have mean 14.78, SD 1.99.

Treatment	What happens to 95	New mean
Remove (delete row)	gone, $n = 9$	14.78
Trimming (10% each tail)	drop 12 and 95, $n = 8$	15.13
Winsorize (cap at next value)	$95 \to 18$	15.10
Capping at 5th/95th pct.	$95 \to 60.35$, $12 \to 12.45$	19.38
Median replacement	$95 \to 15$	14.80

Trimming *removes* the tails; winsorizing *keeps* the rows but pulls the extreme values in to a chosen percentile. Note how weak 5/95 capping is with only ten points: the 95th percentile is itself dragged up by the outlier.

Treatments in one place

- **Remove**: only when it is a confirmed error, or clearly from another population.
- **Trim**: drop the top and bottom k% (`scipy.stats.trim_mean`).
- **Winsorize / cap**: replace values beyond the pth and $(100 - p)$th percentiles with those percentiles (`scipy.stats.mstats.winsorize`, or `Series.clip(lo, hi)`).
- **Replace by median / mean**: treat the outlier like a missing value and impute.
- **Range-random**: replace with a random value drawn from the normal range (keeps some spread instead of piling values on one number).
- **Transform**: a log or Box–Cox transform pulls a long right tail in without deleting anything (next chapter).
- **Robust methods**: keep the data, but use median, IQR, robust scaling or tree models that are not sensitive to extremes.

How to write it in Python

```
import scipy.stats as ss
from scipy.stats.mstats import winsorize
x_removed = x[(x >= lo) & (x <= hi)]                    # remove
    beyond IQR fences
x_trim    = ss.trim_mean(x, 0.10)                        # 10%
    trimmed mean -> 15.125
x_wins    = winsorize(x.to_numpy(), limits=[0, 0.1])     # top 10%
    capped -> 95 becomes 18
x_cap     = x.clip(*x.quantile([.05, .95]))              # cap at 5th
    /95th percentiles
x_med     = x.mask((x < lo) | (x > hi), x.median())      # replace
    outliers by median
```

Questions & Variations

Q. Why is the IQR rule preferred over the z-score rule for skewed or small data?

A. Quartiles are robust: one extreme value barely moves them. The mean and SD are not, so a big outlier inflates σ and lowers its own z-score (masking). And with $n \leq 10$,

$|z|$ cannot exceed 3 at all.

Q. Winsorizing vs trimming?

A. Trimming deletes the extreme rows. Winsorizing keeps the rows but replaces the extreme values with a chosen percentile value.

Q. Can a point be an outlier in 2D but not in either 1D column?

A. Yes — when it breaks the correlation between the columns. Mahalanobis distance, scatter plots and LOF catch it; per-column fences do not.

Studio — now you try

21.1 For $x = [3, 4, 4, 5, 5, 5, 6, 6, 7, 30]$, compute Q_1, Q_3 (numpy linear), the IQR fences, and the z-score of 30 (population SD). Which rule flags 30?
21.2 Winsorize the same data by capping the top value at the next-largest value. Report the new mean and compare it with the original mean and the 10% trimmed mean.
21.3 Explain in two sentences why LOF can flag a point in a sparse region between two dense clusters even when its distance to the overall mean is small.

Chapter 23

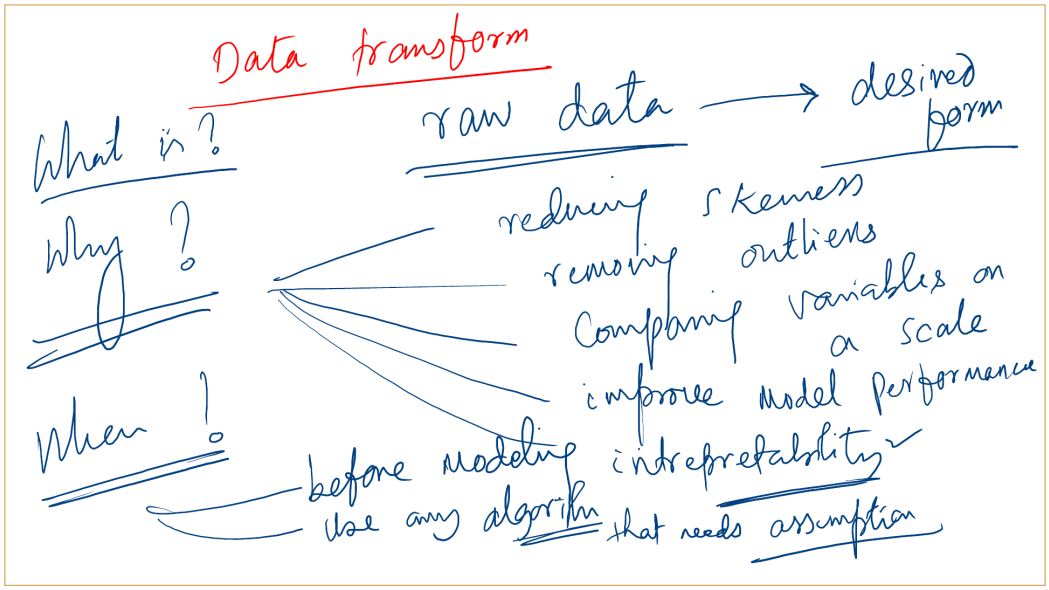
Data Transformation I: Scaling, Function, Power and Quantile Transforms

Right now we are: managing, before modelling — reshaping a numeric column so that its *scale* is comparable with other columns, or its *shape* is closer to what a method assumes.

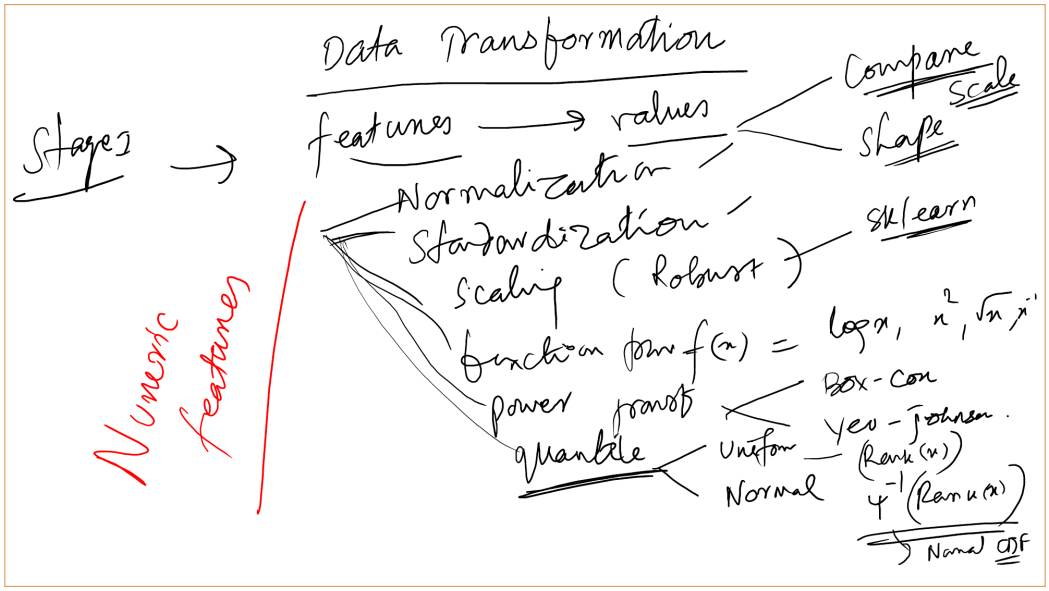
From the class board

Data transform. *What?* raw data → desired form. *Why?* reduce skewness, remove (the effect of) outliers, compare variables on a scale, improve model performance, improve interpretability. *When?* before modelling; whenever an algorithm needs an assumption (normality, equal scale). *How?* types: (1) scaling — z-score, min-max, robust; (2) function, power, quantile.

23.1 The three stages



Sir's board, Insem 2 notes, page 28 — what, why and when of data transformation.



Sir's board, Insem 2 notes, page 31 — Stage I for numeric features: normalization, standardization, robust scaling, function, power, quantile.

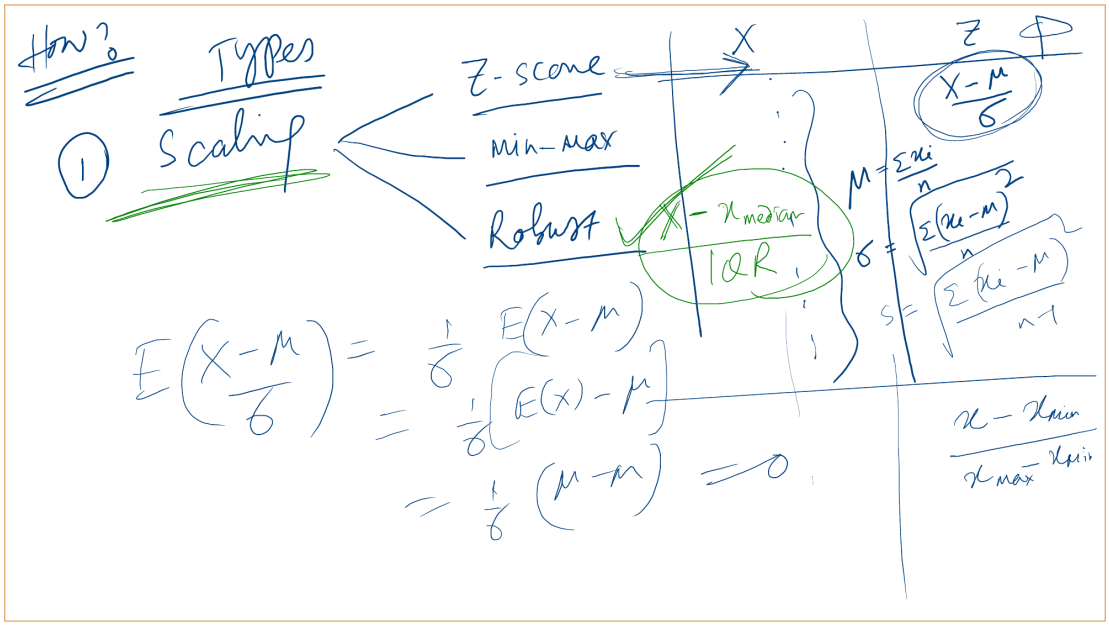
sir organised the whole topic as three stages. This chapter is Stage I; the next chapter does Stages II and III.

Stage	Direction	Methods
I	numeric → numeric	change the **scale** (normalization, standardization, robust scaling) or the **shape** (function $f(x) = \log x,\ x^2,\ \sqrt{x},\ 1/x$; power: Box–Cox, Yeo–Johnson; quantile: uniform, normal)
II	numeric → category	discretization / binning: equal-width, equal-frequency, domain-specific, cluster-based
III	category → numeric	encoding: one-hot, label, ordinal, binary, target, hashing, entity embedding

In Plain Words

Scaling changes the *units* (the ruler). Function, power and quantile transforms change the *shape* (the histogram). A scaled column has exactly the same skewness as before; a logged column does not.

23.2 Scaling



Sir's board, Insem 2 notes, page 29 — z-score, min–max, robust; μ, σ, s; the derivation $E[(X-\mu)/\sigma] = 0$.

One running feature, monthly income in thousands of rupees, with one high earner:

$$x = [20,\ 25,\ 30,\ 35,\ 40,\ 200].$$

$\bar{x} = 58.33$, population SD $\sigma = 63.68$, median $= 32.5$, $Q_1 = 26.25$, $Q_3 = 38.75$, IQR $= 12.5$, min $= 20$, max $= 200$.

23.2.1 Standardization (z-score)

$$z = \frac{x-\mu}{\sigma} \qquad \Rightarrow \qquad \text{mean } 0,\ \text{SD } 1,\ \text{unbounded, can be negative.}$$

Derivation — step by step

Why the mean becomes 0 and the SD becomes 1. Using $E[X] = \mu$ and linearity of expectation (the board's derivation):

$$E\left[\frac{X-\mu}{\sigma}\right] = \frac{1}{\sigma}\big(E[X]-\mu\big) = \frac{1}{\sigma}(\mu-\mu) = 0, \qquad \mathrm{Var}\left[\frac{X-\mu}{\sigma}\right] = \frac{1}{\sigma^2}\mathrm{Var}[X] = \frac{\sigma^2}{\sigma^2} = 1.$$

Subtracting a constant shifts the mean; dividing by a constant divides the SD. Nothing else changes, so the shape (skewness, kurtosis) is untouched.

On the board, $\mu = \frac{\sum x_i}{n}$ and $\sigma = \sqrt{\frac{\sum (x_i-\mu)^2}{n}}$ for a population, $s = \sqrt{\frac{\sum (x_i-\bar{x})^2}{n-1}}$ for a sample. scikit-learn's `StandardScaler` uses the population form (n).

23.2.2 Normalization (min–max)

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \qquad \Rightarrow \qquad \text{bounded in } [0,1],\ \text{shape preserved.}$$

23.2.3 Robust scaling

$$x' = \frac{x - \text{median}(x)}{\text{IQR}} \qquad \Rightarrow \qquad \text{median becomes } 0, \text{ unbounded; for data with many outliers.}$$

(The students' notes wrote $X_{\text{median}}/\text{IQR}$; the numerator is $x - \text{median}$.)

Three scalers on the same feature, compared

x	z-score $\frac{x-58.33}{63.68}$	min–max $\frac{x-20}{180}$	robust $\frac{x-32.5}{12.5}$
20	−0.602	0.000	−1.0
25	−0.523	0.028	−0.6
30	−0.445	0.056	−0.2
35	−0.366	0.083	0.2
40	−0.288	0.111	0.6
200	2.225	1.000	13.4

Read it. Min–max squeezed the five normal earners into $[0, 0.11]$; one outlier used up almost the whole $[0, 1]$ range. z-score left the five bunched (all near -0.4) because the outlier inflated σ. Robust scaling spread the five normal earners evenly over $[-1, 0.6]$ and let the outlier sit far away at 13.4, where you can still see it. With outliers, robust wins.

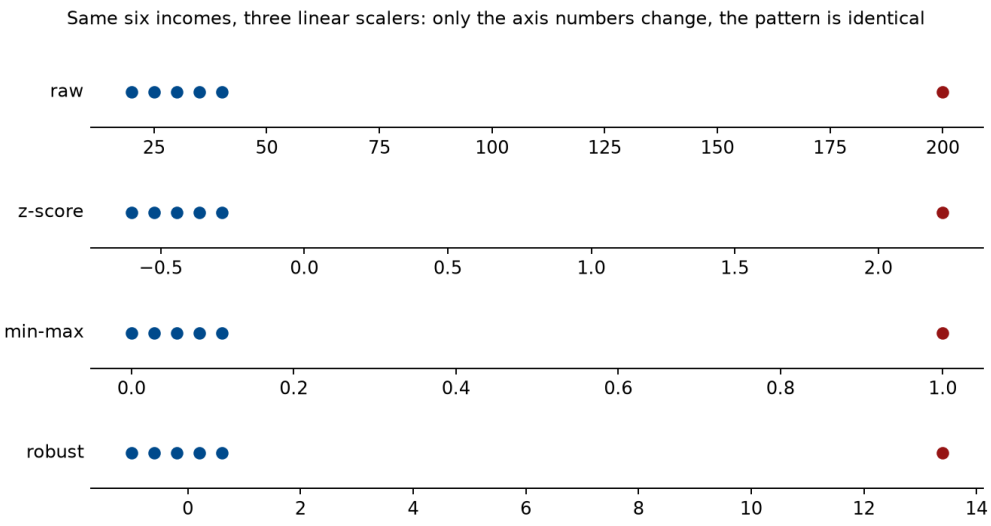


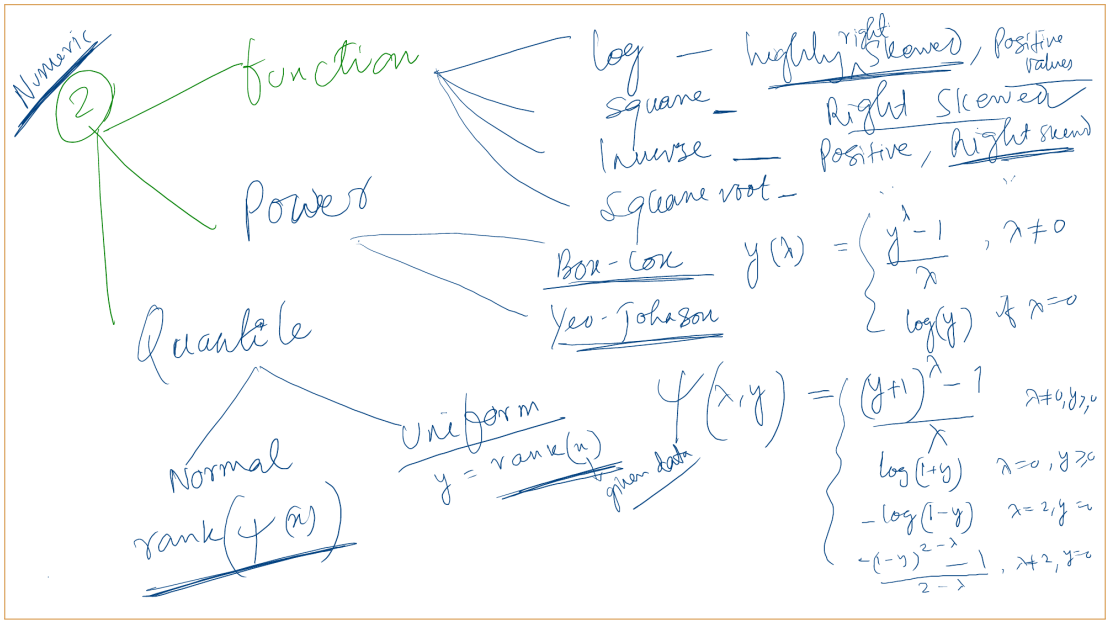
Figure — The six incomes under each scaler. All three are straight-line maps, so the picture is the same; what changes is the scale on the axis (range $[0, 1]$, mean 0, or median 0).

	Standardization (z)	Normalization (min–max)	Robust
Formula	$(x-\mu)/\sigma$	$(x-\min)/(\max-\min)$	$(x-\text{median})/\text{IQR}$
Range	unbounded, negatives	bounded $[0,1]$	unbounded
Centre after	mean 0, SD 1	min 0, max 1	median 0
Outliers	sensitive (mean, SD)	very sensitive (min, max)	resistant (median, IQR)
Use for	PCA, linear/logistic regression, SVM, k-means; roughly normal data	neural nets, image pixels, KNN when a bounded range is needed	data with many outliers
sklearn	`StandardScaler`	`MinMaxScaler`	`RobustScaler`

Watch Out

Scaling does not fix skewness. All three are *linear* maps $x \mapsto ax+b$, so the histogram keeps the same shape — only the axis labels change. To change the shape, use the transforms below. And do not scale tree-based models (decision trees, random forests, boosting): splits do not care about units.

23.3 Function transformation



Sir's board, Insem 2 notes, page 30 — function (log, square, inverse, square root), power (Box–Cox, Yeo–Johnson) and quantile (uniform, normal) transforms.

From the class board

log — highly right-skewed, positive values; square — skewed data; inverse — positive, right-skewed; square root.

Which function for which skew

- $\log x$ — strong **right** skew; needs $x > 0$. Use $\log(1+x)$ (`np.log1p`) when zeros occur. Turns multiplicative effects into additive ones.
- $\sqrt{x}$ — moderate right skew, counts; needs $x \geq 0$. Gentler than log.
- $1/x$ — very strong right skew; needs $x > 0$; *reverses the order* (largest becomes smallest), so interpret with care. Turns "time per task" into "tasks per time".
- x^2 (or e^x) — **left** skew. Squaring stretches large values, so it pushes a right tail further right.

Watch Out

Square is for left skew. The board and the notes list "square — right skewed". Check the numbers below: squaring a right-skewed column *raises* its skewness. Square (and other powers > 1) is the fix for *left*-skewed data; $\sqrt{\ }$, log, $1/x$ (powers < 1) fix right skew.

One right-skewed column, five functions

$y = [1, 2, 2, 3, 4, 6, 9, 15, 30, 60]$ (e.g. days to resolve a complaint). Sample skewness (`scipy.stats.skew`):

	raw y	y^2	$\sqrt{y}$	$1/y$	$\log y$
skewness	1.81	2.41	1.22	1.25	**0.42**

Log brings it closest to symmetric. Square makes it worse. $1/y$ has a similar size of skew to $\sqrt{y}$, but now the long tail is on the side of the *small* original values.

Tukey's ladder of powers

Order the functions by power:

$$\ldots,\ x^{-1},\ x^{-1/2},\ \log x \text{ (power 0)},\ x^{1/2},\ x^{1} \text{ (no change)},\ x^{2},\ x^{3}, \ldots$$

Move *down* the ladder (left) to fix right skew, *up* the ladder (right) to fix left skew. Box–Cox, next, is this ladder with the power chosen automatically.

23.4 Power transformation

23.4.1 Box–Cox (positive data only)

From the class board

$$y(\lambda) = \begin{cases} \dfrac{y^{\lambda} - 1}{\lambda}, & \lambda \neq 0 \\ \log y, & \lambda = 0 \end{cases}$$

The "-1 then $/\lambda$" makes the family continuous: as $\lambda \to 0$, $(y^{\lambda} - 1)/\lambda \to \log y$. $\lambda = 1$ is (a shift of) no change, $\lambda = 0.5$ is square-root-like, $\lambda = 0$ is log, $\lambda = -1$ is inverse-like. λ is chosen by **maximum likelihood**: the value that makes the transformed data look most normal.

Box–Cox by hand and by MLE

By hand for $y = 9$: $\lambda = 0.5 \Rightarrow (9^{0.5} - 1)/0.5 = (3 - 1)/0.5 = 4$; $\lambda = 0 \Rightarrow \log 9 = 2.197$.
By MLE on $y = [1, 2, 2, 3, 4, 6, 9, 15, 30, 60]$: `scipy.stats.boxcox` gives $\hat{\lambda} = \boxed{-0.189}$ and transformed skewness $\boxed{0.08}$ — better than plain log (0.42), because MLE found a power slightly below zero.

23.4.2 Yeo–Johnson (any real data)

Box–Cox fails for zeros and negatives (profit/loss, temperature change). Yeo and Johnson (2000) extended it to the whole real line:

$$\psi(\lambda, y) = \begin{cases} \dfrac{(y+1)^{\lambda} - 1}{\lambda}, & \lambda \neq 0,\ y \geq 0 \\ \log(y+1), & \lambda = 0,\ y \geq 0 \\ -\dfrac{(1-y)^{2-\lambda} - 1}{2-\lambda}, & \lambda \neq 2,\ y < 0 \\ -\log(1-y), & \lambda = 2,\ y < 0 \end{cases}$$

For $y \geq 0$ it is Box–Cox applied to $y + 1$. For $y < 0$ it is the mirror image with power $2 - \lambda$.

Yeo–Johnson

By hand, $y = -3$, $\lambda = 0.5$ (negative branch, $2 - \lambda = 1.5$): $-\dfrac{(1+3)^{1.5} - 1}{1.5} = -\dfrac{8-1}{1.5} = \boxed{-4.667}$.
By MLE on $[-3, -1, 0, 1, 2, 4, 8, 20]$: $\hat{\lambda} = 0.50$, skewness drops from 1.45 to 0.02.
On the positive y above: $\hat{\lambda} = -0.366$, skewness 0.16.

23.5 Quantile transformation

From the class board

Quantile → **Uniform**: $y = \text{rank}(x)$ **Normal**: $y = \Phi^{-1}(\text{rank}(x))$, where Φ is the standard normal CDF.

Replace each value by its *position* in the data (its empirical CDF, i.e. a scaled rank). That gives a uniform $[0, 1]$ column. Push the uniform through Φ^{-1} (the normal quantile function, `norm.ppf`) and you get an exactly bell-shaped column, whatever the original shape.

Quantile transform of five values

$v = [3, 100, 7, 1, 50]$. Ranks $[2, 5, 3, 1, 4]$. scikit-learn maps rank r to $(r - 1)/(n - 1)$:

v	rank	uniform $(r-1)/4$	normal $\Phi^{-1}(\cdot)$
3	2	0.25	-0.674
100	5	1.00	$+5.199$ (clipped)
7	3	0.50	0.000
1	1	0.00	-5.199 (clipped)
50	4	0.75	$+0.674$

$\Phi^{-1}(0) = -\infty$ and $\Phi^{-1}(1) = +\infty$, so the library clips the ends at about ± 5.2. The outlier 100 is now simply "the largest" — its distance from 50 is forgotten.

Watch Out

Quantile transforms are powerful and robust to outliers, but they are **non-linear** and destroy distances: equal gaps in the output do not mean equal gaps in the input. They also need a reasonable sample (sklearn's default uses 1000 quantiles). Use them when the shape matters more than the original units.

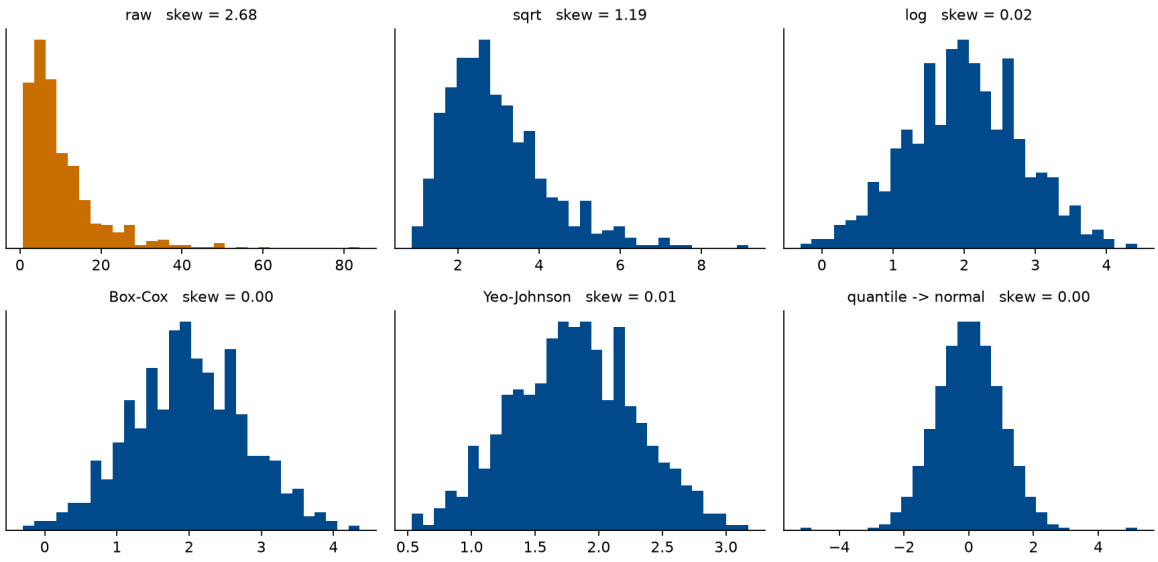


Figure — 500 right-skewed (log-normal) values. Square root helps, log nearly fixes it, Box–Cox and Yeo–Johnson pick the best power by maximum likelihood, and the quantile transform forces an exact bell.

23.6 Choosing a transform

Goal / data	Use	sklearn / numpy
Put features on comparable scales	z-score, or min–max for a bounded range	`StandardScaler`, `MinMaxScaler`
Scale with outliers present	robust scaling	`RobustScaler`
Right skew, positive	log, sqrt, Box–Cox	`np.log1p`, Box–Cox `PowerTransformer`
Skew with zeros / negatives	Yeo–Johnson	`PowerTransformer()` (default)
Left skew	square, cube, exp	`np.square`
Any shape → uniform / normal	quantile	`QuantileTransformer`

How to write it in Python

```python
import numpy as np, scipy.stats as ss
from sklearn.preprocessing import (StandardScaler, MinMaxScaler,
    RobustScaler,
                                   PowerTransformer,
                                       QuantileTransformer)
x = np.array([20, 25, 30, 35, 40, 200]).reshape(-1, 1)   # sklearn
    wants 2-D
for sc in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
    print(type(sc).__name__, sc.fit_transform(x).ravel().round(3))

y = np.array([1, 2, 2, 3, 4, 6, 9, 15, 30, 60])
print(ss.skew(y), ss.skew(np.log(y)))                       # 1.81 ->
    0.42
yb, lam = ss.boxcox(y);      print(lam, ss.skew(yb))       # -0.189,
    0.08
yj, lam = ss.yeojohnson(y);  print(lam)                     # -0.366
PowerTransformer(method="yeo-johnson").fit_transform(y.reshape(-1,
    1))  # also standardizes
QuantileTransformer(n_quantiles=10, output_distribution="normal").
    fit_transform(y.reshape(-1, 1))

# by hand, to match the library:
z  = (x - x.mean()) / x.std()                               # ddof=0,
     like StandardScaler
mm = (x - x.min()) / (x.max() - x.min())
q1, q3 = np.percentile(x, [25, 75]); rb = (x - np.median(x)) / (q3 -
     q1)
```

Watch Out

Fit on train, apply to test. `scaler.fit(X_train)` learns μ, σ (or λ, or the quantiles) from training data only; then `scaler.transform(X_test)` reuses them. Fitting on the full dataset leaks information from the test set.

Questions & Variations

Q. Normalization vs standardization in one line each?
A. Normalization (min–max) maps to a bounded $[0, 1]$ range and keeps the shape. Standardization (z-score) gives mean 0 and SD 1, unbounded, and can be negative.

Q. Why can't Box–Cox be applied to a profit column?
A. Profit can be zero or negative; y^{λ} and $\log y$ are undefined there. Use Yeo–Johnson.

Q. What does $\lambda = 0$ mean in Box–Cox?
A. The log transform.

Q. Does z-scoring a skewed column make it normal?
A. No. It is a linear shift and stretch; skewness is unchanged.

Studio — now you try

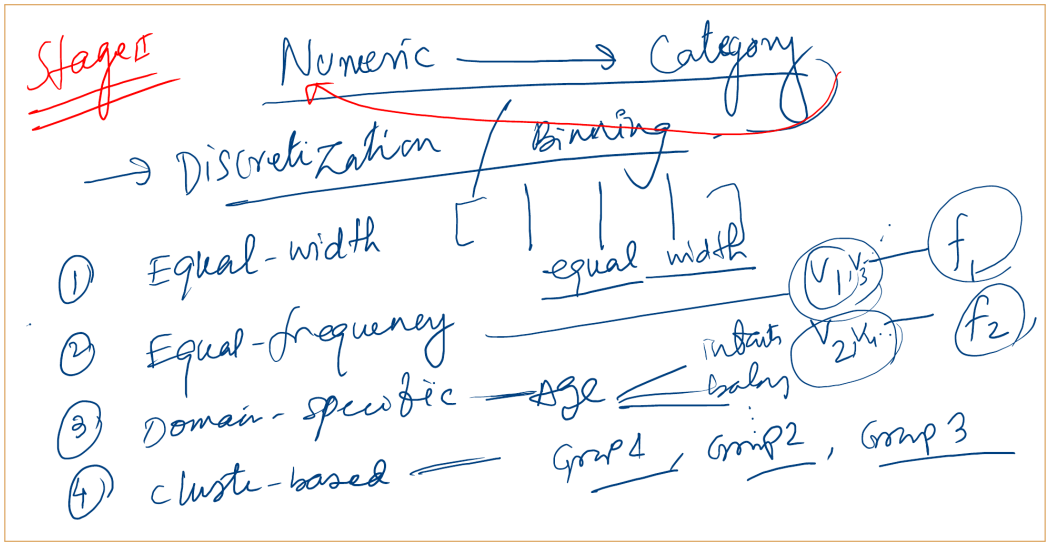
22.1 Scale $x = [2, 4, 6, 8, 100]$ with z-score (population SD), min–max and robust scaling (numpy linear quartiles). Present a table like the worked example and say which scaler keeps the first four values best separated.
22.2 Compute Box–Cox by hand for $y = 4$ at $\lambda = -1,\ 0,\ 0.5,\ 1,\ 2$.
22.3 Apply the uniform quantile transform (sklearn's $(r - 1)/(n - 1)$ convention) to [10, 40, 20, 30], then the normal version. What happens at the two ends, and why?

Chapter 24

Data Transformation II: Binning and Encoding

Right now we are: managing — Stage II turns numbers into categories (binning); Stage III turns categories into numbers (encoding), because almost every model only accepts numbers.

24.1 Stage II: numeric → category (discretization / binning)



Sir's board, Insem 2 notes, page 32 — Stage II: equal-width, equal-frequency, domain-specific, cluster-based binning.

From the class board

Stage II: numeric → category, i.e. **discretization / binning**. (1) equal-width, (2) equal-frequency, (3) domain-specific (age: infant, baby, …), (4) cluster-based (group 1, group 2, group 3).

Why bin a perfectly good number?

- **Interpretability**: "age group 20–59" is easier to report than a slope per year.
- **Non-linear effects**: risk may jump at a threshold rather than rise smoothly.
- **Robustness**: an outlier age of 112 simply lands in the top bin.

- **Required input**: some methods (chi-square tests, crosstabs, naive Bayes on categories) need categories.

The price: information inside a bin is lost (25 and 58 become "adult"), and the edges are a choice that can change conclusions.

One running feature, twelve ages:

$$\text{age} = [2,\ 5,\ 8,\ 13,\ 16,\ 19,\ 25,\ 31,\ 38,\ 45,\ 52,\ 67].$$

24.1.1 Equal-width binning

Split the range into k intervals of the same width $w = (\max - \min)/k$.

Equal-width, $k = 3$

$w = (67 - 2)/3 = 21.67$. Edges $2,\ 23.67,\ 45.33,\ 67$.

$$[2, 23.67] : \{2, 5, 8, 13, 16, 19\}\ (6) \qquad (23.67, 45.33] : \{25, 31, 38, 45\}\ (4)$$
$$(45.33, 67] : \{52, 67\}\ (2).$$

Simple and keeps the distances, but counts are uneven; with an outlier (say age 112) almost everyone falls into the first bin.

24.1.2 Equal-frequency (quantile) binning

Choose edges at quantiles so that every bin has (about) the same number of points.

Equal-frequency, $k = 3$

$12/3 = 4$ points per bin. `pd.qcut` edges: $2,\ 15,\ 33.33,\ 67$.

$$\{2, 5, 8, 13\} \qquad \{16, 19, 25, 31\} \qquad \{38, 45, 52, 67\}.$$

Balanced counts, robust to outliers, but bin widths differ (13 vs 34 years) and identical values can be forced into different bins. (scikit-learn's `KBinsDiscretizer` uses a slightly different quantile rule and reports edges 14.5 and 34.5 — same groups.)

24.1.3 Domain-specific binning

Edges come from knowledge, not from the data.

Life-stage bins

infant $[0, 2)$, child $[2, 13)$, teen $[13, 20)$, adult $[20, 60)$, senior $60+$:

$$\text{child } \{2, 5, 8\} \quad \text{teen } \{13, 16, 19\} \quad \text{adult } \{25, 31, 38, 45, 52\} \quad \text{senior } \{67\}.$$

Other everyday examples: BMI classes (under 18.5, 18.5–25, 25–30, 30+), income tax slabs, letter grades from marks.

24.1.4 Cluster-based binning

Run a 1-D clustering (usually k-means) and let each cluster be a bin. Edges fall at the midpoints between neighbouring cluster centres.

k-means, $k = 3$

Converged clusters: $\{2, 5, 8, 13, 16, 19\}$ (centre 10.5), $\{25, 31, 38, 45\}$ (centre 34.75), $\{52, 67\}$ (centre 59.5). Edges: $(10.5 + 34.75)/2 = 22.625$ and $(34.75 + 59.5)/2 = 47.125$. Bins follow natural gaps in the data.

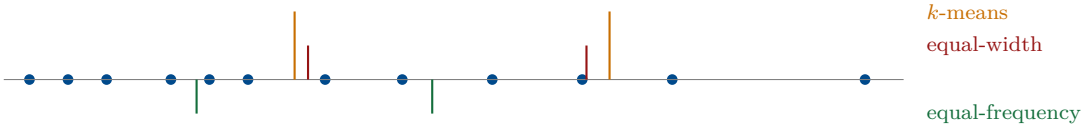


Figure — The twelve ages and where three strategies put their two cut points.

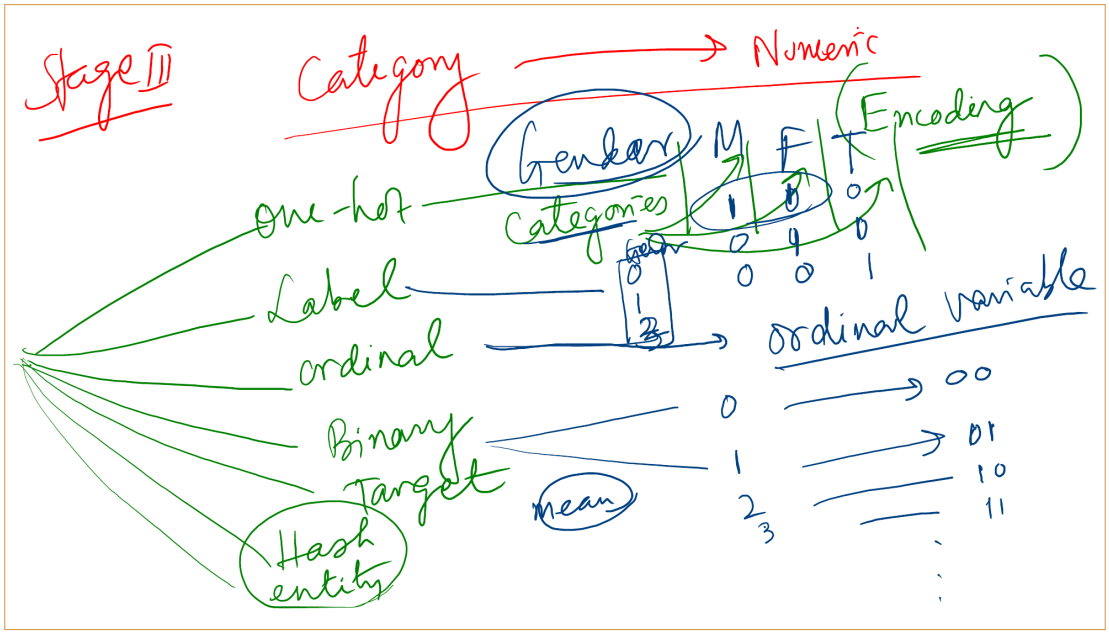
How to write it in Python

```
import pandas as pd, numpy as np
from sklearn.preprocessing import KBinsDiscretizer
age = pd.Series([2,5,8,13,16,19,25,31,38,45,52,67])
pd.cut(age, bins=3)                                          # equal-width
pd.qcut(age, q=3)                                            # equal-
    frequency
pd.cut(age, bins=[0,2,13,20,60,np.inf], right=False,
       labels=["infant","child","teen","adult","senior"])    # domain
           -specific
kb = KBinsDiscretizer(n_bins=3, encode="ordinal", strategy="kmeans")
       # uniform | quantile | kmeans
kb.fit_transform(age.to_frame()); kb.bin_edges_
```

24.2 Stage III: category → numeric (encoding)

From the class board

Stage III: category → numeric (**encoding**). $y_n = (\ \ ) + \alpha_1 M + \alpha_2 F + \alpha_3 T$.
One-hot, label, ordinal, binary, target (mean), hash, entity (embedding).



Sir's board, Insem 2 notes, page 33 — Stage III encodings: one-hot (Gender M/F/T), label, ordinal, binary (0→00, 1→01, …), target (mean), hash, entity.

Why the board wrote a regression. A linear model computes $y = \beta_0 + \alpha_1 x_1 + \ldots$: it multiplies each input by a number. You cannot multiply "Female" by α_2. Encoding turns each category into numbers so it can enter such a formula. The board's example: gender with categories M, F, T becomes three 0/1 columns, and the model is $y = \beta_0 + \alpha_1 M + \alpha_2 F + \alpha_3 T$.

Running example (eight students, from Gujarat):

city	Ahd	Surat	Vad	Surat	Ahd	Rajkot	Surat	Vad
passed	1	0	1	1	0	1	0	1

24.2.1 One-hot encoding

One new 0/1 column per category; exactly one "1" per row.

One-hot for city

city	Ahmedabad	Rajkot	Surat	Vadodara
Ahmedabad	1	0	0	0
Surat	0	0	1	0
Vadodara	0	0	0	1
Rajkot	0	1	0	0

4 categories → 4 columns. No order is implied: every city is equally far from every other.

Watch Out

The dummy variable trap. In the board's model, $M + F + T = 1$ on every row, which is exactly the intercept column. The four columns (intercept, M, F, T) are perfectly collinear, and $\beta_0, \alpha_1, \alpha_2, \alpha_3$ have no unique solution. Fix: drop one category (`drop_first=True`). The dropped one becomes the **reference**: with M dropped, $y = \beta_0 + \alpha_2 F + \alpha_3 T$, where β_0 is the mean for M and α_2 is "F minus M". Tree models and regularised models do not need the drop.

24.2.2 Label encoding

Each category gets an integer. `LabelEncoder` assigns them alphabetically: Ahmedabad 0, Rajkot 1, Surat 2, Vadodara 3. One column, but it *invents an order*: a linear model now believes Vadodara is "three times" Rajkot. Fine for the target y of a classifier and for tree models; wrong for nominal inputs to linear models.

24.2.3 Ordinal encoding

Integers that follow a **real** order, given by you. Use it for ordinal columns (the board: "ordinal variable"): Likert (strongly disagree $= 0, \ldots,$ strongly agree $= 4$), size (S $= 0$, M $= 1$, L $= 2$), education level. The integers respect order, though not necessarily equal spacing.

24.2.4 Binary encoding

Label-encode, then write each integer in base 2, one column per bit. The board: $0 \to 00$, $1 \to 01$, $2 \to 10$, $3 \to 11$.

Binary encoding of four cities

Ahmedabad $0 \to (0,0)$, Rajkot $1 \to (0,1)$, Surat $2 \to (1,0)$, Vadodara $3 \to (1,1)$. k categories need only $\lceil \log_2 k \rceil$ columns: 4 cities $\to$ 2 columns, 1000 categories $\to$ 10 columns (one-hot would need 1000). (The `category_encoders` library starts counting at 1, so it needs $\lceil \log_2(k+1) \rceil$ columns.)

24.2.5 Target (mean) encoding

Replace each category by the **mean of the target** for that category (the board circled "mean").

Target encoding of city by pass rate

Overall pass rate $= 5/8 = 0.625$.

city	n	mean(passed)	smoothed, $m = 2$
Ahmedabad	2	$1/2 = 0.500$	$(2 \cdot 0.5 + 2 \cdot 0.625)/4 = 0.5625$
Rajkot	1	$1/1 = 1.000$	$(1 \cdot 1 + 2 \cdot 0.625)/3 = 0.7500$
Surat	3	$1/3 = 0.333$	$(3 \cdot 0.333 + 2 \cdot 0.625)/5 = 0.4500$
Vadodara	2	$2/2 = 1.000$	$(2 \cdot 1 + 2 \cdot 0.625)/4 = 0.8125$

Smoothing formula: $\text{enc}(c) = \dfrac{n_c\,\bar{y}_c + m\,\bar{y}}{n_c + m}$. Rajkot has one student; without smoothing it would claim a 100% pass rate from a single row. Smoothing pulls rare categories towards the global mean.

Watch Out

Target leakage. Each row's own label is inside its encoding, so a model can "cheat" and look perfect in training. Compute encodings out-of-fold (cross-fitting) on training data only. scikit-learn's `TargetEncoder` does this automatically in `fit_transform`.

24.2.6 Frequency (count) encoding

Replace a category by how often it appears: Surat $3/8 = 0.375$, Ahmedabad 0.25, Vadodara 0.25, Rajkot 0.125. One column, no target needed; ties (Ahmedabad and Vadodara) become indistinguishable.

24.2.7 Hash encoding (feature hashing)

Pass each category through a hash function h and use $h(c)$ mod d as the column index; a second hash bit decides the sign ± 1. You fix the number of columns d in advance, no matter how many categories exist, and new categories never seen in training still get a column.

FeatureHasher with $d = 4$ columns

scikit-learn's `FeatureHasher(n_features=4)` (MurmurHash3) gives

$$\text{Ahmedabad} \to (-1, 0, 0, 0), \quad \text{Vadodara} \to (+1, 0, 0, 0),$$

$$\text{Surat} \to (0, 0, -1, 0), \quad \text{Rajkot} \to (0, 0, +1, 0).$$

Four categories, four columns, and still two **collisions**: Ahmedabad and Vadodara share column 0, Surat and Rajkot share column 2 (the opposite signs keep them apart here, but with more categories they add up and blur). Hashing trades a little accuracy for a fixed, small memory footprint. Used for user IDs, URLs, words: millions of categories (Weinberger et al., 2009).

24.2.8 Entity embedding

Learn a short dense vector for each category inside a neural network (an `Embedding` layer), trained together with the model. Similar categories end up with similar vectors: cities with similar pass rates land close together in, say, a 2-D embedding. Introduced for tabular data by Guo and Berkhahn (2016) in a Kaggle sales-forecasting competition. Needs a neural network and enough data; powerful for high-cardinality columns.

24.3 Choosing an encoder

Encoder	Columns for k cats	Implies order?	Use when
One-hot	k (or $k-1$)	no	nominal, small k (under about 15); linear models
Label	1	yes (fake)	target labels; tree models
Ordinal	1	yes (real)	ordinal columns (Likert, grades, sizes)
Binary	$\lceil \log_2 k \rceil$	partly	nominal, medium-to-large k
Target (mean)	1	by target	high k, supervised task; guard against leakage
Frequency	1	by count	frequency itself is informative
Hashing	d (fixed)	no	huge or streaming k; collisions acceptable
Entity embedding	chosen dim	learned	neural nets, very high k

How to write it in Python

```python
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, OrdinalEncoder, TargetEncoder
from sklearn.feature_extraction import FeatureHasher
d = pd.DataFrame({"city": ["Ahmedabad","Surat","Vadodara","Surat","Ahmedabad","Rajkot","Surat","Vadodara"],
                  "passed": [1,0,1,1,0,1,0,1]})

pd.get_dummies(d["city"], dtype=int)                                # one-hot
pd.get_dummies(d["city"], drop_first=True, dtype=int)               # avoid the dummy trap
OneHotEncoder(handle_unknown="ignore").fit_transform(d[["city"]]).toarray()
LabelEncoder().fit_transform(d["city"])                             # [0 2 3 2 0 1 2 3]
OrdinalEncoder(categories=[["low","medium","high"]])                # you give the order
d.groupby("city")["passed"].mean()                                  # target encoding by hand
d["city"].map(d["city"].value_counts(normalize=True))               # frequency encoding
TargetEncoder(target_type="binary").fit_transform(d[["city"]], d["passed"])  # cross-fitted
FeatureHasher(n_features=4, input_type="string").transform([[c] for c in d["city"]]).toarray()
# Binary encoding: pip install category_encoders
# import category_encoders as ce; ce.BinaryEncoder(cols=["city"]).fit_transform(d)
```

Questions & Variations

Q. Why not label-encode a nominal column for linear regression?
A. The integers create a false order and false spacing, so the model fits one slope across "Ahmedabad < Rajkot < Surat", which means nothing.

Q. What is the dummy variable trap?
A. With an intercept, the full set of one-hot columns sums to the intercept column, causing perfect multicollinearity. Drop one category as the reference.

Q. Equal-width vs equal-frequency in one line each?
A. Equal-width: same interval length, uneven counts. Equal-frequency: same counts, uneven lengths.

Q. When is hashing better than one-hot?
A. When there are very many categories, or new ones keep appearing, and a fixed number of columns matters more than avoiding the occasional collision.

Studio — now you try

23.1 For marks $[35, 42, 48, 55, 61, 67, 72, 78, 85, 93]$ make 3 equal-width bins and 3 equal-frequency bins (use `pd.qcut`). List the members of each bin.
23.2 A `size` column has values S, M, L, XL. Write the ordinal encoding, the one-hot encoding with `drop_first`, and the binary encoding (label order S, M, L, XL $= 0, 1, 2, 3$).
23.3 Using the eight-student table, recompute the smoothed target encoding with $m = 4$. Which city moved most towards the global mean, and why?

Chapter 25

Insem 2 Revision Kit

Right now we are: revising — one chapter that maps the Insem 2 syllabus onto this book, drills the Python functions Category 3 asks for, and ends with an MCQ and short-answer bank.

25.1 What Insem 2 covers

From the class board

Insem 2: data collection & sampling, data import & cleaning, missing data analysis, descriptive summary, univariate & bivariate analysis — and data transform (from 30/09/26).
Exam style (1 hour): Category 1 — MCQ, no negative marking, theory (data types, measurement scales, tools, distributions). Category 2 — short answers. Category 3 — Python code, specifically the *functions*.

Syllabus item	Where in this book
Data types, NOIR scales	the Data and Measurement-scales parts (nominal, ordinal, interval, ratio; which operations are legal)
Data collection, sampling, design	the sampling and experiments chapters (SRS, cluster, stratified, sampling bias, the three Rs)
Data import & cleaning	the import lab (features half: dimension, type, names, units, date-time, duplicates; values half: range, feasibility, missing, outliers)
Missing data analysis	Chapters 19–21: mechanisms, detection, treatment
Outliers	Chapter 22
Descriptive summary, univariate	means, median/mode, dispersion, percentiles, shape (skewness, kurtosis), distributions
Bivariate analysis	association and covariance/correlation chapters (Pearson, Spearman, Kendall, point-biserial, Cramér's V)
Data transformation	Chapters 23–24: scaling, function/power/quantile, binning, encoding

25.2 Category 3: the Python functions, in one place

25.2.1 The nine distributions on the list

Every `scipy.stats` distribution has the same methods: `pmf` / `pdf`, `cdf`, `ppf` (inverse CDF), `rvs` (random samples), `mean`, `var`. numpy's `default_rng()` has a sampler for each as well.

How to write it in Python

```
import numpy as np, scipy.stats as ss
rng = np.random.default_rng(42)
n = 100
# ---------- discrete ----------
rng.binomial(1, 0.3, n)          ; ss.bernoulli.rvs(p=0.3, size=n)
            # Bernoulli(p)
rng.binomial(10, 0.3, n)         ; ss.binom.rvs(n=10, p=0.3, size=n)
           # Binomial(n, p)
rng.poisson(2, n)                ; ss.poisson.rvs(mu=2, size=n)
                # Poisson(lambda)
rng.geometric(0.2, n)            ; ss.geom.rvs(p=0.2, size=n)
                  # Geometric(p)
# ---------- continuous ----------
rng.uniform(5, 10, n)            ; ss.uniform.rvs(loc=5, scale=5,
    size=n)   # Uniform(a=5, b=10): scale = b-a
rng.normal(50, 10, n)            ; ss.norm.rvs(loc=50, scale=10,
    size=n)    # Normal(mu, sigma)
rng.exponential(3, n)            ; ss.expon.rvs(scale=3, size=n)
                # Exponential, mean = scale = 1/lambda
rng.chisquare(4, n)              ; ss.chi2.rvs(df=4, size=n)
                  # Chi-square(k)
rng.standard_cauchy(n)           ; ss.cauchy.rvs(loc=0, scale=1,
    size=n)    # Cauchy(x0, gamma)

ss.binom.pmf(3, 10, 0.3)    # 0.2668   P(X = 3)
ss.norm.cdf(115, 100, 15)   # 0.8413   P(X < 115)
ss.chi2.ppf(0.95, 4)        # 9.4877   critical value
```

The three distributions not covered earlier

Geometric(p) — number of trials until the first success. $P(X = k) = (1-p)^{k-1}p$, $k = 1, 2, \ldots$; mean $1/p$, variance $(1-p)/p^2$. Example: $p = 0.2$, $P(X = 3) = 0.8^2 \times 0.2 = 0.128$; mean 5, variance 20.

Chi-square(k) — sum of squares of k independent standard normals, $\chi^2_k = Z_1^2 + \cdots + Z_k^2$. Mean k, variance $2k$; right-skewed; the reference distribution for Little's MCAR test and the contingency-table test. $\chi^2_{0.05}(4) = 9.49$.

Cauchy(x_0, γ) — $f(x) = \dfrac{1}{\pi\gamma\left[1 + \left(\frac{x-x_0}{\gamma}\right)^2\right]}$. Bell-like but with tails so heavy that the **mean and variance do not exist** (`ss.cauchy.mean()` returns `nan`). Median x_0, quartiles $x_0 \pm \gamma$. The sample mean of Cauchy data never settles down, no matter how large n is — a favourite MCQ.

25.2.2 "Generate 100 random numbers from an exponential distribution between a and b"

An exponential lives on $[0, \infty)$, so "between a and b" needs a shift (to start at a) and a cut (to stop at b): a **truncated exponential**.

How to write it in Python

```python
import numpy as np, scipy.stats as ss
a, b, scale = 2, 10, 3            # scale = mean of the un-truncated exponential = 1/lambda

# Way 1: scipy's truncated exponential. Its 'b' is the cut-off in units of scale.
x = ss.truncexpon(b=(b - a) / scale, loc=a, scale=scale).rvs(size=100, random_state=0)

# Way 2: draw, shift, keep only values <= b (rejection), take the first 100
rng = np.random.default_rng(0)
y = a + rng.exponential(scale, 1000)
y = y[y <= b][:100]

print(x.min(), x.max(), len(x))   # all values in [2, 10], 100 of them
```

Just shifting (`a + rng.exponential(scale, 100)`) gives values $\geq a$ but not $\leq b$; just clipping (`np.clip`) piles fake values at exactly b. Both are wrong answers.

25.2.3 "Convert int64 to string" and other type changes

How to write it in Python

```python
df["roll"] = df["roll"].astype(str)           # int64 -> object (Python str)
df["roll"] = df["roll"].astype("string")      # int64 -> pandas StringDtype
df["roll"].map(str)                           # same idea, element by element
pd.to_numeric(df["marks"], errors="coerce")   # string -> number; junk becomes NaN
df["city"] = df["city"].astype("category")    # nominal column as category dtype
pd.to_datetime(df["date"], format="%d/%m/%Y")
```

25.2.4 Features and values: the checklist functions

How to write it in Python

```python
df = pd.read_csv(PATH, na_values=["NA", "-", "?"])
df.shape              # (rows, columns)          -> dimension
df.columns            # feature names            -> naming convention
```

```
df.dtypes            # type of each column      -> measurement scale check
df.info()            # dtypes + non-null counts + memory
df.head(); df.tail(); df.sample(5, random_state=0)
df.rename(columns=str.lower)                    # clean names
df.describe()                                   # count, mean, std, min, quartiles, max
df.describe(include="object")                   # count, unique, top, freq for text columns
df["city"].value_counts()                       # count the values
df["city"].unique(); df["city"].nunique()       # distinct values / how many
df.duplicated().sum(); df.drop_duplicates()     # duplication
df["marks"].between(0, 100).all()               # feasible range
df.isnull().sum()                               # missing values
```

25.2.5 Sampling, univariate, bivariate in a few lines

How to write it in Python

```
# sampling
df.sample(n=10, random_state=1)                                  # simple random sample
df.groupby("city", group_keys=False).sample(frac=0.2, random_state=1)  # stratified
from sklearn.model_selection import train_test_split
train, test = train_test_split(df, test_size=0.2, stratify=df["passed"], random_state=1)

# univariate
s = df["marks"]
s.mean(), s.median(), s.mode()[0], s.var(), s.std()      # pandas var/std use ddof=1
s.quantile([.25, .5, .75]), s.skew(), s.kurt()           # kurt() = excess kurtosis

# bivariate
df["hours"].corr(df["marks"])                            # Pearson
df["hours"].corr(df["marks"], method="spearman")         # or "kendall"
ss.pointbiserialr(df["passed"], df["marks"])             # binary vs numeric
ct = pd.crosstab(df["gender"], df["passed"])
chi2, p, dof, exp = ss.chi2_contingency(ct)              # two categoricals
ss.contingency.association(ct, method="cramer")          # Cramer's V
```

25.3 Category 1: MCQ bank

1. The arithmetic mean is meaningful for which lowest scale? (a) nominal (b) ordinal (c) interval (d) none

2. Missingness that depends only on observed variables is (a) MCAR (b) MAR (c) MNAR (d) ignorable never

3. Who introduced the MCAR/MAR/MNAR classification? (a) Tukey 1977 (b) Little 1988 (c) Rubin 1976 (d) Box & Cox 1964

4. Little's MCAR test statistic follows approximately a (a) t (b) F (c) χ^2 (d) normal distribution

5. $p = 0.03$, $\alpha = 0.05$ in Little's test. Decision: (a) fail to reject, data MCAR (b) reject H_0, data not MCAR (c) accept H_1 as proven (d) increase α

6. `msno.heatmap` shows (a) correlation of values (b) correlation of missingness indicators (c) the missing values themselves (d) Little's p-value

7. Listwise deletion is also called (a) available case (b) pairwise deletion (c) complete case (d) imputation

8. Mean imputation tends to (a) increase variance (b) decrease variance (c) leave variance unchanged (d) remove bias under MNAR

9. Tukey's upper fence is (a) $Q_3 + 1.5\,\mathrm{IQR}$ (b) $Q_3 + 3\sigma$ (c) $\bar{x} + 1.5\,\mathrm{IQR}$ (d) $Q_2 + 1.5\,\mathrm{IQR}$

10. With $n = 10$ and population SD, the largest possible $|z|$ is (a) 2 (b) 3 (c) 10 (d) unbounded

11. Winsorization (a) deletes extreme rows (b) replaces extremes with percentile values (c) log-transforms the tail (d) standardizes the column

12. Which is a multivariate outlier method? (a) IQR fences (b) z-score (c) LOF (d) histogram

13. Min–max scaling maps data to (a) mean 0, SD 1 (b) $[0, 1]$ (c) median 0 (d) a normal shape

14. Robust scaler uses (a) mean and SD (b) min and max (c) median and IQR (d) mode and range

15. Box–Cox with $\lambda = 0$ is (a) square root (b) log (c) inverse (d) no change

16. Which power transform accepts negative values? (a) Box–Cox (b) Yeo–Johnson (c) log (d) square root

17. To reduce *left* skew, use (a) $\log x$ (b) $\sqrt{x}$ (c) x^2 (d) $1/x$

18. Equal-frequency binning gives bins with equal (a) width (b) counts (c) means (d) variances

19. The dummy variable trap is caused by (a) too few categories (b) perfect multicollinearity with the intercept (c) missing values (d) target leakage

20. Which encoding can leak the target? (a) one-hot (b) binary (c) target (mean) (d) hashing

21. 8 categories in binary encoding need (labels 0–7) (a) 8 (b) 4 (c) 3 (d) 1 columns

22. The Cauchy distribution has (a) mean 0, variance 1 (b) no finite mean or variance (c) mean equal to its scale (d) a light tail

Answers. 1 c, 2 b, 3 c, 4 c, 5 b, 6 b, 7 c, 8 b, 9 a, 10 b, 11 b, 12 c, 13 b, 14 c, 15 b, 16 b, 17 c, 18 b, 19 b, 20 c, 21 c, 22 b.

25.4 Category 2: short-answer bank

Q. Define MCAR, MAR and MNAR with one example each, and the probability statement for each.
A. MCAR: $P(R \mid Y_{\text{obs}}, Y_{\text{mis}}, \psi) = P(R \mid \psi)$ — pages lost at random. MAR: $= P(R \mid Y_{\text{obs}}, \psi)$ — younger people skip income, age recorded. MNAR: depends on Y_{mis} — high earners hide income.

Q. Distinguish complete case and available case analysis.
A. Complete case drops any row with a missing value and uses one common set of rows. Available case keeps all rows and uses, for each statistic, the rows that have the needed values; each statistic has its own n.

Q. List the steps of a hypothesis test for MCAR.
A. State H_0: MCAR and H_1: not MCAR, choose α; choose the test statistic (χ^2 for Little); compute the observed statistic and p-value from the sample; reject H_0 if $p < \alpha$, otherwise fail to reject.

Q. Why does the IQR rule resist masking while the z-score rule does not?
A. Quartiles barely move when an extreme value is added; the mean and SD move a lot, inflating σ and shrinking the outlier's own z-score.

Q. Compare normalization, standardization and robust scaling.
A. See the comparison table in Chapter 23: bounded $[0, 1]$ vs mean 0 / SD 1 vs median 0 / IQR 1; outlier sensitivity high, medium, low.

Q. When would you use Yeo–Johnson instead of Box–Cox?
A. When the data contain zeros or negative values; Box–Cox needs strictly positive data.

Q. Explain one-hot encoding and the dummy variable trap.
A. One 0/1 column per category. With an intercept, the full set sums to 1 on every row, so drop one category as the reference.

Q. What are the four types of binning? Give one example of domain-specific binning.
A. Equal-width, equal-frequency, domain-specific, cluster-based. Domain: age → child / teen / adult / senior; BMI classes.

Q. What is the role of `missingno`, and why is it called non-confirmatory?
A. It visualises missingness (bar, matrix, heatmap, dendrogram) to spot patterns. It cannot give a probability statement or detect MNAR, so a formal test is needed to confirm.

25.5 Last-night summary

In Plain Words

Missing: count it, picture it, test it (Little), name it (MCAR/MAR/MNAR), then match the fix (delete only under MCAR; impute with the explaining columns under MAR; model or flag under MNAR). **Outliers**: IQR and modified z beat plain z on small or skewed data; check feasibility before deleting; trim, winsorize, transform, or use robust methods. **Transform**: scaling changes units, not shape; log/Box–Cox/Yeo–Johnson change shape; quantile forces a shape. Fit on train only. **Encode**: one-hot for nominal, ordinal for ordered, target/hash/embedding for huge cardinality; drop one

dummy with an intercept.

Appendix A

Optional 2026 Toolkit (Not the Course)

Right now we are: optional — same checklist, heavier files, louder agents

IT462 is pandas, a pencil, and the two tables. This chapter is a sidecar. Skip it for the exam. Read it if a 200 MB CSV makes `pd.read_csv` swap, or if an agent offers to "do the EDA" in one click.

A.1 Polars and DuckDB — faster rectangles, same NOIR

In Plain Words

Polars is a DataFrame library (Rust, lazy, parallel). **DuckDB** is an in-process SQL engine that reads CSV / Parquet without a warehouse. Neither invents a sixth measurement scale. A city column is still nominal; a mean of `student_id` is still a crime.

How to write it in Python

```python
import polars as pl
import duckdb
lf = pl.scan_csv("it462_marks.csv")
print(lf.collect_schema())
print(lf.select(pl.col("marks").mean()).collect())    # 63.5

print(duckdb.sql("""
    SELECT gender, median(marks) AS med, count(*) AS n
    FROM 'it462_marks.csv'
    GROUP BY 1
    ORDER BY n DESC
""").df())
```

Watch Out

`scan_csv` is lazy: nothing runs until `collect`. That is a feature. It is also how beginners print nothing and declare the file empty. Start with `.head(5).collect()` on a new file, the same way you start pandas with `.head()`.

CSV is a letter you send. **Parquet** (columnar, typed) is the filing cabinet. Pandera / Great Expectations turn the L10 feasible-range checks into a failing CI job — IDA as code, still the

board's "usable: valid, reliable, authentic, error-hunted".

A.2 ydata-profiling — a portrait, not a substitute

One call builds an HTML of every column: types, histograms, missingness, warnings. Use it as a *start*, the way `df.describe()` is a start.

How to write it in Python

```
from ydata_profiling import ProfileReport
df = __import__("pandas").read_csv("it462_marks.csv")
ProfileReport(df, title="IT462 marks").to_file("profile.html")
```

A profile will not tell you that Pearson misses a boomerang, that Likert is ordinal, or that the four fails are real. It will tell you $n = 30$ and that marks look roughly symmetric — which you already know from Chapter 0's histogram.

A.3 LLM-EDA, with an audit trail or not at all

Papers of 2023–2026 (LIDA; DA-Studio; DataClawBench; Walk Before You Run) all rhyme: an agent that writes pandas/Polars can speed the walk; an agent that returns a paragraph with no code is astrology. Honest use on `it462_marks.csv`:

1. You name the question and the target (`passed`, or `marks`).
2. The agent emits *runnable* code, not a vibe.
3. You run it against the asserts in Chapter 0: shape $(30, 8)$, marks sum 1905, mean 63.5.
4. You keep the script. A chat log is not provenance.

In Plain Words

If the agent cannot show the code, it is not EDA. Tukey's warning, in 2026 clothes: exploration is an evaluable stage, not a mood.

How to write it in Python

```
# audit stub: whatever the agent wrote must pass this
import pandas as pd
out = pd.read_csv("it462_marks.csv")
assert out.shape == (30, 8)
assert out["marks"].sum() == 1905
assert abs(out["marks"].mean() - 63.5) < 1e-9
# then *you* read the emitted groupby / corr / plot
```

That is the whole sidecar. Back to the thirty rows.

Appendix B

Where Every Page of the Notes Lives in This Book

This appendix is a checklist. Every page of *sir*'s Insem 2 board notes and every page of the students' handwritten EDA notes is listed with the chapter that teaches it. If a line of the notes is not on this map, it is not in the book — tell the author. Every one of *sir*'s 33 board pages also appears in the book as a framed screenshot (orange border, caption "Sir's board, Insem 2 notes, page n"), placed beside the section that explains it.

B.1 *sir*'s board notes (Insem 2, 33 pages)

Page	Board content	In the book
1	Outliers (univariate): definition, detection numerical (z-score, IQR) / graphical (histogram, pdf/cdf, boxplot), treatment	Ch. 22, § Definition, numerical rules, graphical
2	Multivariate: extend IQR to correlation matrix; heatmap, scatter, pair plots; PCA, LOF, PyOD, AOD	Ch. 22, § Multivariate outliers
3	Treatment: remove; replace (range-random, median, mean, percentile; winsorization, trimming); imputation via ML	Ch. 22, § Treatment
4	Missing Data Mechanism: definition (absence / inconsistent forms), causes (may not exist / error), Rubin 1976	Ch. 19, § Definition and causes
5	Classification: MCAR, MAR, NMAR/MNAR	Ch. 19, § The three mechanisms
6	MCAR / MAR tables (Age, Gender, Score; Y_m independent of Y_o; Y_m depends on Y_o)	Ch. 19, § MCAR, § MAR
7	Types: MCAR / MAR / MNAR one-line definitions	Ch. 19, board box and table
8–9	Scatter pictures of MCAR, MAR, MNAR holes	Ch. 19, three-panel figure
10	Detection: numerical (describe, isnull, isna); visual (histogram, scatter, missingno)	Ch. 20, § Numerical, § Visual
11	missingno package: bar, matrix, heatmap, dendrogram	Ch. 20, § The missingno package
12	R indicator, $Y = (Y_{\text{obs}}, Y_{\text{mis}})$, ψ parameters	Ch. 19, § The notation on the board
13–14	$P(R = 0 \mid Y_{\text{obs}}, Y_{\text{mis}}, \psi)$ for MCAR, MAR, MNAR	Ch. 19, formulas in each mechanism
15	Nullity-correlation matrices labelled MCAR / MAR / MNAR	Ch. 20, § Reading the board's three heatmaps

Page	Board content	In the book
16–18	Dendrogram sketches; cluster sketch	Ch. 20, missingno table (dendrogram row)
19	msno is a non-confirmatory signal; statistical tests are confirmatory	Ch. 20, Watch Out box
20	Little's (overall), Dixon's (univariate), Hawkins' (multivariate) MCAR tests	Ch. 20, § Little, § Dixon, § Hawkins
21	Hypothesis tests: H_0, H_1, α, one/two-tailed, χ^2 statistic, p	Ch. 20, § Primer on hypothesis tests
22	Missing value treatment summary (missingno + confirmatory tests)	Ch. 20, detection workflow
23	χ^2_0 vs χ^2_α, rejection region, univariate histogram / Q-Q	Ch. 20, χ^2 figure; Q-Q in Ch. 13
24	$p = 0.26$ fail to reject; $p = 0.01$ reject, not MCAR	Ch. 20, worked "The board's two decisions"
25	MCAR treatment: % missing via isnull/isna/info; 5–10% → complete case	Ch. 21, § Deletion, Watch Out
26	Complete case (listwise) vs available case (pairwise) grid	Ch. 21, § Deletion, worked six-row table
27	Imputation: mean, median; function → regression $V_1 = c_0 + c_1V_2 + c_3V_3 + \varepsilon$	Ch. 21, § Simple imputation, § Regression
28	Data transform: what, why, when	Ch. 23, opening board box
29	Scaling: z-score, min-max, robust; $E[(X - \mu)/\sigma] = 0$; σ vs s	Ch. 23, § Scaling, derivation box
30	Function (log, square, inverse, sqrt); power (Box–Cox, Yeo–Johnson); quantile (uniform, normal)	Ch. 23, § Function, § Power, § Quantile
31	Stage I numeric features: normalization, standardization, robust, function, power, quantile	Ch. 23, § The three stages
32	Stage II numeric → category: equal-width, equal-frequency, domain-specific, cluster-based	Ch. 24, § Stage II
33	Stage III category → numeric: one-hot, label, ordinal, binary, target (mean), hash, entity	Ch. 24, § Stage III

B.2 Students' handwritten notes (16 pages)

Page	Notes content	In the book
1	Randomization, replication, control/block; collect → IDA (reliable?) → EDA (patterns?); Part 1 data: scales, tools, design; Part 2: workspace, pandas, import online/local, CSV steps, PATH, metadata	Ch. 17 (three Rs, two questions); Ch. 18, § Where this sits in the course
2	Data, statistical tools, measurement scales; experiment (vaccination); sample → statistic, population → parameter, $N \gg n$; ${}^{N}C_{n}$; SRS ($1/{}^{N}C_{n}$), cluster, stratified; sampling bias; problem definition → experimental unit, factors/levels, response	Ch. 16 (sampling), Ch. 17 (design of experiments)

Page	Notes content	In the book
3	df.info, dtype, object (category), shape, columns, astype; feature checklist (dimension, type, naming, units, date-time, duplication, context, feature engineering; target clear); values (range, unitless, missing, outlier); exam style Cat 1	Ch. 18 (board box, both halves, column functions); Ch. 25 (exam style)
4	Mean appropriate for interval; Cat 2, Cat 3; distributions list (Bernoulli … Cauchy); 100 exponential numbers from a to b; int64 → string; value counts, unique; imported data → nominal, ordinal, binomial, numeric (interval, ratio)	Ch. 25; Ch. 13; Ch. 6, § The fourth label: binomial features
5	Skewness: mean = median = mode symmetric; positive / negative skew; normal skew 0; converse not true; `df.skew()`; kurtosis	Ch. 12, incl. worked "A lopsided column with skewness exactly 0"
6	Measures of association: linear / non-linear; Pearson (df.corr, interval/ratio); Spearman (ordinal); point-biserial; Cramér ϕ/V; scipy functions; crosstab; contingency table in statsmodels	Ch. 14 (incl. statsmodels codenote and chooser box); Ch. 15
7	Outliers: numeric scale only; definition; minority ($< 5\%$); IQR fences; 3σ covers 99.7%; 1.4 vs 1.5 IQR; z-score ± 3	Ch. 22, Definition box, IQR and z sections, "Why 1.5?"
8	Graphical: histogram, boxplot; univariate outliers summary; multivariate: correlation matrix, heatmap, scatter, pair plots, LOF, PCA	Ch. 22
9	PyOD, AOD; treatment tree; MDM definition, causes, Rubin; classification; MCAR: Y_m, Y_o independent	Ch. 22; Ch. 19
10	MAR: Y_m depends on Y_o; types; detection numerical / visual; missingno	Ch. 19; Ch. 20
11	H_0/H_1; Little, Hawkins, Dixon; χ^2; p vs α decisions	Ch. 20
12	missingno plots; R; $Y = (Y_{\text{obs}}, Y_{\text{mis}})$; ψ; MCAR / MAR / MNAR formulas	Ch. 19; Ch. 20
13	Treatment: isnull/isna/info %; complete vs available case; mean, median, regression imputation; workflow import → check → classify → treat (deletion, Simple/Iterative imputer, ML)	Ch. 21, incl. the workflow figure
14	Insem 2 syllabus; data transform what/why/when/how; scaling; normalization vs standardization; robust	Ch. 25; Ch. 23
15	Function, power (Box–Cox, Yeo–Johnson), quantile; stages	Ch. 23
16	Quantile uniform / normal; "find 3 different scalers and compare"; Stage II binning; Stage III encoding, $y = (\ ) + \alpha_1 M + \alpha_2 F + \alpha_3 T$	Ch. 23 (worked "Three scalers …compared"); Ch. 24

B.3 Corrections made while transcribing

Where a note disagrees with the standard definition, the book follows the standard and flags it in a Watch Out box: squaring is for *left* skew; Little's χ^2 test rejects in one tail; the robust scaler subtracts the median ($x - \text{median}$); Spearman needs an order, so nominal columns go to

Cramér's V; the Yeo–Johnson negative branch is $-[(1-y)^{2-\lambda}-1]/(2-\lambda)$.

Appendix C

Answers to studios

Right now we are: check — every gold box, with the arithmetic shown

These are full solutions, not hints. If you have not yet tried the studio, close this chapter. Arithmetic uses the running tables and the board toys only.

C.1 Chapter 0 — the two tables

0.1

Sorted marks in three blocks of ten:

$$28+31+35+39+41+42+45+47+50+53=411,$$
$$55+58+59+61+62+66+68+70+73+74=646,$$
$$74+76+77+81+84+85+88+90+94+99=848.$$

Running the first block: $28+31=59$, $+35=94$, $+39=133$, $+41=174$, $+42=216$, $+45=261$, $+47=308$, $+50=358$, $+53=411$. Second: $55+58=113$, $+59=172$, $+61=233$, $+62=295$, $+66=361$, $+68=429$, $+70=499$, $+73=572$, $+74=646$. Third: $74+76=150$, $+77=227$, $+81=308$, $+84=392$, $+85=477$, $+88=565$, $+90=655$, $+94=749$, $+99=848$.

$$411+646=1057, \qquad 1057+848=1905.$$

Then $\bar{x}=1905/30=63.5$. The file is the file.

0.2

First check: `assert df["marks"].sum() == 1905` (and `df.shape == (30, 8)`). A mean of 71 on thirty rows would require a sum of $71\times 30=2130$, which is not 1905. "The mean looks nicer" is a reason to distrust the file, not to keep it. You are matching a running table, not choosing a prettier compression.

0.3

Question "did the student pass?": target = `passed`. Features = `student_id`, `marks`, `age`, `gender`, `likert`, `city`, `hours_studied`. If the question were the mark itself, `marks` would be the target and `passed` would be a redundant encoding of `marks` ≥ 40, not an independent feature.

C.2 Chapter 1 — data vs information

1.1

Drop 99: $17 + 41 + 30 = 88$, so

$$\bar{x}_3 = \frac{88}{3}.$$

Four-number mean $187/4 = 46.75$. The pull is

$$\frac{187}{4} - \frac{88}{3} = \frac{561 - 352}{12} = \frac{209}{12} = 17.41\overline{6}.$$

The 99 lifted the mean by $209/12$ points. Do not use `it462_marks.csv` here.

1.2

Sorted toy: $17, 30, 41, 99$. Mean 46.75, median 35.5, range 82. (a) Parent, "typical": send the median 35.5. The mean hides that three of four scores sit at or below 41. (b) Dean, "anyone far from the pack": send the four numbers, or at least the range $99 - 17 = 82$ and the 99. A single mean hides the outlier on purpose. (c) Ranking formula allowed one number: send the mean 46.75 (that is what "one number" asked for). It hides the gap between 17 and 99.

1.3

`assert df.shape[0] == 30` and `assert df["marks"].mean() == 63.5`. On a list, $(17 + 41 + 30 + 99)/4 = 46.75$. Those two means must never be reported as one class: one is thirty IT462 marks, the other is four board scores from Lecture 1. Mixing piles is a wrong answer even when both divisions are correct.

C.3 Chapter 2 — sources and files

2.1

Header `student_id,marks,age,gender,likert,city,hours_studied,passed`: seven commas, eight fields. The first data line `S01,74,19,F,4,Ahmedabad,6,yes` still has eight fields because Ahmedabad contains no comma; length of the word is irrelevant. `Old, Rajkot` with no quotes becomes two fields, the row has nine, and every column to the right of `city` shifts — a format break, not a statistics bug.

2.2

(a) Primary. You have the bruises (smudged 17 vs 71). (b) Secondary. The office has the bruises; you inherit them. (c) Tertiary. Wikipedia (or whoever compressed the pile) has the bruises; you have one cell. You may quote 63.5; you may not rebuild a histogram from it.

2.3

`pd.read_csv` and `pd.read_json` on the two copies; then `csv_df.iloc[0].equals(json_df.iloc[0])` with S01 marks 74. Two edits that break the assert without touching that 74: rename a column (e.g. `city`→`City`, so the

column-list assert dies); change `gender` from `F` to `f` (first-row equals dies). Changing `S01` to `s01` also works. None of those is a different exam score.

C.4 Chapter 3 — IDA on the dirty five

3.1

One bruise and one action per row; all three verbs used. Row 1: trailing period on `2026.` — **recode** the ID to the string `2026` (keep it a string). Row 2: age 10 in an IT462 table — **keep-and-flag** as implausible for this course until a codebook speaks. Row 3: ID digits 2026 again — **keep-and-flag** as a duplicate-key clash; do not silently collapse rows 1–3. Row 4: IT462 = −20 and a blank ID — **drop** the row from any marks summary (it is not a keyed student-mark); alternatively recode −20 to missing *and* flag, but a drop must appear somewhere: this is the drop. Row 5: missing age and gender — **keep-and-flag** the holes; do not fill with 0 or with the mode. No finished clean table: the point is the decisions, not a halo.

3.2

Five different cells. **Structure:** ID `2026.` — the column is not a pure integer. **Granularity:** ID 2026 on row 3 (clash with row 2): is one row one student or one attempt? **Scope:** blank ID on row 4 — a present row with an absent key; also a reminder that students never typed are absent rows. **Temporality:** ID 2023 on row 5 sitting next to three 2026s. **Faithfulness:** IT462 = −20 (not a mark unless the codebook says "absent").

3.3

Codebook A (keep): "Age is years since live birth. 0 means a newborn. Keep row 1. Do not treat 0 as missing. Gender M is complete." Codebook B (recode): "Age 0 is the missing-code of the old form. Recode to NaN. Flag the row. Do not impute 20." Both can be professional; the five numbers alone license neither. "Always drop age 0" is not a rule of IDA because 0 is a legal ratio value (a live birth) in one codebook and a sentinel in another. Dropping without writing the codebook changes n and pretends the file always had four rows.

C.5 Chapter 4 — the pipeline

4.1

IDA → EDA → IDA → EDA: (i) opening the CSV is IDA (collection / first hunt); (ii) plotting the Age histogram is EDA (shape); (iii) deciding 0 is a missing-code is IDA (codebook, back around the feedback arrow); (iv) replotting Age without that row is EDA on the rectangle you were then allowed to use.

4.2

Target: `passed`. Features: the other seven columns. Do *not* feed `marks` to a model of pass/fail: `passed` is defined by `marks` ≥ 40, so marks are the answer key, not a feature. That is leakage, not EDA.

4.3

Categorical: `student_id`, `gender`, `likert`, `city`, `passed`. Numerical: `marks`, `age`, `hours_studied`. Storage disagreement: `likert` arrives as `int64` but is an ordered label. A careless `df["likert"].mean()` invents $3.4\overline{6}$ as if the gap $2 \to 3$ equalled $4 \to 5$. (A zip-coded `int64` would be the same crime in another file; here Likert is the one that actually sits in the CSV.)

C.6 Chapter 5 — nominal and ordinal

5.1

Blood type: nominal (tags, no order). Rank in class: ordinal (1st beats 2nd; the gap is not marks). City: nominal (Ahmedabad is not less than Surat). Movie star rating: ordinal (more stars is more; gaps unproven). Jersey number: nominal (14 is not twice 7 as a player). `hours_studied`: left this chapter — a ratio quantity, not a label.

5.2

S01–S05 Likert: $4, 3, 5, 3, 2$. Sorted $2, 3, 3, 4, 5$. Mode 3 (twice). Median $x_{(3)} = 3$ because $n = 5$ is odd. Bar counts on the five labels: 1:0, 2:1, 3:2, 4:1, 5:1. A histogram of those integers is the wrong picture: it pretends the axis is a quantity with equal gaps. The median is legal because order is; equal spacing is not.

5.3

Zip 382007 is nominal (a region name written in digits). Hours 6 is ratio. Five zips $382007 + 380001 + 395003 + 390001 + 360001 = 1907013$, so

$$\frac{1907013}{5} = 381402.6.$$

`.mean()` invents a town nobody lives in — a fake "typical pin code" halfway to a place that is not a place.

C.7 Chapter 6 — interval, ratio, operations

6.1

Four properties: labelled; order; equal difference; true zero. `age`: ✓ ✓ ✓ ✓ (ratio). Celsius: ✓ ✓ ✓ × (interval). They first disagree on absolute zero. That disagreement forbids $\times$ and $\div$ — the "twice as hot" sentence — on Celsius, while it remains legal on age (20 years is twice 10 years).

6.2

Celsius ratio $40/20 = 2$ (arithmetic, not physics). Kelvin: $20°\text{C} \mapsto 293\,\text{K}$, $40°\text{C} \mapsto 313\,\text{K}$, ratio $313/293 \approx 1.068$. Neither 2 nor 1.068 is a licence to say "twice as hot" on Celsius; only Kelvin has a true zero, and on Kelvin the rooms are about seven percent apart, not double. Differences: $40 - 20 = 20$ Celsius degrees and $313 - 293 = 20$ kelvin — the same

gap, as they must be (shared degree size). Gap sentences are legal on both rulers.

6.3

`student_id`: stop at nominal. Illegal verb: + (also <, ×). `likert`: stop at ordinal. Illegal verb: × (and +, unless you have argued the interval convention — even then ratios stay illegal). Year CE: stop at interval. Illegal verb: × (2026/1013 is not "twice as late"). `marks`: reach ratio. Every verb on the operations table is legal; the remaining crime is faithfulness (averaging a −20 the codebook has not called a mark), not a missing tick.

C.8 Chapter 7 — means

7.1

$\{2, 4, 8\}$, $n = 3$:

$$\text{AM} = \frac{14}{3}, \qquad \text{GM} = (64)^{1/3} = 4, \qquad \text{HM} = \frac{3}{\frac{1}{2}+\frac{1}{4}+\frac{1}{8}} = \frac{3}{7/8} = \frac{24}{7}.$$

$$\frac{14}{3} \approx 4.667 \;\geq\; 4 \;\geq\; \frac{24}{7} \approx 3.429.$$

Replace 8 by 0: AM still exists, $(2+4+0)/3 = 2$. GM is not defined on the strictly-positive formula (a zero kills the log; the product formula would give 0, which the AM–GM theorem on $x_i > 0$ does not discuss). HM is undefined (1/0).

7.2

Fails: $39 + 28 + 35 + 31 = 133$. Remaining sum $1905 - 133 = 1772$ on $n = 26$:

$$\frac{1772}{26} \;=\; \frac{68 \cdot 26 + 4}{26} \;=\; 68 + \frac{2}{13} \approx 68.154.$$

Move: $1772/26 - 63.5 = 121/26 \approx 4.65$ marks. The class mean moved a few points on a pile whose range is 71 and whose SD is ≈ 19.6. The four fails are real, but they are not kidnapping 63.5 the way 99 kidnapped the four-score mean. The left tail tugs; it does not own the pin.

7.3

S09, S02, S08: marks $28, 45, 70$. Unweighted

$$\frac{28 + 45 + 70}{3} \;=\; \frac{143}{3} \approx 47.67.$$

Weighted (given 58.3):

$$\frac{1 \cdot 28 + 3 \cdot 45 + 6 \cdot 70}{10} \;=\; \frac{583}{10} = 58.3.$$

"Typical marks" is the legal sentence for 143/3 (each student one vote). The weights licensed "mean of marks with hours as mass", not "marks per hour of study, averaged". Marks-per-hour would be the three rates 28/1, 15, 70/6 and then a mean of rates — a different number (≈ 18.2).

C.9 Chapter 8 — median and mode

8.1

Odd $\{28, 31, 35, 39, 41\}$, $n = 5$: index $(n + 1)/2 = 3$, median $x_{(3)} = 35$. Even $\{28, 31, 35, 39\}$, $n = 4$: average $x_{(2)}$ and $x_{(3)}$, $(31 + 35)/2 = 33$. Thirty marks, even $n = 30$: average $x_{(15)}$ and $x_{(16)}$, which are 62 and 66, so $(62 + 66)/2 = 64$.

8.2

Ahmedabad 9, Rajkot 6. Two new Rajkot students: Rajkot 8, Ahmedabad still 9. The city mode remains Ahmedabad (still unique). The mean of marks cannot be discussed: you were not given the two new marks, and a mode of a nominal column does not constrain a ratio mean.

8.3

White lie: you have argued the Likert-as-interval convention (equal gaps by fiat) and you report $3.4\overline{6}$ as a modelling choice, labelled as such. Crime: you treat the codes as ratio, or you say "the class agrees 3.47 times as much". Licensed centre for an ordinal five-point item: the median (and the mode). The table does not license a mean until you change the scale by argument.

C.10 Chapter 9 — dispersion

9.1

Ten board numbers: $20 - 2 = 18$. Thirty marks: $99 - 28 = 71$. "The class spans 71 marks" describes two people (S09 and S22). A typical gap between two classmates is closer to the SD (≈ 19.6) or the IQR ($76.75 - 47.75 = 29$). Range is a first glance, hostage to the ends.

9.2

$\{2, 3, 5, 6, 8\}$. Sum 24, $\bar{x} = 24/5 = 4.8$.

x_i	$x_i - 4.8$	$(x_i - 4.8)^2$
2	−2.8	7.84
3	−1.8	3.24
5	0.2	0.04
6	1.2	1.44
8	3.2	10.24
		SS = 22.8

Population: $22.8/5 = 4.56$, SD $\sqrt{4.56} \approx 2.135$. Sample: $22.8/4 = 5.7$, SD $\sqrt{5.7} \approx 2.387$. `numpy.var` without `ddof` uses `ddof=0`, matching 4.56.

9.3

Pin codes stored as integers: you can subtract the largest from the smallest and get a large integer. That subtraction is the measurement-scale crime — zip is nominal, a

region name in digits. Legal summary of that column: mode-count (how many people share a code). Range, s, and IQR all require an honest quantity or at least honest order-with-gaps; they are not legal.

C.11 Chapter 10 — percentiles

10.1

8 sits at $i = 5$ in $2, 3, 5, 6, 8, 10, 12, 15, 18, 20$:

$$P(8) = \frac{5-1}{10-1} \times 100 = \frac{4}{9} \times 100 = 44.\overline{4}\%.$$

Invert $P = 40$: $i = 1 + 0.40 \times 9 = 4.6$. Seat 4.6 is 0.6 of the way from $x_{(4)} = 6$ to $x_{(5)} = 8$:

$$P_{40} = 6 + 0.6 \cdot (8 - 6) = 7.2.$$

10.2

Classmate "70" used the alternative i/n: $7/10 \times 100 = 70\%$. You used $(i-1)/(n-1)$: $6/9 \times 100 = 67\%$. Same question (percentile *rank* of the value 12), two recipes. `np.quantile(x, 0.67) == 12.09` answers a different question: it returns the *value* at probability 0.67, by interpolating between 12 and 15, not the rank of the number 12. You and the 70%-classmate were ranking 12; numpy was inverting a probability.

10.3

Class board inversion $i = 1 + p(n-1)$ is Hyndman–Fan **type 7** (numpy / pandas default). Q_3 at $p = 0.75$:

$$i = 1 + 0.75 \times 9 = 7.75, \qquad Q_3 = 12 + 0.75 \cdot (15 - 12) = 14.25.$$

Type 6 (Weibull, plotting position $p(n+1)$): $i = 0.75 \times 11 = 8.25$,

$$Q_3 = 15 + 0.25 \cdot (18 - 15) = 15.75.$$

Two labs, both "correct" inside their recipe, print 14.25 and 15.75 because they interpolated different seats on the same ten numbers. Name the type.

C.12 Chapter 11 — shape

11.1

$\{17, 41, 30, 99\}$, sorted $17, 30, 41, 99$. Mean 46.75, median 35.5. No unique mode (four distinct values). Sample s: SS $= 3928.75$, $s = \sqrt{3928.75/3} \approx 36.188$. Pearson's median skewness

$$\frac{3(46.75 - 35.5)}{36.188} = \frac{33.75}{36.188} \approx 0.93.$$

Positive: the pile leans right. The 99 is doing the leaning (three scores at or below 41, one in the far tail).

11.2

`df["marks"].skew()`≈ -0.075, `kurt()`≈ -0.943 (Fisher excess). Near-zero skew says the pile is almost a mirror (tent, uniform, and bell can all be symmetric). Excess ≈ -0.94 is platykurtic versus a Gaussian (excess 0) but not as flat as a uniform (excess -1.2 on the five-point toy). Combined with the histogram — one mound, no rectangle, no sharp tent — the pile is a slightly flattened **bell**, not a tent and not a uniform. The four fails give the hair of left skew; they do not make a second peak.

11.3

$\{1, 2, 3, 4, 5\}$, $\bar{x} = 3$. Sum of squares 10, sum of fourth powers 34.

$$m_2 = \frac{10}{5} = 2, \qquad m_4 = \frac{34}{5} = 6.8, \qquad \frac{m_4}{m_2^2} = \frac{6.8}{4} = 1.7.$$

Pandas Fisher excess is -1.2 (bias-corrected, 3 already subtracted). A textbook that says "a Gaussian has kurtosis 3" is speaking the moment language: quote 1.7 next to that 3. Quote pandas -1.2 next to "Gaussian excess is 0". Mixing 1.7 with "excess 0" invents a crisis of 3.

C.13 Chapter 12 — association

12.1

Rising line: $\bar{x} = 2.5$, $\bar{y} = 5$. Products $4.5, 0.5, 0.5, 4.5$ sum to 10. $\mathrm{SS}_x = 5$, $\mathrm{SS}_y = 20$,

$$r = \frac{10}{\sqrt{5 \cdot 20}} = \frac{10}{10} = 1.$$

Parabola: $\bar{x} = 0$, $\bar{y} = 2$. Products $-4, 1, 0, -1, 4$ sum to 0, so $r = 0$. What r missed: $y = x^2$ is a perfect curve. A scatter shows the U; Pearson only asks for a straight tilt and reports "none".

12.2

Ranks of x: $1, 2, 3, 4$. Ranks of y: $4, 3, 2, 1$. $d = (-3, -1, 1, 3)$, $\sum d_i^2 = 20$. No ties:

$$r_s = 1 - \frac{6 \cdot 20}{4 \cdot 15} = 1 - \frac{120}{60} = -1.$$

`ss.spearmanr` returns -1. $|r_s| = 1$ because the pairing is a perfect inverse order. Spearman is Pearson-on-ranks, so a strictly monotone cloud becomes a straight line in rank space. On *this* falling line Pearson is also -1, so the neighbour is not wrong about this four-point file. Spearman's extra power is a monotone *curve* (a log, a reciprocal) where $|r| < 1$ and $|r_s| = 1$.

12.3

Observed: F 13 yes / 2 no; M 13 yes / 2 no. $n = 30$, row totals $15, 15$, column totals $26, 4$.

$$E_{\mathrm{F,yes}} = \frac{15 \cdot 26}{30} = 13, \quad E_{\mathrm{F,no}} = \frac{15 \cdot 4}{30} = 2,$$

and the M row is identical. Every $O = E$, so $\chi^2 = 0$ and $\phi = \sqrt{\chi^2/n} = 0$. "Balanced on purpose" means the invented file has no gender–pass story, so a nonzero ϕ would be a bug in the author, not a discovery about Gujarat. If ϕ had been far from zero on this file, you would accuse the author of smuggling a gender effect into a teaching cartoon that claimed none.

C.14 Chapter 13 — sampling

13.1

All six SRS samples of size 2 from $\{1, 2, 3, 4\}$:

$$\{1,2\},\ \{1,3\},\ \{1,4\},\ \{2,3\},\ \{2,4\},\ \{3,4\}.$$

Each equally likely, so $P(\{2,4\}) = 1/6$. Pairs containing 4: $\{1,4\}, \{2,4\}, \{3,4\}$ — three of six — so $P(4 \in \text{sample}) = 3/6 = 1/2$. The n/N formula: $2/4 = 1/2$. Same number.

13.2

Cluster means: H1 = 71, H2 = 41, H3 = 89 (hostel sums 284, 164, 356 each over 4). Stratified $\{\text{T04}, \text{T06}, \text{T12}\} = \{74, 42, 86\}$:

$$\bar{x} = \frac{74 + 42 + 86}{3} = \frac{202}{3} \approx 67.33.$$

This cannot equal 41 because 41 is the all-H2 cluster mean. A stratified draw of one from each hostel always mixes a West $\sim$ 71, an East $\sim$ 41, and a North $\sim$ 89. An all-East sample is impossible under that design.

13.3

63.5 is a **statistic** (Roman $\bar{x}$) if the thirty are an SRS from $N = 120$. The number of equally likely samples is

$$\binom{120}{30} = \frac{120!}{30!\,90!}.$$

If those thirty rows *were* the whole batch, $n = N$ and 63.5 would be a **parameter** (μ of that batch). Same arithmetic, different job.

C.15 Chapter 14 — experiments

14.1

Experimental unit: one student (each independently receives a snack). Factor: which snack. Levels: A and B. Response: Likert sleepiness 1–5 two hours later. Scale before any interval argument: **ordinal**.

14.2

Randomization: a chance process assigns A or B to each of the 40 (a shuffle, not "who asked first"). Replication: forty students, not one anecdote; both levels appear more than once. Control: B is the comparison arm for A (a third "no snack" arm would be an even clearer control; two treatments still beat an uncontrolled before/after). If students *choose* A or B, the missing R is **randomization**. You still have replication and a comparison, but the gym-goers can all pick A.

14.3

(a) Convenience / voluntary sample of one hostel floor: Ahmedabad and already- passed students are over-represented, so the target "all DA-IICT first-years" is not the costume you put on. (b) "Are there duplicate roll numbers?" is IDA (usable / keys). "Do hours and marks move together?" is EDA (pattern).

C.16 Chapter 15 — the import lab

15.1

Target if "did they pass?": `passed`. `df.shape`=(30, 8). NOIR: `student_id` nominal; `marks` ratio; `age` ratio; `gender` nominal; `likert` ordinal; `city` nominal; `hours_studied` ratio; `passed` nominal (binary). Duration: `hours_studied` (hours of study, not a clock). No datetime meaning at all: every column — including `age` (years lived, not a timestamp) and `marks`. There is no datetime field in this file.

15.2

Marks min 28, max 99, feasible inside $[0, 100]$. Age min 18, max 23, feasible inside $[0, 120]$. Missing cells: 0. Tukey on marks: $Q_1 = 47.75$, $Q_3 = 76.75$, IQR $= 29$,

$$47.75 - 1.5 \times 29 = 4.25, \qquad 76.75 + 1.5 \times 29 = 120.25.$$

S09's 28 sits inside $(4.25, 120.25)$. It is a real fail, not a fence-outlier. Do not delete it.

15.3

EDA report — `it462_dirty.csv` (IDA dominates).
File. Five rows, four columns (ID, IT462, Age, Gender). Teaching table from Chapter 0. Not a class.
Question / target. There is no `passed` column. If the question is "did they pass?", you cannot answer it yet: IT462 is not a usable mark column.
Usable? No. ID `2026.` has a trailing period; two further IDs are the digits 2026; one ID is blank; one ID is 2023. Machine-naive import collapses the period and types ID as float. Duplicate-ish keys. IT462 includes −20 (infeasible as a 0–100 mark). Ages 0, 10, 20, 100, and missing. Gender missing on the last row.
Refuse. Do not report $\overline{\text{IT462}} = (10 + 50 + 90 - 20 + 17)/5 = 29.4$. That mean swallows a sentinel. Do not clip −20 to 0 inside the plotting cell and call it EDA.
Univariate (only as alarms). Min IT462 $= -20$, max 90. Min Age 0, max 100. Missing: one ID, one Age, one Gender.
Scope / sampling. Five rows are not a population. Granularity (one row = ?) is

unresolved while IDs collide.
Not yet. No model, no imputed age, no dropped-and-forgotten $n = 4$. Next step is a codebook, then a return to EDA on the cells the codebook called marks.

C.17 Missing data: mechanisms (Chapter 19)

18.1

(a) **MAR**: weight is missing depending on age, which is recorded. (b) **MNAR**: income is missing because of the income value itself. (c) **MCAR**: lost pages are a random subset, unrelated to any value. (d) **MAR**: missingness depends on shift, an observed column (assuming, within each shift, it does not depend on the hidden sleep value).

18.2

Observed scores $[40, 48, 55, 60, 66, 71]$, mean $340/6 = 56.67$ (truth 62.75, biased *down*). "Top students did not bother" $\Rightarrow$ missing because of the score itself: **MNAR**. "The 7+ hours lab lost its sheets" $\Rightarrow$ missing because of the recorded hours: **MAR**. Same holes, same table; only the story decides.

18.3

`pd.read_csv(path, na_values=[999, -1, "NA"])`. Remaining ages $21, 23, 22$, mean $= 66/3 = 22$. (Without the fix, pandas would read the column as text because of `"NA"`-like junk, or, if numeric, average 999 and -1 into the result.)

C.18 Missing data: detection (Chapter 20)

19.1

Patterns over (A, B, C): `"000"` rows 0, 4, 5 (3 rows); `"010"` row 1; `"100"` row 2; `"001"` row 3. Four patterns. Observed columns per pattern: $3, 2, 2, 2$, so df $= \sum p_j - p = (3 + 2 + 2 + 2) - 3 = 6$.

19.2

H_0: the data are MCAR; H_1: not MCAR; $\alpha = 0.05$. $\chi^2_0 = 11.2 > 9.49 \Rightarrow$ **reject** H_0: the missingness is related to the data (MAR or MNAR). At $\alpha = 0.01$: $11.2 < 13.28 \Rightarrow$ fail to reject. The decision depends on how strict you are; report the p-value (≈ 0.024) rather than only the verdict.

19.3

Missing group hours $[7, 8]$: mean 7.5, variance 0.5. Observed group $[1, \ldots, 6]$: mean 3.5, variance 3.5. $t = (7.5 - 3.5)/\sqrt{0.5/2 + 3.5/6} = 4/0.9129 = 4.38$, $p \approx 0.006$: reject MCAR. Same size as before, opposite sign.

C.19 Missing data: treatment (Chapter 21)

20.1

$P(\text{row complete}) = 0.97^{12} = 0.694$, so about 30.6% of rows are lost. That is far above the 5–10% guideline, so do not delete; use pairwise statistics or impute.

20.2

hours $3 \to 34.83 + 6.187 \times 3 = 53.39$; hours $6 \to 34.83 + 6.187 \times 6 = 71.95$ (truth 55 and 71). Imputed mean $= (40+48+53.39+60+66+71.95+78+84)/8 = 501.34/8 = 62.67$.

20.3

MNAR: missing because the income is high. The observed incomes are mostly the lower ones, so their median is too low, and filling 30% of rows with that low median drags the average down further. Better: (i) ask for an income *band* instead of an exact number, or use other evidence (job, area) in a model of the missingness; (ii) add a "was missing" indicator and run a sensitivity analysis (e.g. assume missing incomes are 20%, 50% higher and see if the conclusion survives).

C.20 Outliers (Chapter 22)

21.1

$Q_1 = 4.25$, $Q_3 = 6.0$, IQR $= 1.75$, fences $4.25 - 2.625 = 1.625$ and $6 + 2.625 = 8.625$. $30 > 8.625$: **IQR flags it**. Mean 7.5, population SD 7.58, $z_{30} = (30-7.5)/7.58 = 2.97 < 3$: **z does not** (masking; and with $n = 10$, $|z| \leq 3$ always).

21.2

Winsorized data $[3, 4, 4, 5, 5, 5, 6, 6, 7, 7]$, mean 5.2. Original mean 7.5. 10% trimmed mean (drop 3 and 30) $= 42/8 = 5.25$. Both robust versions agree; the raw mean is pulled up by the single 30.

21.3

LOF compares a point's local density with the density around its nearest neighbours. A point in the empty gap between two dense clusters has neighbours that sit in crowded regions while it does not, so its LOF is much greater than 1, even if it happens to lie near the overall mean, which a global distance (z or Mahalanobis) would call "typical".

C.21 Transformation I (Chapter 23)

22.1

$\bar{x} = 24$, $\sigma = 38.05$; min 2, max 100; median 6, $Q_1 = 4$, $Q_3 = 8$, IQR $= 4$.

x	z	min–max	robust
2	−0.578	0.000	−1.0
4	−0.526	0.020	−0.5
6	−0.473	0.041	0.0
8	−0.420	0.061	0.5
100	1.997	1.000	23.5

Robust scaling keeps the first four values best separated (steps of 0.5 vs 0.05 and 0.02).

22.2

$\lambda = -1$: $(4^{-1} - 1)/(-1) = 0.75$; $\lambda = 0$: $\log 4 = 1.386$; $\lambda = 0.5$: $(2 - 1)/0.5 = 2$; $\lambda = 1$: 3; $\lambda = 2$: $(16 - 1)/2 = 7.5$.

22.3

Ranks of $[10, 40, 20, 30]$ are $[1, 4, 2, 3]$, uniform $(r - 1)/3 = [0, 1, 0.333, 0.667]$. Normal: Φ^{-1} gives $[-\infty, +\infty, -0.431, 0.431]$. The two ends are infinite because $\Phi^{-1}(0)$ and $\Phi^{-1}(1)$ do not exist, so the library clips them (to about ± 5.2).

C.22 Transformation II (Chapter 24)

23.1

Equal-width ($w = 58/3 = 19.33$, edges 35, 54.33, 73.67, 93): $\{35, 42, 48\}$, $\{55, 61, 67, 72\}$, $\{78, 85, 93\}$. Equal-frequency (`pd.qcut` edges 35, 55, 72, 93): $\{35, 42, 48, 55\}$, $\{61, 67, 72\}$, $\{78, 85, 93\}$. Ten points cannot split exactly into three equal bins, so the counts are $4, 3, 3$.

23.2

Ordinal: S 0, M 1, L 2, XL 3. One-hot with `pd.get_dummies(drop_first=True)`: pandas sorts the labels alphabetically (L, M, S, XL) and drops the first, so **L** is the reference. Columns (M, S, XL): S $(0, 1, 0)$, M $(1, 0, 0)$, L $(0, 0, 0)$, XL $(0, 0, 1)$. To make S the reference instead, set the column to an ordered `Categorical` first. Binary: S 00, M 01, L 10, XL 11.

23.3

$m = 4$, global 0.625: Ahmedabad $(1 + 2.5)/6 = 0.583$; Rajkot $(1 + 2.5)/5 = 0.700$; Surat $(1 + 2.5)/7 = 0.500$; Vadodara $(2 + 2.5)/6 = 0.750$. Rajkot moved most ($1.0 \to 0.70$) because it has only one student, so the prior weight $m = 4$ dominates its single observation.

Closing: The Whole Story in One Page

Raw stuff is **data**. A number you can act on is **information**. IDA asks "is this usable?" EDA asks "what is the shape of this usable thing?" CDA asks "does my model survive a fair test?"

Columns are features. Rows are observations. Every feature has a **measurement scale** (NOIR). The scale decides which operations, which plots, and which association coefficient are legal. Central tendency is a pin in the map; dispersion is how spread the pins are; skewness and kurtosis are the *shape* of the pile; association is how two piles move together. A sample statistic is a noisy photograph of a population parameter. Randomization, replication, and control are how you stop the photograph from being a selfie of your own bias.

After `pd.read_csv`, you do not model. You walk the L10 checklist on `it462_marks.csv`: shape, types, names, units, time, duplicates, missingness, feasible ranges, outliers. Then you look. Then — and only then — you encode, engineer, and confirm.

The studios in this book are not decoration. If you can do them without peeking at Chapter 99, you have finished IT462's EDA half.

Annotated bibliography (what we actually opened)

All of the following were downloaded into `references/` (gitignored) and *read for ideas*. Nothing is copy-pasted. Pedagogy — one idea, one picture, one hand calculation — is borrowed; the numbers and the sentences are ours.

1. UC Berkeley Data 8, *materials-sp26* (CC BY-NC) and *Computational and Inferential Thinking*. Tables before formulas; one running dataset.
2. UC Berkeley Data 100 course notes, "Data Cleaning and EDA" (BSD-3-Clause). Structure / granularity / scope / temporality / faithfulness — rewritten here in IT462 words (validity, reliability, authenticity, error-free).
3. Allen B. Downey, *Think Stats* 3e notebooks (Green Tea Press, CC). Process order: import → univariate → pairwise. From-scratch then library.
4. OpenIntro Statistics (CC). Density: definition, figure, example, guided exercise.
5. Stanford STATS 60 public pages. Shape (variability, concentration, modality) as its own unit, not a footnote on the mean.
6. Hadley Wickham, "Tidy Data", *J. Statistical Software* 59 (2014).
7. F. J. Anscombe, "Graphs in Statistical Analysis", *The American Statistician* (1973).
8. Hyndman & Fan, "Sample quantiles in statistical packages", *The American Statistician* 50 (1996) — the nine recipes pandas and numpy still argue about.
9. Gareth James et al., *An Introduction to Statistical Learning* (ISLR PDF). What happens *after* EDA. Not a substitute for looking.
10. Polars, DuckDB, ydata-profiling, DA-Studio / DataClawBench / Walk Before You Run (2026 arXiv). Optional appendix only — IT462 has not asked for a lakehouse.